המכללה האקדמית להנדסה סמי שמעון
המחלקה להנדסת תעשיה וניהול

פרויקט גמר

בנושא:

SuperWaze

מנוע השוואת מחירים חכם עם חיזוי תורים לסופרמרקטים בישראל

הוגש על ידי:
עדן כהן
חובב שוקורון

בהנחיית:
ד"ר רונית שמלו

מרץ 2026

תודות

ברצוננו להביע את תודתנו העמוקה לכל מי שסייע ותמך בנו לאורך הדרך בפרויקט זה.

תודה מיוחדת למנחה שלנו, ד"ר רונית שמלו, על ההנחיה המקצועית, הסבלנות והזמן שהקדישה לנו. הידע והניסיון שלה היוו עבורנו מצפן לאורך כל שלבי הפרויקט.

תודה למשפחותינו ולחברינו על התמיכה הבלתי מתפשרת, ההבנה והעידוד בתקופות המאתגרות של העבודה על הפרויקט.

תודה לצוות המחלקה להנדסת תעשיה וניהול במכללה האקדמית להנדסה סמי שמעון על הכלים והידע שהעניקו לנו במהלך שנות הלימוד.

עדן כהן וחובב שוקורון

מרץ 2026

תקציר

בשנים האחרונות חלה עלייה משמעותית ביוקר המחיה בישראל, ובמיוחד במחירי מוצרי המזון. למרות חוק שקיפות המחירים שנכנס לתוקף בשנת 2015, המחייב את רשתות השיווק לפרסם את מחירי כל מוצריהן באופן מקוון, הצרכן הישראלי מתקשה לנצל מידע זה באופן אפקטיבי. הנתונים מפוזרים על פני עשרות אתרים בפורמטים שונים, וחסר כלי מרכזי שמאפשר השוואה נוחה ואינטליגנטית.

פרויקט **SuperWaze** מציע פתרון טכנולוגי מקיף לבעיה זו. המערכת מהווה מנוע השוואת מחירים חכם המשלב חיזוי זמני המתנה בתורים, מנוע מבצעים עם תמיכה בכפל הנחות (Promo Stacking), חישוב עלות דלק ונסיעה, וניווט חכם בתוך החנות. המערכת מאפשרת לצרכן לבנות רשימת קניות, להשוות מחירים בין רשתות שיווק מרובות, ולקבל המלצה אופטימלית לחנות הכדאית ביותר — תוך שקלול עלות הסל (לאחר מבצעים), עלות הנסיעה וזמן ההמתנה הצפוי בקופות.

חידוש מרכזי במערכת הוא מנגנון זיהוי קופות אוטומטי: האפליקציה הניידת מזהה עמידה ממושכת באמצעות חיישן תאוצה (Accelerometer) ושיקול Bayesian Prior, ודיווחי המשתמשים מצטברים לזיהוי מיקומי קופות פיזיים באמצעות אלגוריתם K-Means עם בחירת K אופטימלי על בסיס BIC. על בסיס הקופות המזוהות, המערכת מחשבת תחזית זמן המתנה לכל סוג קופה (רגילה, מהירה, שירות עצמי) באמצעות מודל M/M/c מתורת התורים, בשילוב מודל EWMA לחיזוי קהילתי.

המערכת נבנתה בארכיטקטורה מודרנית הכוללת שרת ASP.NET Core 8 עם בסיס נתונים PostgreSQL, ממשק ווב מבוסס Next.js עם ניהול מצב ב-Zustand, ואפליקציית מובייל ב- React Native (Expo). עדכון המחירים והמבצעים מתבצע באופן אוטומטי באמצעות צינורות ETL שמייבאים נתונים ממאגרי המידע הפתוחים של רשתות השיווק מכוח חוק שקיפות המחירים.

הפרויקט כולל מנגנון קהילתי שבו משתמשים מדווחים על זמני המתנה בתורים באמצעות זיהוי אוטומטי של מיקום (Geofencing), מעקב ביקורים בחנויות, מערכת ניווט פנים-חנות מבוססת אזורים, ותמיכה בסוגי רכב שונים (בנזין, חשמלי, היברידי, הולך רגל) לחישוב עלות הנסיעה.

מילות מפתח: השוואת מחירים, סופרמרקטים, חיזוי תורים, מנוע מבצעים, EWMA, M/M/c, K-Means Clustering, חוק שקיפות המחירים, Geofencing, ASP.NET Core, React Native, ETL

תוכן עניינים

פרק 1: מבוא

1.1 רקע כללי

יוקר המחיה בישראל מהווה נושא מרכזי בשיח הציבורי והכלכלי כבר למעלה מעשור. על פי נתוני הלשכה המרכזית לסטטיסטיקה, מחירי המזון בישראל גבוהים בכ-19% מהממוצע במדינות ה-OECD. עלויות אלו פוגעות באופן ישיר בתקציב משק הבית הישראלי, כאשר משפחה ממוצעת מוציאה כ-17% מהכנסתה על מזון.

בשנת 2015 נכנס לתוקף חוק שקיפות המחירים, המחייב את כל רשתות השיווק הגדולות בישראל לפרסם באופן מקוון את מחירי כלל המוצרים שלהן בפורמט XML. בנוסף למחירים, החוק מחייב את הרשתות לפרסם גם נתוני מבצעים — כולל סוג המבצע, מחיר מוזל, תנאי סף וטווחי תאריכים. בפועל, היכולת של הצרכן הממוצע לנצל מידע זה היא מוגבלת. הנתונים מפורסמים באתרים שונים, בפורמטים טכניים, וללא כלי ריכוזי נוח שמאפשר השוואה אינטואיטיבית.

בנוסף לבעיית המחירים, הצרכן הישראלי מתמודד עם אתגרים נוספים: זמני המתנה ארוכים בקופות הסופרמרקטים, חוסר מידע על סוגי הקופות הפעילות (רגילה, מהירה, שירות עצמי), וקושי להעריך את זמן ההמתנה בכל אחת מהן. מחקרים מראים כי זמן ההמתנה הממוצע בתור בסופרמרקט בישראל עומד על כ-12 דקות בשעות העומס, כאשר חוויית ההמתנה משפיעה משמעותית על שביעות רצון הלקוח ועל הנכונות לחזור לחנות. שילוב מידע על מחירים ומבצעים עם תחזית זמני המתנה מפורטת לכל סוג קופה יכול לספק לצרכן תמונה שלמה של **"העלות האמיתית"** של הקנייה — הן במונחים כספיים והן במונחי זמן.

1.2 הגדרת הבעיה

כיום קיימים מספר אתגרים מרכזיים בשוק הקמעונאות בישראל מנקודת מבטו של הצרכן:

1. פיזור מידע: מחירי המוצרים מפורסמים באתרים נפרדים לכל רשת, בפורמטים שונים, מה שמקשה על השוואה.

2. התעלמות ממבצעים: אפליקציות קיימות משוות מחירי מדף בלבד ואינן מביאות בחשבון מבצעים, הנחות מועדון וכפל מבצעים — מה שעלול להוביל להמלצה שגויה.

3. חוסר שקיפות בזמני המתנה: אין כלי שמספק לצרכן מידע מפורט או תחזית מבוססת נתונים על תורים בקופות, ובפרט אין הבחנה בין סוגי קופות (רגילה, מהירה, שירות עצמי).

4. התעלמות מעלות הנסיעה: אפליקציות קיימות להשוואת מחירים אינן מביאות בחשבון את עלות ההגעה לחנות (דלק, זמן נסיעה) ואינן תומכות בסוגי רכב שונים.

5. היעדר אופטימיזציה כוללת: לא קיים כלי שמשקלל את כל מרכיבי העלות — מחיר הסל (אחרי מבצעים), עלות הנסיעה וזמן ההמתנה — לכדי המלצה אחת מיטבית.

6. ניווט בתוך החנות: מציאת מוצרים בסופרמרקט גדול יכולה להיות מתסכלת ולגזול זמן יקר.

7. חוסר ground truth: אין מנגנון אוטומטי לאיסוף נתוני אמת על מיקומי קופות ועומסים — התלות בדיווח ידני מגבילה את דיוק החיזויים.

1.3 מטרות הפרויקט

המטרה המרכזית של פרויקט **SuperWaze** היא פיתוח מערכת מידע חכמה שמסייעת לצרכן הישראלי לקבל החלטות קנייה מושכלות באמצעות שילוב מידע ממגוון מקורות ואלגוריתמים מתקדמים. להלן מטרות הפרויקט:

1. בניית מנוע השוואת מחירים רב-רשתי המבוסס על נתוני חוק שקיפות המחירים, עם עדכון יומי אוטומטי של מחירים ומבצעים.

2. פיתוח מנוע מבצעים (Promo Engine) התומך בחמישה סוגי מבצע (הורדת מחיר, אחוז הנחה, קנה-קבל, רב-קנה, אחר) עם תמיכה בכפל מבצעים (Stacking) — מבצעים בלעדיים לעומת מבצעים נוספים.

3. פיתוח מודל חיזוי תורים דו-שכבתי: מודל EWMA קהילתי לחיזוי כללי, ומודל M/M/c מתורת התורים לחיזוי מפורט לכל סוג קופה.

4. מימוש מנגנון זיהוי קופות אוטומטי: זיהוי עמידה ממושכת באמצעות חיישן תאוצה (StandingDetector), צבירת דיווחים ואשכול מיקומים באמצעות K-Means עם בחירת K אופטימלי על בסיס BIC.

5. חישוב עלות נסיעה מדויקת לכל חנות תוך התחשבות בסוג הרכב (בנזין, חשמלי, היברידי, הולך רגל), מרחק ותנאי תנועה.

6. פיתוח אלגוריתם אופטימיזציה ששוקל את כל מרכיבי העלות — מחיר סל לאחר מבצעים, עלות נסיעה וזמן המתנה — ומספק לצרכן המלצה מיטבית.

7. מימוש מערכת ניווט פנים-חנות מבוססת אזורים לשיפור חוויית הקנייה.

8. בניית מנגנון קהילתי חכם לאיסוף דיווחי תורים עם זיהוי מיקום אוטומטי (Geofencing) ומעקב ביקורים בחנויות.

9. תכנון וביצוע המערכת בארכיטקטורה מודרנית, סקלבילית ומאובטחת, עם CI/CD אוטומטי.

1.4 מתודולוגיית הפיתוח

הפרויקט פותח בגישת **Agile** מבוססת **Scrum**, הכוללת ספרינטים (מחזורי פיתוח) של שבועיים כל אחד. בכל ספרינט הוגדרו משימות ברורות, בוצע פיתוח, נערכו בדיקות ונאספה משוב. גישה זו אפשרה גמישות בתכנון, התמודדות מהירה עם שינויים ושיפור מתמיד של המוצר.

שלבי הפיתוח כללו:

1. מחקר ראשוני: סקירת ספרות, ניתוח מתחרים, והגדרת דרישות ראשוניות.

2. אפיון ועיצוב: הגדרת מקרי שימוש, עיצוב ארכיטקטורת המערכת, תכנון בסיס הנתונים ועיצוב ממשק המשתמש.

3. פיתוח הליבה: בניית שרת ה-API, מודל חיזוי התורים, מנוע ההשוואה, מנוע המבצעים ומנגנוני ה-ETL.

4. פיתוח מנגנוני חיזוי מתקדמים: מימוש זיהוי קופות (GMM Clustering), חיזוי M/M/c לכל סוג קופה, וזיהוי עמידה בקופה (StandingDetector).

5. פיתוח ממשקים: בניית ממשק הווב (Next.js + Zustand) ואפליקציית המובייל (React Native/Expo).

6. בדיקות ואינטגרציה: בדיקות יחידה, בדיקות אינטגרציה ובדיקות קצה-לקצה, עם CI/CD אוטומטי ב-GitHub Actions.

7. הטמעה ומשוב: פריסה בסביבת ייצור באמצעות Docker Compose + nginx, ואיסוף משוב ממשתמשי בטא.

1.5 הטכנולוגיות העיקריות

להלן סקירה תמציתית של הטכנולוגיות המרכזיות שנבחרו לפרויקט:

רכיב	טכנולוגיה	תיאור
צד שרת	#ASP.NET Core 8, C	שרת API מבוסס REST עם ארכיטקטורת שכבות
בסיס נתונים	PostgreSQL	בסיס נתונים רלציוני עם תמיכה ב-JSON ו-10 מיגרציות
מטמון	Redis	שכבת מטמון לשיפור ביצועים (עם fallback לזיכרון)
ממשק ווב	Next.js + Zustand	ממשק משתמש מודרני עם ניהול מצב מרכזי
אפליקציית מובייל	React Native (Expo)	אפליקציה חוצת פלטפורמות עם חיישני תאוצה
עדכון נתונים	Python ETL	צינורות ייבוא אוטומטיים (מחירים + מבצעים)
תקשורת בזמן אמת	SignalR	עדכונים חיים למשתמשים על שינויים בתורים
אימות	JWT + HttpOnly Cookies	הזדהות מאובטחת עם חידוש אוטומטי של טוקנים
תשתית	Docker + nginx	הפעלה ופריסה בקונטיינרים עם reverse proxy
CI/CD	GitHub Actions	בדיקות ופריסה אוטומטיים
לוגים	Serilog	מערכת לוגים מובנית עם structured logging

1.6 מושגי מפתח

EWMA (Exponentially Weighted Moving Average) — ממוצע נע משוקלל אקספוננציאלית — שיטה סטטיסטית המעניקה משקל גבוה יותר לתצפיות אחרונות. משמשת בפרויקט לחיזוי זמני המתנה בתורים.

MAD (Median Absolute Deviation) — סטיית מדיאן מוחלטת — מדד סטטיסטי רובסטי לפיזור נתונים, המשמש בפרויקט לסינון דיווחי תורים חריגים.

M/M/c — מודל מתורת התורים המניח הגעות פואסוניות, זמני שירות אקספוננציאליים ו-c שרתים (קופות). משמש בפרויקט לחיזוי זמן המתנה לכל סוג קופה על בסיס מספר הקופות הפעילות וקצב ההגעה.

K-Means Clustering — אלגוריתם אשכול המחלק נקודות נתונים ל-K קבוצות על בסיס מרחק. משמש בפרויקט לזיהוי מיקומי קופות פיזיים מדיווחי checkout.

BIC (Bayesian Information Criterion) — קריטריון סטטיסטי לבחירת מודל אופטימלי. משמש בפרויקט לקביעת מספר הקופות הנכון (K) באופן אוטומטי.

Promo Stacking — כפל מבצעים — מנגנון שבו מבצע נוסף (ADDITIVE) מוחל על גבי מבצע בלעדי (EXCLUSIVE) לחיסכון מקסימלי.

ETL (Extract, Transform, Load) — תהליך של שליפה, עיבוד וטעינה של נתונים — בפרויקט משמש לייבוא מחירים ומבצעים מהמאגרים הפתוחים של רשתות השיווק.

Geofencing — טכנולוגיית גידור גיאוגרפי — זיהוי אוטומטי של כניסה ויציאה של המשתמש מאזור גיאוגרפי מוגדר (כגון חנות).

StandingDetector — רכיב באפליקציה הניידת המזהה עמידה ממושכת באמצעות חיישן תאוצה (Accelerometer) ושיקול Bayesian Prior — לגילוי אוטומטי של שהייה ליד קופה.

חוק שקיפות המחירים — חקיקה ישראלית (2015) המחייבת רשתות שיווק לפרסם מקוון את מחירי כל מוצריהן ומבצעיהן בפורמט XML.

עלות נטו (Net Cost) — העלות הכוללת של הקנייה כפי שמחושבת במערכת: מחיר הסל (לאחר מבצעים) + עלות נסיעה + עלות מוניטרית של זמן ההמתנה.

SignalR — ספריית תקשורת בזמן אמת של מיקרוסופט המאפשרת עדכונים דו-כיווניים בין שרת ללקוח באמצעות WebSockets.

1.7 מבנה העבודה

עבודה זו מחולקת לפרקים הבאים:

פרק 1 — מבוא: הצגת הרקע, הגדרת הבעיה, מטרות הפרויקט, מתודולוגיית הפיתוח ומושגי מפתח.

פרק 2 — סקירת ספרות: סקירה תיאורטית של מערכות מידע, מתודולוגיות פיתוח, תורת התורים, מודלים סטטיסטיים, שירותים מבוססי מיקום וחדשנות טכנולוגית.

פרק 3 — חקר מצב קיים: ניתוח אפליקציות מתחרות בשוק הישראלי, סקר צרכנים והשוואת יכולות.

פרק 4 — אפיון המערכת: הגדרת מטרות, גבולות המערכת, אילוצים, תהליכי מחשב ותרשימי זרימה.

פרק 5 — ניתוח המערכת: דרישות פונקציונליות ולא-פונקציונליות, מקרי שימוש (Use Cases) ותרשים מחלקות לניתוח.

פרק 6 — עיצוב המערכת: תרשים מחלקות לעיצוב, סכמת בסיס נתונים רלציוני ותרשימי רצף.

פרק 7 — מנשק המערכת: הצגת מסכי האפליקציה (מובייל ווב) עם הסברים מפורטים.

פרק 8 — סיכום והמלצות: סיכום התוצאות, מסקנות והמלצות להמשך פיתוח.

פרק 2: סקירת ספרות

פרק זה מציג את הבסיס התיאורטי עליו נשענת מערכת **SuperWaze**. הסקירה מכסה מגוון תחומי ידע הרלוונטיים לפרויקט: מערכות מידע, מתודולוגיות פיתוח תוכנה, ארכיטקטורות תוכנה, מערכות השוואת מחירים וקמעונאות דיגיטלית, מודלים סטטיסטיים לחיזוי, תורת התורים, צינורות עיבוד נתונים (ETL), שירותים מבוססי מיקום, ואינטראקציה אדם-מחשב. לכל נושא מוצג הרקע התיאורטי, ההתפתחויות העיקריות בתחום, והרלוונטיות הישירה לפרויקט.

2.1 מערכות מידע

2.1.1 הגדרה ומהות

מערכת מידע (*Information System*) מוגדרת כמכלול מאורגן של אנשים, חומרה, תוכנה, נתונים ותהליכים הפועלים יחד לאיסוף, עיבוד, אחסון והפצה של מידע בתוך ארגון או עבור קהילת משתמשים. ההגדרה הקלסית של לאודון ולאודון (*Laudon & Laudon, 2020*) מדגישה כי מערכת מידע אינה רק כלי טכנולוגי, אלא מערכת סוציו-טכנית המשלבת ממדים ארגוניים, ניהוליים וטכנולוגיים כאחד.

בהקשר של פרויקט **SuperWaze**, המערכת מהווה מערכת מידע מבוססת רשת (*Web-based Information System*) המשרתת קהילת צרכנים. היא אוספת נתונים ממקורות חיצוניים (מחירי מוצרים, מבצעים, דיווחי תורים), מעבדת אותם באמצעות אלגוריתמים סטטיסטיים ומתמטיים, ומפיקה מהם מידע בעל ערך — המלצות לחנות הכדאית ביותר.

2.1.2 רכיבי מערכת מידע

על פי המודל המקובל בספרות (*Stair & Reynolds, 2021*), מערכת מידע מורכבת מחמישה רכיבים מרכזיים:

רכיב	הגדרה כללית	ביטוי ב-SuperWaze
חומרה	המכשירים הפיזיים שעליהם רצה המערכת	שרת Linux (Docker), מכשירי מובייל (iOS/Android), מחשבי משתמשים
תוכנה	התוכניות המעבדות נתונים ומציגות מידע	ASP.NET Core 8, Next.js, React Native, Python ETL
נתונים	הגלם שממנו מופק מידע	מחירים, מבצעים, מיקומי חנויות, דיווחי תורים, נתוני ביקורים
תהליכים	הכללים והשלבים לעיבוד נתונים	מנוע המלצה, מנוע מבצעים, חיזוי EWMA, M/M/c, ETL
אנשים	המשתמשים המקיימים אינטראקציה עם המערכת	צרכנים (דיווח תורים, בניית רשימות), מפתחים, מנהלי מערכת

2.1.3 סוגי מערכות מידע

מערכות מידע מסווגות על פי מספר קריטריונים. להלן הסיווגים הרלוונטיים ביותר ל-**SuperWaze**:

מערכת תמיכה בהחלטות (*DSS — Decision Support System*): מערכת SuperWaze מסייעת לצרכן לקבל החלטת קנייה מושכלת באמצעות ניתוח נתונים ממספר מקורות. המערכת אינה מקבלת את ההחלטה עבור המשתמש, אלא מציגה לו מידע מעובד (עלות נטו, חיסכון ממבצעים, זמן המתנה צפוי) שעל בסיסו הוא יכול להחליט לאיזו חנות ללכת. זהו מאפיין קלסי של מערכות DSS כפי שהוגדרו על ידי *Turban et al. (2018)*.

מערכת מידע גיאוגרפית (*GIS — Geographic Information System*): המערכת משלבת מרכיבים מרחביים — מיקום חנויות, חישוב מרחקים, זיהוי מיקום המשתמש (GPS), גידור גיאוגרפי (Geofencing) וניווט פנים-חנות. שילוב מימד מרחבי במערכת מידע הוא מאפיין מרכזי של מערכות GIS (*Longley et al., 2015*).

מערכת קהילתית מבוססת *Crowdsourcing*: מנגנון חיזוי התורים מסתמך על דיווחי קהילת המשתמשים — הן ידניים והן אוטומטיים (זיהוי עמידה בקופה). מודל זה, שבו המשתמשים הם גם צרכני המידע וגם ספקיו, מוכר בספרות כ-*Participatory Sensing* (*Burke et al., 2006*) ומאפיין אפליקציות כמו Waze לניווט ו-Yelp לביקורות עסקים.

2.1.4 מחזור חיי מערכת מידע (SDLC)

מחזור חיי פיתוח מערכת מידע (*Systems Development Life Cycle — SDLC*) הוא מסגרת מושגית המתארת את השלבים שעוברת מערכת מידע מרגע זיהוי הצורך ועד להטמעתה ותחזוקתה השוטפת. המודל הקלסי (*Pressman & Maxim, 2020*) כולל את השלבים הבאים:

1. **חקירה וניתוח (*Investigation & Analysis*)**: זיהוי הבעיה, הגדרת דרישות ראשוניות, ניתוח היתכנות כלכלית וטכנית. בפרויקט זה — סקר צרכנים, ניתוח מתחרים, הגדרת תכולה.
2. **אפיון ועיצוב (*Design*)**: תכנון ארכיטקטורת המערכת, עיצוב בסיס הנתונים, הגדרת ממשקים ותכנון אלגוריתמים. בפרויקט — עיצוב סכמת PostgreSQL, תכנון מנוע המבצעים, עיצוב UI.
3. **מימוש (*Implementation*)**: כתיבת הקוד, בניית רכיבי המערכת ואינטגרציה ביניהם. בפרויקט — פיתוח ב-TypeScript, C# ו-Python.
4. **בדיקות (*Testing*)**: אימות שהמערכת עומדת בדרישות ואינה מכילה פגמים. בפרויקט — בדיקות יחידה (PromoEngine, QueueMath, RouteOptimizer), בדיקות אינטגרציה ו-CI אוטומטי.
5. **הטמעה ותחזוקה (*Deployment & Maintenance*)**: פריסת המערכת בסביבת הייצור ותחזוקה שוטפת. בפרויקט — Docker Compose + nginx, GitHub Actions CI/CD, ותהליכי ETL מתוזמנים.

חשוב לציין כי המודל הקלסי (מפל המים) מניח שלבים סדרתיים, אולם בפרויקט זה נבחרה גישת **Agile/Scrum** שבה השלבים מתקיימים באופן איטרטיבי ומקבילי. מתודולוגיה זו תפורט בהרחבה בסעיף 2.2.

2.1.5 ערך מערכות מידע — מנתונים גולמיים לתובנות

אחד המושגים המרכזיים בתחום מערכות המידע הוא ההבחנה בין נתונים (*Data*), מידע (*Information*) וידע (*Knowledge*). נתונים הם עובדות גולמיות ללא הקשר; מידע הוא נתונים מעובדים בעלי משמעות; וידע הוא מידע שעבר הפנמה ומאפשר קבלת החלטות (*Ackoff, 1989*).

במערכת **SuperWaze** ניתן לראות היררכיה זו בבירור:

רמה	דוגמה מהמערכת
נתונים (*Data*)	קובץ XML ממאגר רשת שופרסל: ברקוד 7290000123, מחיר 7.50, תאריך 10/03/2026
מידע (*Information*)	"חלב תנובה 3% עולה 7.50₪ בשופרסל אשדוד, 6.90₪ ברמי לוי, ויש מבצע 3 ב-18₪ ביינות ביתן"
ידע (*Knowledge*)	"עבור רשימת הקניות שלך, רמי לוי אשדוד הוא הכדאי ביותר — חוסך 47₪ כולל מבצעים, עם 8 דק' נסיעה ו-4 דק' תור"

הערך המוסף העיקרי של **SuperWaze** הוא בהפיכת נתונים גולמיים ומפוזרים (קבצי מחירים, מבצעים, דיווחי תורים) לתובנה מעשית — המלצה ברורה שחוסכת לצרכן כסף וזמן.

2.1.6 מערכות מידע מבוססות רשת

המהפכה הדיגיטלית והתפשטות האינטרנט הובילו למעבר ממערכות מידע שולחניות למערכות מבוססות רשת (*Web-based IS*). מערכות אלו מתאפיינות בנגישות מכל מכשיר עם דפדפן, עדכון מרכזי ללא צורך בהתקנה, ויכולת שירות מספר גדול של משתמשים במקביל (*Valacich & Schneider, 2022*).

SuperWaze מיישמת ארכיטקטורה מבוססת רשת מלאה:

1. ממשק ווב מבוסס **Next.js**: נגיש מכל דפדפן מודרני, עם Server-Side Rendering לביצועים וטעינה מהירים.
2. אפליקציית מובייל מבוססת **React Native (Expo)**: חוצת פלטפורמות, עם גישה לחיישנים שאינם זמינים בדפדפן (GPS, Accelerometer).
3. **REST API** מרכזי: כל הלוגיקה העסקית רצה בשרת אחד, וכל הלקוחות (ווב, מובייל) מתקשרים דרך ממשק API אחיד.
4. תקשורת בזמן אמת באמצעות **SignalR**: עדכונים חיים על שינויים בתורים מגיעים למשתמשים ללא צורך ברענון.
5. פריסה באמצעות **Docker Compose**: כל רכיבי המערכת (API, DB, Frontend, Redis) ארוזים בקונטיינרים לפריסה מהירה ואחידה.

2.1.7 מערכות מידע בעידן הניידות

המעבר לעידן הנייד (*Mobile Era*) שינה באופן מהותי את הדרך שבה מערכות מידע מתוכננות ומשמשות. מכשירים ניידים מספקים הקשר שאינו זמין במחשב שולחני — מיקום גיאוגרפי, חיישני

תנועה, זמן אמיתי, ותקשורת מיידית. מאפיינים אלו מאפשרים בניית מערכות מודעות הקשר (*Context-Aware Systems*) שמגיבות לסביבה הפיזית של המשתמש (*Dey, 2001*).

במערכת **SuperWaze**, רכיב המובייל מנצל מאפיינים אלה בצורה מהותית:

1. **מיקום (GPS)**: חישוב מרחק לחנויות, זיהוי כניסה ויציאה מחנות (Geofencing).
2. **חיישן תאוצה (Accelerometer)**: זיהוי עמידה ממושכת לגילוי שהייה ליד קופה (StandingDetector). חידוש ייחודי לפרויקט זה — שימוש בחיישן פיזי כ-proxy לאירוע עסקי ("הלקוח בקופה").
3. **הקשר זמני**: חיזוי תורים מותאם ליום בשבוע ושעה ביום. Bayesian Prior מעדכן את ההסתברות לזיהוי קופה לפי הקשר עתי.
4. **מצב שימוש (Cart Mode)**: האפליקציה מזהה האם הטלפון ביד המשתמש או בעגלה, ומתאימה את ספי הזיהוי בהתאם — כי עגלה מייצרת רעש תאוצה שונה מהליכה.

2.1.8 סיכום — רלוונטיות למערכת

SuperWaze היא מערכת מידע מרובת ממדים המשלבת מאפיינים של מערכת תמיכה בהחלטות (DSS), מערכת מידע גיאוגרפית (GIS), ומערכת קהילתית מבוססת Crowdsourcing. היא פועלת הן ברשת (וב) והן בנייד, ומנצלת חיישני המכשיר הנייד לאיסוף נתונים שלא ניתן להשיג בדרך אחרת. השילוב בין מקורות מידע מגוונים (חוק שקיפות המחירים, דיווחי משתמשים, חיישנים) לבין אלגוריתמים סטטיסטיים ומתמטיים מאפשר הפקת ערך שאינו קיים באף מערכת קיימת בשוק הישראלי.

2.2 מתודולוגיות פיתוח תוכנה

מתודולוגיית פיתוח תוכנה היא מסגרת עבודה המגדירה את התהליכים, השלבים, הכלים והעקרונות המנחים את בניית מערכת תוכנה. בחירת המתודולוגיה הנכונה משפיעה באופן ישיר על איכות המוצר, עמידה בלוחות זמנים, שביעות רצון המשתמשים ועלות הפיתוח. סעיף זה סוקר את המתודולוגיות המרכזיות ומנמק את הבחירה שנעשתה בפרויקט **SuperWaze**.

2.2.1 מודל מפל המים (Waterfall)

מודל מפל המים *(Waterfall)*, שתואר לראשונה על ידי *Royce (1970)*, הוא המתודולוגיה הקלאסית ביותר לפיתוח תוכנה. במודל זה, הפיתוח מתנהל בשלבים סדרתיים — כל שלב מסתיים לפני שמתחיל השלב הבא, בדומה לזרימת מים במפל:

השלבים: הגדרת דרישות → עיצוב מערכת → מימוש (קידוד) → בדיקות → הטמעה → תחזוקה.

יתרונות: פשוט להבנה וליישום, תיעוד מקיף בכל שלב, מתאים לפרויקטים עם דרישות יציבות ומוגדרות היטב מראש.

חסרונות: קשיחות — קשה לחזור לשלב קודם; המשתמש רואה את המוצר רק בסוף התהליך; אינו מתאים לפרויקטים שבהם הדרישות משתנות תוך כדי פיתוח.

לפי *Sommerville (2016)*, מודל מפל המים מתאים בעיקר לפרויקטים ממשלתיים גדולים או לפרויקטים בהם הדרישות קבועות מראש (כגון מערכות בטיחותיות). עבור פרויקטים חדשניים עם אי-ודאות גבוהה — כמו SuperWaze — מודל זה אינו מיטבי.

2.2.2 מתודולוגיות Agile

בשנת 2001 פורסם **מניפסט האג'ייל** *(Agile Manifesto)* על ידי קבוצה של 17 מפתחים מובילים, אשר הגדיר ארבעה ערכים מרכזיים לפיתוח תוכנה:

1. **אנשים ואינטראקציות** מעל תהליכים וכלים.
2. **תוכנה עובדת** מעל תיעוד מקיף.
3. **שיתוף פעולה עם הלקוח** מעל משא ומתן על חוזים.
4. **היענות לשינויים** מעל מעקב אחר תוכנית.

גישת **Agile** מתבססת על מחזורי פיתוח קצרים (איטרציות), שיתוף פעולה הדוק עם המשתמשים, ויכולת להגיב לשינויים במהירות. לפי *Beck et al. (2001)*, הגישה מניחה שדרישות משתנות הן בלתי נמנעות ושהמפתח שלמידה מתמדת ושיפור מתמיד.

מחקר של *VersionOne (2020)* מצא כי 95% מהארגונים משתמשים בגישות Agile כלשהן, כאשר היתרונות העיקריים שדווחו הם: יכולת לנהל שינוי סדרי עדיפויות (70%), שקיפות הפרויקט (65%), ושיפור שביעות רצון הלקוח (62%).

2.2.3 מסגרת העבודה Scrum

Scrum היא מסגרת העבודה הפופולרית ביותר ליישום Agile. פותחה על ידי *Schwaber & Sutherland (2020)* והיא מבוססת על שלושה עקרונות יסוד: שקיפות, בחינה והתאמה. Scrum מגדירה תפקידים, אירועים ותוצרים ברורים:

תפקידים:

1. **Product Owner** — אחראי על הגדרת סדר העדיפויות של הדרישות (*Product Backlog*). בפרויקט — הסטודנטים עצמם בתיאום עם המנחה.
2. **Scrum Master** — מוודא עמידה בתהליך ומסיר חסמים. בפרויקט — אחד מחברי הצוות.
3. **Development Team** — צוות רב-תחומי האחראי על פיתוח המוצר. בפרויקט — עדן כהן וחובב שוקורון.

אירועים:

1. **Sprint** — מחזור פיתוח בן 1-4 שבועות שבסיומו נוצרת גרסה עובדת של המוצר. בפרויקט: ספרינטים בני שבועיים.
2. **Sprint Planning** — תכנון תחילת הספרינט — בחירת משימות מה-Backlog.
3. **Daily Standup** — פגישה יומית קצרה (15 דקות) לסנכרון הצוות.
4. **Sprint Review** — הצגת התוצרים בסוף הספרינט.
5. **Sprint Retrospective** — ניתוח לקחים ושיפור התהליך.

תוצרים:

1. **Product Backlog** — רשימה ממוינת של כל הדרישות, הפיצ'רים והתיקונים הנדרשים.
2. **Sprint Backlog** — תת-קבוצה של ה-Product Backlog שנבחרה לספרינט הנוכחי.
3. **Increment** — גרסה עובדת של המוצר בסוף כל ספרינט — חייבת לעמוד בהגדרת ה-"Done".

2.2.4 שיטת Kanban

Kanban היא שיטת ניהול זרימת עבודה שמקורה בתעשיית הייצור היפנית (טויוטה) ואומצה לעולם התוכנה על ידי *Anderson (2010)*. בניגוד ל-Scrum שמבוססת על ספרינטים בזמן קבוע, Kanban מתמקדת בזרימה רציפה של משימות דרך שלבים מוגדרים.

עקרונות מרכזיים:

1. **הגבלת עבודה במקביל** *(WIP Limit)* — מגבלה על מספר המשימות ב"תהליך" בו-זמנית, למניעת צווארי בקבוק.
2. **ויזואליזציה של זרימת העבודה** — לוח Kanban עם עמודות: To Do → In Progress → Review → Done.

3. **שיפור מתמיד** *(Kaizen)* — ניתוח קבוע של זרימת העבודה וצמצום זמני המתנה.

4. **ניהול מבוסס מדידה** — מדדים כמו *Lead Time* (זמן מבקשה ועד מסירה) ו-*Cycle Time* (זמן עבודה בפועל).

2.2.5 השוואת מתודולוגיות

קריטריון	Waterfall	Scrum	Kanban
גישה	סדרתית	איטרטיבית (ספרינטים)	זרימה רציפה
גמישות לשינויים	נמוכה — שינוי דורש חזרה לשלב קודם	גבוהה — שינוי בכל ספרינט	גבוהה מאוד — שינוי בכל רגע
מעורבות לקוח	בתחילה ובסוף בלבד	בסוף כל ספרינט (Review)	רציפה
תיעוד	מקיף ומפורט	מינימלי — דגש על תוכנה עובדת	מינימלי
מתאים ל-	דרישות יציבות, פרויקטים גדולים	פרויקטים בינוניים עם אי-ודאות	תחזוקה, תמיכה, זרימה רציפה
תכנון	מראש (מלא)	לכל ספרינט	לפי קיבולת
מדדי הצלחה	עמידה בתוכנית	ערך עסקי בכל ספרינט	זמן מסירה (Lead Time)

2.2.6 הבחירה המתודולוגית בפרויקט SuperWaze

בפרויקט **SuperWaze** נבחרה גישת **Scrum** כמתודולוגיית הפיתוח המרכזית, בשילוב אלמנטים מ-**Kanban** (לוח משימות ויזואלי). הבחירה נבעה מהשיקולים הבאים:

1. **אי-ודאות גבוהה בדרישות:** בתחילת הפרויקט לא היה ברור אילו מבצעים ניתן לייבא, כמה סוגי קופות קיימים, או איך ייראה חיזוי התורים. הגישה האיטרטיבית אפשרה גילוי הדרגתי.

2. **צוות קטן:** Scrum מיועדת לצוותים קטנים (3-9 חברים). צוות של שני מפתחים מתאים היטב.

3. **צורך במשוב מוקדם:** ספרינטים של שבועיים אפשרו הצגת גרסה עובדת למנחה ולמשתמשי בטא בכל שלב.

4. **מורכבות טכנית גבוהה:** שילוב של מנוע מבצעים, חיזוי תורים, זיהוי קופות ו-ETL — כל אחד דורש למידה ואיטרציות. ספרינטים אפשרו התמקדות ברכיב אחד בכל פעם.

5. **CI/CD אוטומטי:** גישת Agile תומכת באינטגרציה רציפה — כל ספרינט מסתיים בגרסה עובדת שעוברת בדיקות אוטומטיות ב-GitHub Actions.

תכנון הספרינטים בפרויקט:

ספרינט	משך	תוצר עיקרי
1-2	4 שבועות	מחקר ואפיון — סקר צרכנים, ניתוח מתחרים, הגדרת דרישות
3-4	4 שבועות	תשתית — PostgreSQL schema, API skeleton, אימות JWT

5-6	4 שבועות	ETL מחירים + מבצעים, BasketService, PromoEngine
7-8	4 שבועות	ממשק QueueService (EWMA), RecommendationService ווב בסיסי
9-10	4 שבועות	זיהוי קופות (GMM, LaneClassifier), חיזוי M/M/c, StandingDetector
11-12	4 שבועות	אפליקציית מובייל (Expo), ניווט פנים-חנות, CI/CD
13-14	4 שבועות	בדיקות מקיפות, תיקון באגים, בדיקות בטא, כתיבת תיעוד

2.2.7 DevOps ואינטגרציה רציפה (CI/CD)

תרבות **DevOps** משלבת בין פיתוח *(Dev)* ותפעול *(Ops)* לכדי תהליך אחד רציף. לפי *Kim et al. (2016)*, DevOps מבוססת על שלושה עקרונות: זרימה מהירה (*Flow*), משוב מהיר (*Feedback*), ולמידה מתמדת (*Continual Learning*).

בפרויקט **SuperWaze**, עקרונות DevOps יושמו באמצעות:

1. **CI (Continuous Integration)** — כל push ל-GitHub מפעיל הרצת בדיקות אוטומטיות (ci.yml). אם בדיקה נכשלת — ה-merge נחסם.
2. **CD (Continuous Deployment)** — פריסה אוטומטית באמצעות Docker Compose עם nginx כ-reverse proxy.
3. **ETL Scheduling** — עדכון מחירים מתוזמן ב-(nightly-etl.yml) GitHub Actions כל לילה ב-06:00 שעון ישראל.
4. **Structured Logging** — שימוש ב-Serilog לניטור מובנה ומעקב אחר שגיאות.

2.2.8 סיכום ביניים

סקירת מתודולוגיות הפיתוח מבססת את הבחירה בגישת **Scrum** לפרויקט **SuperWaze**. הגישה האיטרטיבית אפשרה התמודדות עם אי-ודאות בדרישות, למידה הדרגתית של תחומי הידע המגוונים (מבצעים, חיזוי תורים, זיהוי קופות), ושילוב משוב ממשתמשים בכל שלב. השילוב עם עקרונות DevOps ו-CI/CD הבטיח שכל ספרינט מסתיים בגרסה יציבה ובדוקה. בסעיף הבא נסקור את הארכיטקטורות הטכניות שעומדות בבסיס המערכת.

2.3 ארכיטקטורות תוכנה

ארכיטקטורת תוכנה *(Software Architecture)* מתארת את המבנה העילי של מערכת תוכנה — הרכיבים שלה, הקשרים ביניהם, ועקרונות הפעולה שלהם. לפי *Bass, Clements & Kazman (2012)*, הארכיטקטורה היא ההחלטה העיצובית החשובה ביותר בפרויקט תוכנה, כיוון שהיא קובעת את תכונות האיכות של המערכת — ביצועים, סקלביליות, תחזוקתיות ואבטחה. סעיף זה סוקר את הדפוסים הארכיטקטוניים המרכזיים ומציג את הבחירות שנעשו ב-**SuperWaze**.

2.3.1 ארכיטקטורת לקוח-שרת (Client-Server)

ארכיטקטורת לקוח-שרת *(Client-Server)* היא הדפוס הבסיסי ביותר במערכות מבוזרות. במודל זה, הלקוח (דפדפן, אפליקציה ניידת) שולח בקשות לשרת מרכזי, והשרת מעבד אותן ומחזיר תגובה. לפי *Tanenbaum & Van Steen (2017)*, המודל מאפשר הפרדה ברורה בין הצגה (לקוח) ללוגיקה עסקית (שרת).

SuperWaze מיישמת ארכיטקטורת לקוח-שרת עם שלושה לקוחות:

1. **ממשקווב** (Next.js) — דפדפן שמתקשר עם ה-API באמצעות REST ו-SignalR.
2. **אפליקציית מובייל** (React Native/Expo) — אפליקציה ניידת עם יכולות מקומיות (חיישן תאוצה, Geofencing).
3. **תהליכי ETL** — סקריפטים ב-Python שמתקשרים ישירות עם בסיס הנתונים (לא דרך ה-API).

השרת המרכזי (ASP.NET Core 8) אחראי על כל הלוגיקה העסקית — חישוב מחירים, מבצעים, חיזוי תורים, ניווט ואימות. התקשורת בין הלקוחות לשרת מתבצעת באמצעות HTTPS, כאשר nginx משמש כ-reverse proxy ומנתב בקשות בין הממשק לשרת ה-API.

2.3.2 ארכיטקטורת שכבות (Layered Architecture)

ארכיטקטורת שכבות *(Layered / N-Tier Architecture)* מחלקת את המערכת לשכבות אופקיות, כאשר כל שכבה אחראית על תחום מוגדר ומתקשרת רק עם השכבות הסמוכות לה. לפי *Richards (2015)*, זהו הדפוס הנפוץ ביותר בתעשייה מכיוון שהוא מאפשר הפרדת אחריות ברורה, תחזוקתיות גבוהה ובדיקות יחידה אפקטיביות.

השכבות ב-**SuperWaze**:

שכבה	אחריות	מיפוי ב-SuperWaze
הצגה (Presentation)	ממשק משתמש — הצגת נתונים וקלט	Next.js (וווב), React Native (מובייל)
API / בקרה	ניתוב בקשות, אימות קלט, serialization	Controllers (AuthController, PredictController...)
לוגיקה עסקית (Business)	חישובים, אלגוריתמים, כללי עסק	Services (PromoEngine, QueueService, RecommendationService...)

גישה לנתונים (Data Access)	שאילתות, ORM, מטמון	AppDbContext (Entity Framework), Redis Cache
בסיס נתונים (Database)	אחסון פרסיסטנטי של כל הנתונים	PostgreSQL (10 מיגרציות, סכמה מנורמלת ל-3NF)

ההפרדה לשכבות מאפשרת, למשל, להחליף את שכבת בסיס הנתונים מ-PostgreSQL ל-SQL Server מבלי לשנות את הלוגיקה העסקית, או להוסיף לקוח חדש (למשל, צ'טבוט) מבלי לגעת בשירותים הקיימים.

2.3.3 ארכיטקטורת REST API

REST (Representational State Transfer) הוא סגנון ארכיטקטוני לתכנון ממשקי תוכנה מבוזרים, שהוגדר על ידי *Fielding (2000)* במסגרת עבודת הדוקטורט שלו. REST מבוסס על שישה עקרונות:

1. **Client-Server** — הפרדה בין לקוח לשרת.
2. **Stateless** — כל בקשה מכילה את כל המידע הנדרש; השרת אינו שומר מצב בין בקשות. ב-SuperWaze — הזיהוי מתבצע באמצעות JWT token שנשלח עם כל בקשה.
3. **Cacheable** — תגובות ניתנות לשמירה במטמון. ב-SuperWaze — תוצאות סל ומחירים נשמרות ב-Redis לזמן מוגדר.
4. **Uniform Interface** — ממשק אחיד: משאבים מזוהים ב-URI, פעולות באמצעות HTTP verbs (GET, POST, PUT, DELETE).
5. **Layered System** — מערכת בשכבות; הלקוח אינו יודע אם הוא מתקשר ישירות עם השרת או דרך proxy. ב-SuperWaze — nginx פועל כשכבת ביניים.
6. **Code on Demand** (אופציונלי) — השרת יכול לשלוח קוד הרצה ללקוח. לא מיושם ב-SuperWaze.

נקודות הקצה המרכזיות ב-**SuperWaze API**:

Endpoint	פעולה	תיאור
POST /api/auth/register	הרשמה	יצירת חשבון משתמש חדש
POST /api/auth/login	התחברות	הנפקת JWT + Refresh Token
POST /api/predict	השוואת מחירים	חישוב עלות נטו לכל חנות + חיזוי תורים
POST /api/queue/report	דיווח תור	קבלת דיווח תור מהמשתמש
GET /api/queue/predict/{storeId}	חיזוי תור	תחזית EWMA + M/M/c
POST /api/navigation/plan	ניווט בחנות	מסלול אופטימלי בתוך החנות
GET /api/stores	רשימת חנויות	כל החנויות הפעילות בעיר
POST /api/checkout/event	אירוע קופה	דיווח checkout לזיהוי קופות

2.3.4 מונוליט לעומת מיקרו-שירותים

אחת ההחלטות הארכיטקטוניות המרכזיות בכל פרויקט תוכנה היא הבחירה בין ארכיטקטורת **מונוליט** *(Monolith)* לבין ארכיטקטורת **מיקרו-שירותים** *(Microservices)*.

קריטריון	**מונוליט** (Monolith)	**מיקרו-שירותים** (Microservices)
מבנה	אפליקציה אחת שמכילה את כל הקוד	אוסף שירותים עצמאיים שכל אחד רץ בנפרד
פריסה	כל האפליקציה נפרסת כיחידה אחת	כל שירות נפרס בנפרד
סקלביליות	סקלביליות אנכית (שרת חזק יותר)	סקלביליות אופקית (יותר מופעים)
מורכבות תפעולית	נמוכה — אפליקציה אחת לנהל	גבוהה — עשרות שירותים, רשתות, ניטור
תקשורת	קריאות פונקציה ישירות (מהירות)	קריאות רשת בין שירותים (HTTP/gRPC)
מתאים ל-	צוותים קטנים, פרויקטים בתחילת דרכם	ארגונים גדולים עם צוותים רבים
סיכון	שינוי ברכיב אחד עלול לשבור את הכל	כשל בשירות אחד לא מפיל את המערכת

בפרויקט **SuperWaze** נבחרה ארכיטקטורת **מונוליט מודולרי** *(Modular Monolith)*. זוהי גישת ביניים: קוד בסיס אחד (מונוליט) אך עם הפרדה פנימית ברורה למודולים עצמאיים. הבחירה נבעה מהשיקולים הבאים:

1. **צוות קטן:** שני מפתחים — מיקרו-שירותים מוסיפים מורכבות תפעולית לא מוצדקת.
2. **פשטות פריסה:** Docker Compose אחד מריץ את כל המערכת — שרת, בסיס נתונים, מטמון וממשק.
3. **ביצועים:** קריאות בין רכיבי המערכת (למשל, RecommendationService → PromoEngine → QueueService) הן קריאות פונקציה ישירות — אין תקורת רשת.
4. **מודולריות פנימית:** למרות שזהו מונוליט, הקוד מחולק בבירור: Controllers (ניתוב), Services (לוגיקה), Domain (ישויות), Dtos (חוזים), Validation (אימות) — כל מודול ניתן לבדיקה ולהחלפה בנפרד.
5. **אפשרות למעבר עתידי:** לפי *Newman (2021)*, מונוליט מודולרי הוא נקודת המוצא הטובה ביותר — ניתן לחלץ שירותים לאחר שמזהים את גבולות התחום הנכונים.

2.3.5 תקשורת בזמן אמת (WebSockets / SignalR)

מערכות מודרניות רבות דורשות עדכונים בזמן אמת — ללא צורך בריענון עמוד. פרוטוקול **WebSocket** מאפשר ערוץ תקשורת דו-כיווני מתמשך בין לקוח לשרת. לפי *Fette & Melnikov (RFC 6455, 2011)*, WebSocket מפחית את התקורה לעומת HTTP polling באופן דרמטי, כיוון שה-handshake מתבצע פעם אחת בלבד.

SuperWaze משתמשת ב-**SignalR** — ספריית תקשורת בזמן אמת של מיקרוסופט שבונה על גבי WebSocket (עם fallback ל-Server-Sent Events ו-Long Polling). ה-Hub המרכזי *(StoreActivityHub)* שולח עדכונים חיים כאשר:

1. משתמש מדווח על תור בחנות — כל המחוברים לאותה חנות מקבלים עדכון מיידי.
2. חיזוי התורים משתנה (למשל, בעקבות דיווח חדש) — המלצות מתעדכנות בזמן אמת.
3. ביקור בחנות מתחיל או מסתיים — עדכוני סטטוס חיים.

2.3.6 הזרקת תלויות (Dependency Injection)

הזרקת תלויות *(Dependency Injection — DI)* היא דפוס עיצוב שבו אובייקט מקבל את התלויות שלו מבחוץ במקום ליצור אותן בעצמו. לפי DI *Martin (2017),* מאפשרת:

1. **צימוד רופף** *(Loose Coupling)* — רכיבים תלויים בממשקים, לא במימושים.
2. **בדיקתיות** *(Testability)* — ניתן להחליף תלויות ב-mock לצורך בדיקות יחידה.
3. **גמישות** — החלפת מימוש (למשל Redis → InMemoryCache) דורשת שינוי שורה אחת בקובץ הקונפיגורציה.

ב-**SuperWaze**, כל השירותים נרשמים באמצעות DI המובנה של ASP.NET Core בקובץ *Program.cs*. לדוגמה, IQueueService הוא ממשק שמאחוריו עומד QueueService — אך בבדיקות ניתן להחליפו ב-mock. באופן דומה, שכבת המטמון מוגדרת כ-IDistributedCache — ב-production זה Redis, בפיתוח זה InMemoryCache.

2.3.7 סיכום ביניים

הארכיטקטורה של **SuperWaze** משלבת מספר דפוסים ארכיטקטוניים מוכחים: ארכיטקטורת לקוח-שרת עם שלושה לקוחות (וב, מובייל, ETL), ארכיטקטורת שכבות בשרת, ממשק REST API סטנדרטי, מונוליט מודולרי לפשטות ופריסה, תקשורת בזמן אמת באמצעות SignalR, והזרקת תלויות לגמישות ובדיקתיות. שילוב זה מייצר מערכת שהיא גם פשוטה לתחזוקה וגם מסוגלת להתמודד עם מורכבות עסקית גבוהה. בסעיף הבא נסקור את תחום מערכות השוואת המחירים והקמעונאות הדיגיטלית.

2.4 מערכות השוואת מחירים וקמעונאות דיגיטלית

סעיף זה סוקר את הרקע התיאורטי של מערכות השוואת מחירים, את חוק שקיפות המחירים בישראל שמהווה את מקור הנתונים המרכזי של **SuperWaze**, ואת המגמות בקמעונאות דיגיטלית שמשפיעות על עיצוב המערכת.

2.4.1 מערכות השוואת מחירים — רקע תיאורטי

מערכות השוואת מחירים *(Price Comparison Systems / Shopbots)* הן מערכות מידע שמרכזות מחירים ממקורות שונים ומציגות אותם לצרכן באופן מאורגן וניתן להשוואה. לפי *Brynjolfsson & Smith (2000)*, מערכות כאלו הן אחד הגורמים המשמעותיים ביותר להגברת התחרותיות בשווקים מקוונים, מכיוון שהן מפחיתות באופן דרמטי את עלויות החיפוש של הצרכן.

מחקרם הפורץ דרך של *Baye, Morgan & Scholten (2004)* על מנועי השוואת מחירים הראה כי עצם קיומן של מערכות כאלו מוביל להפחתת פיזור מחירים בשוק. ככל שיותר צרכנים משתמשים במנוע השוואה, כך המוכרים נאלצים להתקרב למחיר תחרותי — ובכך השוק כולו מתייעל.

ניתן לסווג מערכות השוואת מחירים לפי מספר ממדים:

ממד סיווג	סוגים	דוגמאות
מקור הנתונים	סריקה אוטומטית (web scraping) / נתונים פתוחים (Open Data) / דיווח ידני	SuperWaze: נתונים פתוחים (חוק שקיפות)
היקף	רשת בודדת / רב-רשתי / חוצה ענפים	SuperWaze: רב-רשתי (10+ רשתות)
מה משווים	מוצר בודד / סל קניות שלם / עלות כוללת	SuperWaze: עלות כוללת (סל + נסיעה + זמן)
התחשבות במבצעים	ללא / מבצעים בסיסיים / כפל מבצעים	SuperWaze: כפל מבצעים (Promo Stacking)
מודל עסקי	חינמי / פרמיום / עמלה מהמוכר (affiliate)	SuperWaze: חינמי (פרויקט אקדמי)

2.4.2 חוק שקיפות המחירים בישראל

בשנת 2014 חוקק תיקון לצו הגנת הצרכן (סימון ומחירים של מצרכים), המכונה בשם **"חוק שקיפות המחירים"**. החוק נכנס לתוקף בינואר 2015 ומחייב כל רשת קמעונאית עם מחזור שנתי מעל חצי מיליארד ש"ח או עם יותר מ-10 סניפים לפרסם באופן מקוון את מלוא מחירי המוצרים שלה.

עיקרי החוק:

1. **חובת פרסום מחירים:** כל הרשתות הגדולות חייבות לפרסם את מחירי כל המוצרים בפורמט XML באתר שלהן, עם עדכון יומי לפחות.

2. **חובת פרסום מבצעים:** נוסף למחירי מדף, הרשתות חייבות לפרסם את כל המבצעים הפעילים — כולל סוג מבצע, מחיר מוזל, תנאי סף (כמות מינימלית, חברות במועדון), תאריכי תחילה וסיום.

3. **פורמט אחיד:** הנתונים מפורסמים בקבצי XML (לעיתים דחוסים ב-GZ) עם סכמה מוגדרת הכוללת: ברקוד, שם מוצר, יצרן, מחיר, מחיר ליחידת משקל, ותאריך עדכון.

4. **זיהוי סניף:** המחירים מפורסמים ברמת סניף — לא רמת רשת. מחיר של אותו מוצר יכול להשתנות בין סניפים.

5. **נגישות ציבורית:** הנתונים חייבים להיות נגישים ללא רישום או תשלום, אם כי אין API אחיד — כל רשת מפרסמת באתר שלה בפורמט מעט שונה.

אתגרים מעשיים:

למרות כוונות החוק הטובות, ישנם אתגרים משמעותיים בניצול הנתונים. לפי דוח מבקר המדינה (2018), רשתות רבות מפרסמות את הנתונים בפורמטים שונים, עם חוסר עקביות בשמות מוצרים (למשל, "חלב תנובה 3%" לעומת "תנובה חלב 3 אחוז"), שגיאות קידוד עברית, וקבצים שלעיתים אינם זמינים. אתגרים אלו הם הסיבה המרכזית לקיומם של צינורות ה-ETL המורכבים ב-SuperWaze, הכוללים נירמול שמות בעברית (*hebrew_nlp.py*), טיפול בחריגות פורמט, וגילוי אוטומטי של כתובות URL עדכניות.

2.4.3 מודלים של מבצעי שיווק בקמעונאות

מבצעי שיווק *(Sales Promotions)* הם כלי מרכזי בשוק הקמעונאות. לפי *Kotler & Keller (2016)*, מבצעים נועדו להגביר מכירות בטווח הקצר, למשוך לקוחות חדשים, ולנקות מלאי. בישראל, מבצעים מהווים חלק משמעותי מהוצאות הצרכן — מחקר של *Nielsen (2019)* מצא כי כ-35% מהמכירות בסופרמרקטים בישראל מתבצעות במסגרת מבצעים.

סוגי המבצעים העיקריים ומיפויים למנוע המבצעים של **SuperWaze**:

סוג מבצע	תיאור	מיפוי ב-**PromoEngine**
הורדת מחיר ישירה	מחיר מוזל קבוע למוצר	**PRICE_CUT**
הנחה באחוזים	אחוז הנחה מהמחיר המלא	**PCT_OFF**
רב-קנה	מחיר מיוחד לכמות (3 ב-10₪)	**MULTI_BUY**
קנה-קבל	קנה X יחידות קבל Y חינם	**BUY_GET**
מבצע מועדון	הנחה לחברי מועדון בלבד	כל סוג עם IsMemberOnly = true
כפל מבצעים	שני מבצעים או יותר על אותו מוצר	**EXCLUSIVE + ADDITIVE Stacking**

החידוש המרכזי של **SuperWaze** בתחום המבצעים הוא תמיכה ב**כפל מבצעים (Promo Stacking)**. בפועל, רשתות רבות מאפשרות לצרכן ליהנות ממבצע מועדון (ADDITIVE) על גבי מבצע כללי (EXCLUSIVE). אפליקציות קיימות מתעלמות מכך ומציגות רק מחיר מדף או

מבצע בודד — ובכך מספקות תמונה לא מדויקת של המחיר לתשלום. מנוע המבצעים של SuperWaze מחשב את המחיר הסופי המדויק כולל כפל, ומציג לצרכן את החיסכון האמיתי.

2.4.4 מושג העלות האמיתית של הקנייה (Total Cost of Shopping)

רוב מערכות השוואת המחירים הקיימות מתמקדות במחיר הסל בלבד. עם זאת, **"העלות האמיתית"** של הקנייה כוללת מרכיבים נוספים שצרכנים נוטים להתעלם מהם. מושג זה מבוסס על תיאוריית **"העלות הכוללת של הצריכה"** *(Total Cost of Consumption)* של *Wouters, Anderson & Wynstra (2005)*, שמרחיבה את מושג העלות מעבר למחיר הרכישה.

SuperWaze מגדירה את העלות הכוללת של הקנייה (Net Cost) כסכום של שלושה מרכיבים:

מרכיב	תיאור	חישוב במערכת
עלות סל (Basket Cost)	סכום מחירי כל המוצרים ברשימה לאחר הפעלת מבצעים	PromoEngine.ApplyToBasket()
עלות נסיעה (Fuel Cost)	עלות דלק/חשמל הלוך-חזור לחנות, מותאמת לסוג הרכב	TrafficService × צריכת דלק × מחיר ליטר
עלות זמן (Time Cost)	ערך מוניטרי של הזמן שהקנייה לוקחת (נסיעה + חנות + תור)	(נסיעה×2 + חנות + תור) × 0.5₪/דקה

נוסחת העלות הנטו: NetCost = BasketCost + FuelCost + TimeCost

גישה זו מאפשרת, למשל, להמליץ על חנות שבה הסל יקר יותר ב-5₪ אך הנסיעה קצרה יותר ב-20 דקות — כי העלות הכוללת נמוכה יותר. המשתמש יכול גם לבחור מצב מיון: עלות נטו (ברירת מחדל), מחיר סל בלבד, או זמן כולל בלבד — בהתאם להעדפותיו.

2.4.5 מגמות בקמעונאות דיגיטלית

שוק הקמעונאות עובר טרנספורמציה דיגיטלית מואצת. לפי דוח *McKinsey (2022)*, מגפת הקורונה האיצה את האימוץ של כלים דיגיטליים בקנייה פיזית (לא רק מקוונת). להלן מגמות מרכזיות הרלוונטיות ל-**SuperWaze**:

1. **BOPIS** *(Buy Online, Pick up In Store)* — צרכנים מזמינים מקוון ואוספים בחנות. SuperWaze יכולה בעתיד לשמש ככלי תכנון לבחירת נקודת האיסוף האופטימלית.
2. **פרסונליזציה** — התאמה אישית של מבצעים ומלצות על בסיס היסטוריית קניות. SuperWaze כבר שומרת העדפות סוג רכב ועיר — בסיס להרחבה עתידית.
3. **שקיפות מחירים בזמן אמת** — צרכנים דורשים מידע מחירים מעודכן ומדויק. SuperWaze מעדכנת נתונים יומית ומספקת חיזוי תורים בזמן אמת.
4. ***Frictionless Shopping*** — צמצום חיכוך בחוויית הקנייה. SuperWaze מצמצמת חיכוך בשלושה ממדים: חיפוש (השוואה אוטומטית), ניווט (מסלול בחנות), והמתנה (בחירת קופה אופטימלית).

5. **Crowdsourcing ו-Community Commerce** — מיצוי כוח הקהילה למידע. SuperWaze מבוססת על דיווחי תורים קהילתיים ועל זיהוי קופות אוטומטי מנתוני משתמשים.

2.4.6 שוק השוואת המחירים בישראל

בשוק הישראלי פעילות מספר אפליקציות ואתרים להשוואת מחירי מזון. הפופולריות שבהן הן **Pricez** (פרייסז), **מסלול הזול**, ו**שופרסל אונליין** (שמשווה מחירים בתוך הרשת בלבד). ניתוח מפורט של מתחרים אלו יוצג בפרק 3 (חקר מצב קיים).

מבט על השוק מראה כי לאף מתחרה קיים אין פתרון שמשלב את כל הממדים שמציע **SuperWaze**: השוואת סל שלם עם כפל מבצעים, חיזוי תורים מפורט לכל סוג קופה, חישוב עלות נסיעה מותאם לסוג רכב, ניווט פנים-חנות, וזיהוי קופות אוטומטי. פער זה מהווה את ההזדמנות העסקית והאקדמית של הפרויקט.

2.4.7 סיכום ביניים

סקירת תחום מערכות השוואת המחירים והקמעונאות הדיגיטלית מבססת את הייחודיות של **SuperWaze**: המערכת ממנפת את חוק שקיפות המחירים לא רק למחירי מדף אלא גם למבצעים (עם תמיכה בכפל), מחשבת עלות כוללת (סל + נסיעה + זמן), ומספקת חיזוי תורים שאינו קיים באף מתחרה. הגישה של "עלות אמיתית" מתיישבת עם התיאוריה הכלכלית של *Frictionless Shopping*. בעלות כוללת של צריכה ועם המגמה בקמעונאות לכיוון בסעיף הבא נסקור את המודלים הסטטיסטיים שעומדים בבסיס חיזוי התורים.

2.5 מודלים סטטיסטיים לחיזוי

חיזוי זמני המתנה בתורים הוא אחד האתגרים המרכזיים של **SuperWaze**. סעיף זה סוקר את המודלים הסטטיסטיים שנבחרו לחיזוי — ממוצע נע משוקלל אקספוננציאלית (EWMA) לחיזוי קהילתי כללי, ושיטות סינון חריגים (MAD) להבטחת איכות הנתונים.

2.5.1 טורים עתיים — רקע תיאורטי

טור עתי *(Time Series)* הוא סדרה של תצפיות הנמדדות לאורך זמן. לפי *Hyndman & Athanasopoulos (2021)*, ניתוח טורים עתיים נועד לזהות דפוסים בנתוני עבר ולהשתמש בהם לחיזוי העתיד. הדפוסים המרכזיים כוללים:

1. **מגמה** *(Trend)* — שינוי ארוך-טווח בערך הטור. למשל: עלייה הדרגתית בזמני המתנה לאורך השנים.
2. **עונתיות** *(Seasonality)* — דפוס מחזורי קבוע. למשל: תורים ארוכים יותר ביום שישי ולפני חגים.
3. **מחזוריות** *(Cyclical)* — תנודות ארוכות-טווח שאינן בתדירות קבועה.
4. **רעש** *(Noise)* — תנודות אקראיות שאינן חלק מדפוס. למשל: תור חריג בגלל תקלה בקופה.

בהקשר של חיזוי תורים ב-**SuperWaze**, הנתונים מציגים עונתיות ברורה (יום בשבוע × שעה ביום) ורעש משמעותי (שונות בין דיווחי משתמשים). האתגר הוא לחלץ את האות (signal) מתוך הרעש (noise) — ולכך נבחרו שיטות EWMA ו-MAD.

2.5.2 ממוצע נע משוקלל אקספוננציאלית (EWMA)

ממוצע נע משוקלל אקספוננציאלית *(Exponentially Weighted Moving Average — EWMA)* הוא שיטת חיזוי שמעניקה משקל דועך אקספוננציאלית לתצפיות ישנות יותר. השיטה הוצגה לראשונה על ידי *Roberts (1959)* ופותחה לשימוש בחיזוי על ידי *Holt (1957)*. הנוסחה הבסיסית:

$$S_t = \alpha \cdot X_t + (1 - \alpha) \cdot S_{t-1}$$

כאשר:

1. S_t — הערך המוחלק (החיזוי) בזמן t.
2. X_t — התצפית (הדיווח) החדשה בזמן t.
3. α (אלפא) — פרמטר ההחלקה ($0 < \alpha < 1$). ערך גבוה → תגובה מהירה לשינויים; ערך נמוך → חלקות רבה יותר.
4. S_{t-1} — החיזוי הקודם.

למה EWMA נבחרה ל-SuperWaze?

1. **פשטות חישובית:** דורשת שמירה של ערך אחד בלבד (S_{t-1}) — יעילה מאוד בזיכרון ובזמן ריצה.

2. **עדכון מקוון:** *(Online Update)* — כל דיווח חדש מעדכן את החיזוי באופן מיידי, ללא צורך לעבד מחדש את כל ההיסטוריה.

3. **טיפול בעונתיות:** ב-SuperWaze, נשמרת מטריצת EWMA נפרדת לכל צירוף של (יום בשבוע × שעה ביום) — 7×24 = 168 תאים — מה שמטפל בעונתיות באופן טבעי.

4. **שני אופקי חיזוי:** המערכת מחזיקה שני ערכי α: $\alpha = 0.25$ (ארוך-טווח, היסטורי) ו- $\alpha = 0.10$ (קצר-טווח, זמן אמת). השילוב מאפשר חיזוי יציב ורגיש בו-זמנית.

5. **רובסטיות:** בשילוב עם סינון חריגים (MAD), השיטה עמידה בפני דיווחים שגויים או קיצוניים.

2.5.3 סינון חריגים — Median Absolute Deviation (MAD)

כאשר נתוני התורים מגיעים מדיווחי משתמשים (Crowdsourcing), יש סיכון לדיווחים חריגים — שגויים, מוגזמים, או זדוניים. סינון חריגים הוא שלב קריטי לשמירה על איכות החיזוי.

השיטה הנפוצה ביותר לזיהוי חריגים היא שימוש בסטיית תקן (σ). עם זאת, סטיית תקן רגישה מאוד לערכים קיצוניים — ערך חריג בודד יכול "לנפח" את סטיית התקן ולגרום לכך שחריגים נוספים לא יזוהו. לפי *Leys et al. (2013)*, שיטת **MAD (Median Absolute Deviation)** רובסטית הרבה יותר:

$$\mathbf{MAD = Median(|X_i - Median(X)|)}$$

כדי להשוות MAD לסטיית תקן, מכפילים בקבוע:

$$\hat{\boldsymbol{\sigma}} \mathbf{= 1.4826 \cdot MAD}$$

תצפית X_i מוגדרת כחריגה אם:

$$\mathbf{|X_i - Median(X)| > k \cdot MAD} \quad \mathbf{(k = 3.0 \text{ ברירת מחדל})}$$

ב-**SuperWaze** מיושם **Weighted Double MAD** — גרסה משופרת שמחשבת MAD בנפרד לערכים מעל ומתחת לחציון. זה מאפשר גבולות אסימטריים — חשוב כי התפלגות זמני המתנה אינה סימטרית ("זנב" ימני ארוך — תורים ארוכים במיוחד נדירים אך אפשריים).

יתרונות השיטה:

1. **רובסטיות:** החציון אינו מושפע מערכים קיצוניים — גם אם 49% מהדיווחים שגויים, החציון עדיין תקין.

2. **אסימטריה:** Double MAD מאפשר סף שונה לחריגים "למטה" (תור קצר מדי — ייתכן שגיאת דיווח) ו"למעלה" (תור ארוך מדי — ייתכן אירוע חריג).

3. **יעילות:** החישוב פשוט ומהיר — מתאים לעדכון בזמן אמת.

2.5.4 השוואת שיטות חיזוי

שיטה	מורכבות	דרישת נתונים	עדכון מקוון	מתאימה ל-
EWMA	נמוכה	דגימה אחת	כן — מיידי	חיזוי זמן אמת מדיווחי קהילה
ARIMA	בינונית-גבוהה	היסטוריה ארוכה	לא — דורש רענון מודל	טורים עתיים עם מגמה ועונתיות
Prophet	בינונית	היסטוריה + חגים	לא — דורש אימון מחדש	חיזוי עם אפקטי חגים
LSTM	גבוהה מאוד	אלפי דגימות	לא — למידה כבדה	דפוסים מורכבים, נתונים רבים
M/M/c	בינונית	λ, μ, c	כן — פרמטרי	חיזוי לכל סוג קופה

הבחירה ב-**EWMA** כשיטה העיקרית נבעה מהצורך בעדכון מקוון מיידי עם כל דיווח חדש, ודרישת נתונים מינימלית (אין צורך בהיסטוריה ארוכה לפני שהמודל מתחיל לעבוד). לחיזוי מפורט לכל סוג קופה נוסף מודל **M/M/c** — המתואר בסעיף הבא.

2.5.5 סיכום ביניים

המודלים הסטטיסטיים שנבחרו ל-**SuperWaze** — EWMA לחיזוי ו-Weighted Double MAD לסינון — משלבים פשטות חישובית, יכולת עדכון בזמן אמת ורובסטיות לנתונים רועשים. השילוב של שני ערכי α (ארוך-טווח וקצר-טווח) עם מטריצת עונתיות (168 תאים) מאפשר חיזוי מדויק ורגיש. בסעיף הבא נרחיב על מודל M/M/c מתורת התורים, המשלים את EWMA בחיזוי מפורט לכל סוג קופה.

2.6 תורת התורים ומודלים לחיזוי זמני המתנה

תורת התורים *(Queueing Theory)* היא ענף במתמטיקה שימושית העוסק בניתוח מערכות שירות שבהן לקוחות ממתינים לקבלת שירות. התיאוריה פותחה בתחילת המאה ה-20 על ידי *Erlang (1909)* לצורך תכנון רשתות טלפון, ומאז יושמה במגוון רחב של תחומים — מרשתות מחשבים ועד סופרמרקטים.

2.6.1 מושגי יסוד בתורת התורים

מערכת תורים מוגדרת על ידי שלושה מרכיבים:

1. **תהליך ההגעות** *(Arrival Process)* — האופן שבו לקוחות מגיעים למערכת. הפרמטר המרכזי: **λ** (למבדא) — קצב ההגעות (לקוחות ליחידת זמן).
2. **תהליך השירות** *(Service Process)* — משך הזמן שלוקח לשרת לקוח אחד. הפרמטר המרכזי: **μ** (מיו) — קצב השירות (לקוחות ליחידת זמן לשרת אחד).
3. **מספר השרתים** *(Servers)* — כמה שרתים (קופות) פעילים במקביל. מסומן ב-**c**.

מערכות תורים מסומנות בסימון **Kendall** בפורמט **A/B/c**, כאשר A = התפלגות ההגעות, B = התפלגות השירות, ו-c = מספר השרתים. האות M מסמנת התפלגות מרקובית

(אקספוננציאלית) — כלומר **M/M/c** מתארת מערכת עם הגעות פואסוניות, שירות אקספוננציאלי ו-c שרתים.

2.6.2 מודל M/M/c

מודל **M/M/c** הוא ההרחבה הטבעית של מודל M/M/1 (שרת בודד) ל-c שרתים מקביליים. לפי *Gross, Shortle, Thompson & Harris (2008)*, המודל מניח:

1. הגעות לפי תהליך פואסון בקצב λ (לקוחות לשעה).
2. זמני שירות מתפלגים אקספוננציאלית בקצב μ (לקוחות לשעה לשרת).
3. c שרתים (קופות) פעילים — כולם זהים.
4. תור אחד משותף (FIFO) — הלקוח הבא מופנה לשרת הפנוי הראשון.
5. **יציבות:** $\rho = \lambda / (c \cdot \mu) < 1$ — העומס חייב להיות קטן מ-1 כדי שהתור לא יגדל ללא גבול.

הנוסחאות המרכזיות:

הסתברות שהמערכת ריקה (P_0):

$$P_0 = \left[\sum_{n=0}^{c-1} \frac{(\lambda/\mu)^n}{n!} + \frac{(\lambda/\mu)^c}{c! \cdot (1-\rho)} \right]^{-1}$$

תוחלת זמן המתנה בתור (W_q):

$$W_q = \frac{P_0 \cdot (\lambda/\mu)^c \cdot \rho}{c! \cdot (1-\rho)^2 \cdot c \cdot \mu}$$

תוחלת זמן שהייה כוללת במערכת (W):

$$W = W_q + \frac{1}{\mu}$$

2.6.3 יישום M/M/c ב-SuperWaze

ב-**SuperWaze**, מודל M/M/c מיושם ב-*CheckoutLaneService* בנפרד לכל סוג קופה (רגילה, מהירה, שירות עצמי) בכל סניף. הפרמטרים מחושבים כך:

פרמטר	חישוב	מקור הנתונים
λ — קצב הגעת לקוחות	ספירת checkout_events בשעה הנוכחית + יום בשבוע (ממוצע 4 שבועות)	checkout_events (דיווחי משתמשים)
μ — קצב שירות	הופכי של ממוצע actual_wait_min לסוג קופה זה	checkout_events.actual_wait_min
c — מספר קופות פעילות	ספירת checkout_lanes פעילות מאותו סוג	checkout_lanes (תוצר ה- GMM Clustering)

התוצאה: לכל סניף מוחזרת רשימת *LaneQueueEstimate* — לכל סוג קופה (רגילה/מהירה/שירות עצמי) נקבע:

1. **WaitMinutes** — תוחלת זמן ההמתנה (דקות).

2. **Source** — מקור החיזוי: "mmC" (מודל M/M/c), "ewma" (fallback קהילתי), או "default" (ברירת מחדל).

3. **Confidence** — רמת ביטחון: "high" (50+ דגימות), "low" (<10), "medium" (10-49).

2.6.4 מגבלות המודל והתמודדות

מודל M/M/c מבוסס על הנחות מפשטות שלא תמיד מתקיימות בסופרמרקט אמיתי:

הנחה	מציאות	התמודדות ב-SuperWaze
הגעות פואסוניות	לא תמיד — שעות שיא יוצרות גלים	חישוב λ נפרד לכל שעה ויום (מטריצת עונתיות)
שירות אקספוננציאלי	בפועל — שירות עצמי מהיר יותר מקופה רגילה	μ נפרד לכל סוג קופה
שרתים זהים	קופאיות שונות בקצב שונה	ממוצע μ על פני דגימות רבות ממזער את השפעת השונות
תור FIFO אחד	בפועל — לקוחות בוחרים תור	המודל מחשב כל סוג קופה בנפרד; LaneSelector ממליץ על הקופה הטובה
מספר קופות ידוע	קופות נפתחות ונסגרות	GMM Clustering מרענן כל 50 דיווחים — מספר הקופות מתעדכן

2.6.5 השילוב בין EWMA ל-M/M/c

השכבה הדו-שכבתית של חיזוי התורים ב-**SuperWaze** פועלת כך:

1. **שכבה א' — EWMA**: תמיד זמינה. מספקת חיזוי כללי (*GlobalEwmaMinutes*) גם כשאין מספיק נתונים לפירוט לכל סוג קופה.

2. **שכבה ב' — M/M/c**: מופעלת כאשר יש מספיק checkout_events ו-checkout_lanes מזוהות. מספקת חיזוי לכל סוג קופה בנפרד.

3. **Fallback:** אם אין מספיק נתונים ל-M/M/c (פחות מ-30 דגימות לסוג קופה) — המערכת חוזרת ל-EWMA. אם גם ל-EWMA אין מספיק נתונים — ערך ברירת מחדל (4 דקות).

גישה דו-שכבתית זו מבטיחה ש**תמיד** יש חיזוי זמין — גם בחנות חדשה ללא היסטוריה — תוך שיפור הדיוק ככל שנצברים יותר נתונים. לפי *Whitt (1999)*, שילוב בין מודלים אנליטיים (M/M/c) לבין מודלים מבוססי נתונים (EWMA) מניב חיזויים מדויקים יותר מכל שיטה בנפרד.

2.6.6 סיכום ביניים

תורת התורים מספקת ל-**SuperWaze** מסגרת מתמטית מוצקה לחיזוי זמני המתנה. מודל M/M/c מאפשר חיזוי מפורט לכל סוג קופה על בסיס מספר הקופות הפעילות (מזוהות ע"י K-Means), קצב הגעת הלקוחות וקצב השירות. השילוב עם EWMA קהילתי מבטיח זמינות חיזוי בכל מצב,

והמנגנון ההירכי (M/M/c → EWMA → default) מספק חיזוי אמין ככל שניתן. בסעיף הבא נסקור את תהליכי עיבוד הנתונים (ETL) שמספקים את תשתית הנתונים למודלים אלו.

2.7 צינורות ETL ועיבוד נתונים

צינור **ETL (Extract, Transform, Load)** הוא תהליך אוטומטי לשליפת נתונים ממקורות חיצוניים, עיבודם והטענתם לבסיס נתונים יעד. תהליכי ETL מהווים את עמוד השדרה של מערכות מידע מונעות-נתונים. סעיף זה סוקר את העקרונות התיאורטיים של ETL, את האתגרים הייחודיים בעיבוד נתוני חוק שקיפות המחירים, ואת הגישות שנבחרו ב-**SuperWaze**.

2.7.1 מבוא לתהליכי ETL

לפי *Kimball & Caserta (2004)*, תהליך ETL מורכב משלושה שלבים עיקריים:

1. **Extract** (שליפה) — איסוף נתונים ממקורות חיצוניים. המקורות יכולים להיות בסיסי נתונים, קבצים, ממשקי API או אתרי אינטרנט. האתגר המרכזי: המקורות שונים בפורמט, באיכות ובזמינות.

2. **Transform** (עיבוד) — ניקוי, נירמול, העשרה ותיקוף הנתונים. כולל: הסרת כפילויות, תיקון שגיאות, המרת פורמטים, חישוב שדות נגזרים. לפי *Rahm & Do (2000)*, שלב זה הוא המורכב ביותר — עד 80% מזמן הפיתוח של צינור ETL מוקדש לו.

3. **Load** (טעינה) — הכנסת הנתונים המעובדים לבסיס הנתונים היעד. הטעינה יכולה להיות מלאה (Full Load — החלפת כל הנתונים) או מצטברת (Incremental / UPSERT — עדכון רק מה שהשתנה).

בנוסף למודל הקלאסי, קיים גם מודל **ELT (Extract, Load, Transform)** שבו הנתונים נטענים תחילה לבסיס הנתונים "כמות שהם" והעיבוד מתבצע בתוך בסיס הנתונים עצמו. לפי *Vassiliadis (2009)*, גישת ELT מתאימה יותר כאשר בסיס הנתונים חזק מספיק לבצע את העיבוד. SuperWaze משתמשת בגישה היברידית: רוב העיבוד (פענוח XML, נירמול שמות) מתבצע ב-Python לפני הטעינה, אך חלק מהחישובים (כגון UPSERT ואינדקסים) מתבצעים בתוך PostgreSQL.

2.7.2 נתונים פתוחים (Open Data)

נתונים פתוחים *(Open Data)* הם נתונים שזמינים לשימוש חופשי ללא הגבלות זכויות יוצרים. לפי *Janssen, Charalabidis & Zuiderwijk (2012)*, נתונים פתוחים מייצרים ערך כלכלי וחברתי באמצעות הנגשת מידע ממשלתי ומוסדי לציבור.

חוק שקיפות המחירים בישראל יצר את אחד ממאגרי הנתונים הפתוחים הגדולים בעולם בתחום הקמעונאות — מיליוני רשומות מחירים ומבצעים, מעודכנים מדי יום, לכל מוצר בכל סניף של כל רשת גדולה. עם זאת, נתונים אלו אינם **"פתוחים"** במובן הקלאסי: הם מפורסמים בפורמטים שונים, ללא API מתועד, ועם חריגות רבות בין הרשתות.

האתגרים העיקריים בעבודה עם נתוני חוק השקיפות:

1. **פורמטים שונים:** כל רשת מפרסמת בפורמט XML שונה במקצת — שמות שדות שונים, מבנה שונה, קידודי תווים שונים.

2. **דחיסה:** הקבצים לרוב דחוסים ב-GZ — נדרש פענוח streaming כדי לא לצרוך זיכרון מופרז.

3. **גודל:** קובץ מחירים של רשת גדולה יכול להכיל מאות אלפי רשומות. עיבוד כל הרשתות דורש טיפול במיליוני שורות.

4. **עברית:** שמות מוצרים בעברית דורשים נירמול מיוחד — הסרת ניקוד, אחידות כיווניות, טיפול במילים נרדפות.

5. **זמינות:** לפעמים קבצים חסרים או פגומים — נדרש מנגנון retry וטיפול בשגיאות.

6. **כתובות URL** משתנות: הרשתות מחליפות כתובות פרסום ללא הודעה מוקדמת — נדרש גילוי אוטומטי.

2.7.3 עקרונות תכנון צינורות ETL

לפי *Kimball & Caserta (2004)*, צינור ETL איכותי צריך לעמוד בעקרונות הבאים:

1. **אידמפוטנטיות** *(Idempotency)* — הפעלת התהליך פעמיים ברציפות צריכה לייצר את אותה תוצאה. ב-SuperWaze: שימוש ב-UPSERT (ON CONFLICT DO UPDATE) מבטיח שטעינה כפולה לא יוצרת כפילויות.

2. **עמידות בתקלות** *(Fault Tolerance)* — כשל ברשת אחת לא צריך לעצור את עיבוד שאר הרשתות. ב-SuperWaze: כל רשת מעובדת בנפרד עם try/except — שגיאה נרשמת בלוג והתהליך ממשיך.

3. **ניטור ולוגים** *(Observability)* — תיעוד מלא של כל הרצה: כמה רשומות עובדו, כמה שגיאות, זמן ריצה. ב-SuperWaze: עדכון שדות LastEtlSuccessAt ו-LastEtlItemCount + רשומות ב-MIGRATION_LOG.

4. **תזמון אוטומטי** *(Scheduling)* — הפעלה ללא התערבות ידנית בשעות שפל. ב-SuperWaze: GitHub Actions מפעיל ב-06:00 שעון ישראל (*nightly-etl.yml*).

5. **עיבוד זרימתי** *(Streaming)* — עיבוד הנתונים תוך כדי קריאה ולא אחרי טעינת הכל לזיכרון. ב-SuperWaze: שימוש ב-iterparse של lxml לעיבוד קבצים גדולים ללא צריכת זיכרון מופרזת.

6. **הפרדת קונפיגורציה** — פרטי כל רשת (שם, פורמט, URL pattern) מוגדרים בקובץ קונפיגורציה ולא בקוד. ב-SuperWaze: *chains_config.json* מגדיר את כל הפרמטרים לכל רשת.

2.7.4 עיבוד שפה טבעית בעברית (Hebrew NLP)

אחד האתגרים הייחודיים של **SuperWaze** הוא נירמול שמות מוצרים בעברית. רשתות שונות מתארות את אותו מוצר בצורות שונות — למשל: "חלב תנובה 3% שומן 1 ליטר", "חלב תנובה 3% 1ל", "חלב 3% תנובה ליטר". מודול ה-*hebrew_nlp.py* מבצע:

1. **נירמול תווים:** הסרת ניקוד, אחידות סימני פיסוק, המרת ספרות.

2. **נירמול יחידות:** "ליטר" ↔ "ל" ↔ "1L"; "גרם" ↔ "ג" ↔ "g" — המרה לפורמט אחיד.

3. **הסרת מילות עצירה:** מילים כמו "של", "עם", "טעם" שאינן תורמות לזיהוי.

4. **ניקוי רווחים ותווים מיוחדים:** רווחים כפולים, תווי Unicode לא תקניים, סימני HTML.

5. **זיהוי יצרן:** חילוץ שם היצרן מתוך שם המוצר כאשר הוא לא מופיע בשדה נפרד.

לפי *Tsarfaty (2006)*, עיבוד טקסט בעברית מורכב יותר משפות אירופיות בשל המורפולוגיה העשירה של השפה (תחיליות, סופיות, ניקוד), כתיבה מימין לשמאל, והיעדר רישיות (case-insensitive באופן טבעי). האתגרים בהקשר של שמות מוצרים פחות חמורים מאשר בטקסט חופשי, אך עדיין דורשים טיפול ייעודי.

2.7.5 איכות נתונים (Data Quality)

איכות הנתונים *(Data Quality)* היא מידת ההתאמה של הנתונים לשימוש המיועד. לפי *Wang & Strong (1996)*, ארבעה ממדים מגדירים איכות נתונים:

ממד	הגדרה	יישום ב-SuperWaze ETL
דיוק (Accuracy)	הנתונים משקפים את המציאות	אימות מחירים (טווח סביר), סינון מוצרים עם ברקוד לא תקני
שלמות (Completeness)	אין נתונים חסרים	בדיקה שלכל מוצר יש ברקוד, שם ומחיר; דיווח על חנויות עם כיסוי נמוך
עקביות (Consistency)	הנתונים אחידים בין מקורות	נירמול hebrew_nlp — אותו מוצר מזוהה גם כשהשם שונה בין רשתות
עדכניות (Timeliness)	הנתונים מעודכנים	ריצה יומית ב-06:00; מעקב LastEtlSuccessAt לכל רשת

2.7.6 ארכיטקטורת ה-ETL ב-SuperWaze

צינור ה-ETL של **SuperWaze** מורכב מארבעה רכיבים עיקריים:

רכיב	קובץ	תפקיד
גילוי כתובות	*fetch_transparency_urls.py*	מגלה אוטומטית את כתובות ה-URL העדכניות של קבצי כל רשת
ייבוא מחירים	*import_pricesfull_stream.py*	פענוח XML/GZ → נירמול → UPSERT ל-store_price_latest
ייבוא מבצעים	*import_promosfull_stream.py*	פענוח מבצעים → מיפוי לסוגי מבצע (5 סוגים) → UPSERT ל-promotions
ייבוא חנויות	*import_storesfull_stream.py*	עדכון רשימת סניפים: שם, כתובת, קוד, פורמט חנות

הרכיבים מתואמים על ידי סקריפט מתזמר *(import_all_chains.py)* שמריץ את כל השלבים לכל רשת ב-*chains_config.json*. התזמון מתבצע ב-GitHub Actions עם מטריצת רשתות — כל רשת רצה ב-job נפרד לבידוד תקלות.

2.7.7 דפוסי עיצוב בצינורות נתונים

לפי *Kleppmann (2017)*, עיצוב צינורות נתונים מודרניים מתבסס על מספר דפוסים:

1. **Batch Processing** — עיבוד נתונים בקבוצות גדולות בזמנים קבועים (לא בזמן אמת). ב-SuperWaze: ה-ETL הלילי הוא תהליך Batch — מעדכן את כל המחירים פעם ביום.
2. **Stream Processing** — עיבוד נתונים ברגע שהם מגיעים. ב-SuperWaze: דיווחי תורים מעובדים בזמן אמת (עדכון EWMA מיידי).
3. **UPSERT Pattern** — "עדכן אם קיים, הוסף אם חדש". מבטיח אידמפוטנטיות — הפעלה כפולה של ה-ETL אינה יוצרת כפילויות.
4. **Schema Versioning** — ניהול שינויים בסכמת בסיס הנתונים באמצעות מיגרציות. ב-SuperWaze: 10 מיגרציות (V001 עד V010) מתועדות ב-MIGRATION_LOG.md.

2.7.8 סיכום ביניים

צינורות ה-ETL של **SuperWaze** מהווים את תשתית הנתונים של המערכת כולה. הם מתמודדים עם אתגרים ייחודיים של נתוני חוק שקיפות המחירים — פורמטים שונים, עברית, קבצים גדולים וכתובות משתנות — תוך שמירה על עקרונות איכות נתונים מוכחים. השילוב של עיבוד Batch (מחירים ומבצעים) עם עיבוד Stream (דיווחי תורים) מאפשר למערכת לספק מידע עדכני ומדויק. הנתונים שנאספים ב-ETL מזינים את כל המודלים — מנוע המבצעים, חיזוי התורים ואלגוריתם האופטימיזציה. בסעיף הבא נסקור את תחום השירותים מבוססי המיקום.

2.8 שירותים מבוססי מיקום

שירותים מבוססי מיקום *(Location-Based Services — LBS)* הם שירותים דיגיטליים המנצלים את המיקום הגיאוגרפי של המשתמש כדי לספק מידע או פונקציונליות מותאמת. סעיף זה סוקר את הטכנולוגיות המרכזיות בתחום — GPS, Geofencing, מיקום פנים-מבנה וחיישני תנועה — ואת יישומן ב-**SuperWaze**.

2.8.1 רקע תיאורטי — שירותים מבוססי מיקום

לפי *Schiller & Voisard (2004)*, שירות מבוסס מיקום מוגדר כשירות מידע הנגיש באמצעות מכשיר נייד דרך רשת סלולרית, ומסוגל להשתמש במיקום הגיאוגרפי של המכשיר. LBS מורכב מחמישה רכיבים:

1. **מכשיר נייד** — הטלפון החכם שמריץ את האפליקציה ומספק גישה לחיישנים.
2. **רשת תקשורת** — מעבירה את נתוני המיקום ואת בקשות המשתמש לשרת.
3. **רכיב מיקום** *(Positioning Component)* — טכנולוגיה שקובעת את מיקום המכשיר (GPS, Wi-Fi, חיישנים).
4. **ספק שירות** — השרת שמעבד את נתוני המיקום ומספק את השירות.
5. **ספק נתונים** — מאגרי המידע הגיאוגרפיים (מפות, נקודות עניין, כתובות חנויות).

לפי *Junglas & Watson (2008)*, שירותים מבוססי מיקום מחולקים לשני סוגים:

1. **Pull-based LBS** — המשתמש יוזם את הבקשה. למשל: "הראה לי חנויות קרובות". ב-SuperWaze: חיפוש חנויות בעיר, השוואת מחירים.
2. **Push-based LBS** — המערכת יוזמת פנייה למשתמש על בסיס מיקומו. למשל: "נראה שהגעת לקופה". ב-SuperWaze: התראת כניסה לחנות (Geofencing), הצעת דיווח תור (StandingDetector).

2.8.2 מערכת מיקום גלובלית (GPS)

GPS (Global Positioning System) היא מערכת לוויינית שמאפשרת קביעת מיקום בכל מקום על פני כדור הארץ. המערכת מורכבת מ-24+ לווייים ומספקת דיוק של 3-5 מטרים בתנאים אופטימליים. לפי *Hofmann-Wellenhof, Lichtenegger & Wasle (2007)*, הדיוק יורד משמעותית בתוך מבנים סגורים (30-50 מטרים) בשל חסימת אותות הלוויין.

ב-GPS, **SuperWaze** משמש לשתי מטרות עיקריות:

1. **חישוב מרחק נסיעה:** המיקום הנוכחי של המשתמש משמש לחישוב מרחק, זמן נסיעה ועלות דלק לכל חנות. כאשר זמין — דרך Google Distance Matrix API; כגיבוי — נוסחת Haversine לחישוב מרחק אוויר.
2. **Geofencing**: זיהוי כניסה ויציאה מאזור חנות (ראה סעיף 2.8.3).

נוסחת **Haversine** מחשבת את המרחק בין שתי נקודות על פני כדור הארץ:

$$d = 2R \cdot \arcsin\left(\sqrt{\left[\sin^2(\Delta\varphi/2) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2(\Delta\lambda/2)\right]}\right)$$

כאשר R = רדיוס כדור הארץ (6,371 ק"מ), φ = קו רוחב, λ = קו אורך. הנוסחה מיושמת ב-*MathHelpers.cs* ומשמשת כ-fallback כאשר Google API אינו זמין.

2.8.3 גידור גיאוגרפי (Geofencing)

Geofencing הוא טכניקה שבה מוגדר אזור גיאוגרפי וירטואלי ("גדר") סביב מיקום פיזי, והמכשיר מזהה באופן אוטומטי כניסה ויציאה מהאזור. לפי *Namiot & Sneps-Sneppe (2013)*, Geofencing מהווה את הבסיס לשירותי Push-based LBS — המערכת יכולה להגיב למיקום המשתמש מבלי שהמשתמש יבצע פעולה כלשהי.

ב-**SuperWaze**, Geofencing מיושם באפליקציית המובייל *(GeofencingService.ts)* באמצעות ה-API של Expo Location. לכל חנות מוגדר אזור מעגלי (רדיוס ברירת מחדל: 50 מטר). כאשר המשתמש:

1. **נכנס לאזור:** נפתחת רשומת UserStoreVisit עם זמן כניסה וקואורדינטות. האפליקציה מפעילה את StandingDetector לזיהוי עמידה בקופה.
2. **יוצא מהאזור:** הביקור נסגר עם זמן יציאה ומשך שהייה כולל. אם המשתמש לא דיווח על תור — מוצעת לו שאלה.
3. **נשאר באזור:** StandingDetector עוקב אחרי תנועת המשתמש וזיהוי עמידה ממושכת (ראה סעיף 2.8.4).

אתגרי **Geofencing**:

אתגר	תיאור	פתרון ב-SuperWaze
צריכת סוללה	ניטור GPS רציף מרוקן סוללה	שימוש ב-Significant Location Changes במקום מעקב רציף
דיוק בסביבה עירונית	מבנים גבוהים גורמים לשגיאות מיקום	רדיוס גדר גדול יחסית (50 מטר) מפצה על אי-דיוקים
הרשאות משתמש	נדרשת הרשאת מיקום "תמיד" (Always)	הסבר למשתמש למה ההרשאה נדרשת; fallback לדיווח ידני
False positives	זיהוי כניסה שגוי (למשל, חניה סמוכה)	אישור מהמשתמש לפני פתיחת ביקור; מנגנון debounce

2.8.4 חיישני תנועה וזיהוי פעילות

בתוך מבנה סגור, GPS אינו מספיק מדויק לזיהוי מיקום ספציפי (כגון קופות). הפתרון של **SuperWaze** מבוסס על חיישן תאוצה (Accelerometer) — חיישן מובנה בכל סמארטפון שמודד תאוצה בשלושה צירים (x, y, z).

לפי *Kwapisz, Weiss & Moore (2011)*, ניתן לזהות סוגי פעילות שונים (הליכה, ריצה, עמידה, ישיבה) מנתוני תאוצה בלבד. הגישה מבוססת על חישוב magnitude של וקטור התאוצה:

$$\textbf{magnitude} = \sqrt{(x^2 + y^2 + z^2)}$$

כאשר המכשיר סטטי (עומד), ה-magnitude שווה בקירוב לכוח הכבידה (9.81 מ/ש"ר). תנועה כלשהי גורמת לסטייה מערך זה.

ב-**SuperWaze**, ה-**StandingDetector** מיישם זיהוי עמידה בשתי רמות:

פרמטר	**מצב יד** (hand)	**מצב עגלה** (cart)
סף תנועה	0.8 מ/ש"ר	1.5 מ/ש"ר (גבוה יותר — עגלה יוצרת רעש)
תדירות דגימה	500ms	500ms
סף עמידה (ברירת מחדל)	90 שניות	90 שניות
מתודולוגיה	סטטי אם |magnitude - 9.81| < סף	סטטי אם |magnitude - 9.81| < סף

2.8.5 שיקול Bayesian Prior בזיהוי קופות

אחד החידושים של **SuperWaze** הוא שימוש ב-**Bayesian Prior** לשיפור דיוק זיהוי הקופה. הרעיון מבוסס על משפט Bayes: ההסתברות שהמשתמש נמצא ליד קופה תלויה לא רק בנתוני החיישן (עמידה ממושכת) אלא גם בהקשר — שעה ביום ויום בשבוע.

למשל: עמידה של 90 שניות בשעה 12:00 ביום שישי (שעת שיא) סבירה הרבה יותר כ"קופה" מאותה עמידה בשעה 06:00 ביום שלישי (חנות כמעט ריקה — ייתכן שהמשתמש פשוט עומד ליד מדף). ה-**Prior** מיוצג כמטריצת הסתברויות [hourOfDay][dayOfWeek] — 7×24 = 168 ערכים — שמתעדכנת מנתוני checkout_events אמיתיים.

ערך Prior גבוה (למשל, 0.4 ביום שישי בשעה 12:00) מוריד את סף הזמן הנדרש לזיהוי; ערך נמוך (למשל, 0.05 ביום שלישי בשעה 06:00) מעלה את הסף. כך המערכת רגישה יותר בשעות שיא ושמרנית יותר בשעות שפל — מה שמפחית false positives.

לפי *Murphy (2012)*, גישה בייסיאנית מתאימה במיוחד כאשר יש מידע מוקדם (prior knowledge) על ההסתברות של אירועים — ובמקרה של סופרמרקטים, דפוסי העומס הם צפויים ומחזוריים.

2.8.6 מיקום פנים-מבנה (Indoor Positioning)

מיקום פנים-מבנה *(Indoor Positioning System — IPS)* הוא אתגר פתוח בתחום ה-LBS. לפי *Liu, Darabi, Banerjee & Liu (2007)*, הטכנולוגיות העיקריות כוללות:

טכנולוגיה	דיוק	עלות	מתאימה ל-
Wi-Fi Fingerprinting	2-5 מטר	נמוכה (תשתית קיימת)	מבנים גדולים עם רשת Wi-Fi
Bluetooth Beacons	1-3 מטר	בינונית (צריך להתקין)	חנויות קמעונאיות
UWB (פס רחב)	10-30 ס"מ	גבוהה	מפעלים, חדרים נקיים

זיהוי פעילות, לא מיקום מדויק	אפס (מובנה בטלפון)	משתנה (סחף)	חיישני תנועה (IMU)

SuperWaze בחרה בגישה שאינה דורשת תשתית חיצונית — זיהוי פעילות באמצעות חיישן תאוצה בלבד. הגישה אינה מספקת מיקום מדויק בתוך החנות, אלא מזהה **מצב** (עמידה ממושכת = קופה אפשרית). לאחר מכן, מיקום נרמל (normX, normY) מחושב יחסית לכניסה ויציאה מהחנות ומשמש לאשכול קופות (K-Means). גישה זו מאפשרת זיהוי קופות ללא צורך בהתקנת Beacons או כל תשתית נוספת בחנות.

2.8.7 ניווט פנים-חנות מבוסס אזורים

בעוד שמיקום מדויק בתוך חנות דורש תשתית יקרה, ניווט **מבוסס אזורים** *(Zone-Based Navigation)* מספק ערך משמעותי ללא תשתית כלל. הרעיון: החנות מחולקת לאזורים לוגיים (ירקות, מוצרי חלב, שימורים, קופות...) וכל מוצר ממופה לאזור. המערכת מחשבת את הסדר האופטימלי של ביקור באזורים.

ב-**SuperWaze**, הניווט מיושם באמצעות:

1. **ZoneCatalog** — טבלת התייחסות שממפה כל קטגוריית מוצר לאזור לוגי (ZoneCode) עם סדר הצגה (AisleOrder).
2. **StoreZones** — מפת האזורים של כל סניף, כולל קואורדינטות הצגה (MapX, MapY) וסדר הצגה.
3. **RouteOptimizer** — אלגוריתם שמסדר את האזורים הנדרשים (לפי רשימת הקניות) בסדר יעיל מכניסה, דרך כל האזורים, עד הקופות.
4. **NavigationPlanService** — מרכיב את הכל: לכל צעד — אזור, שם, רשימת מוצרים לאסוף, וזמן הליכה משוער.

לפי *Hui, Bradlow & Fader (2009)*, מסלול קנייה יעיל מפחית את זמן השהייה בחנות ב-15%-25% ומשפר את שביעות רצון הצרכן. מחקרם הראה שרוב הלקוחות אינם מתכננים את מסלולם ומבזבזים זמן רב על חזרות בין מעברים.

2.8.8 סיכום ביניים

שירותים מבוססי מיקום מהווים רכיב מרכזי ב-**SuperWaze**. המערכת משלבת שלוש רמות של שירותי מיקום: GPS לחישוב מרחק נסיעה, Geofencing לזיהוי כניסה ויציאה מחנויות, וחיישן תאוצה עם Bayesian Prior לזיהוי עמידה בקופה. הגישה המבוססת אזורים לניווט פנים-חנות מספקת ערך משמעותי ללא תשתית יקרה. החידוש במיקום נרמל ואשכול K-Means מאפשר לזהות קופות פיזיות ללא Beacons — מה שמוריד את מחסום הכניסה לאפס ומאפשר פריסה בכל חנות. בסעיף הבא נסקור את תחום אינטראקציה אדם-מחשב ומערכות קהילתיות.

2.9 אינטראקציה אדם-מחשב ומערכות קהילתיות

סעיף זה סוקר שני תחומים משלימים החיוניים להצלחת **SuperWaze**: אינטראקציה אדם-מחשב (HCI) — העוסקת בתכנון ממשקים יעילים ונעימים לשימוש, ומערכות קהילתיות (Crowdsourcing) — העוסקות בגיוס קהילת משתמשים לאיסוף נתונים ושיפור המערכת.

2.9.1 אינטראקציה אדם-מחשב (HCI) — רקע

אינטראקציה אדם-מחשב *(Human-Computer Interaction — HCI)* היא תחום רב-תחומי העוסק בתכנון, הערכה ומימוש מערכות מחשוב אינטראקטיביות לשימוש אנושי. לפי *Dix, Finlay, Abowd & Beale (2004)*, המטרה המרכזית של HCI היא ליצור מערכות שהן שמישות *(usable)*, יעילות ומספקות חוויה חיובית למשתמש.

המושג **חוויית משתמש** *(User Experience — UX)*, שהוגדר על ידי *Norman (2013)*, מרחיב את HCI מעבר לשמישות בלבד: הוא כולל את כל ההיבטים של האינטראקציה בין המשתמש למוצר — הרגשות, התפיסות, התגובות וההתנהגויות שמתרחשות לפני, במהלך ואחרי השימוש.

2.9.2 עקרונות שמישות (Usability)

Nielsen (1994) הגדיר עשרה היוריסטיקות לשמישות שהפכו לסטנדרט בתעשייה. להלן ההיוריסטיקות הרלוונטיות ביותר ל-**SuperWaze** ויישומן:

1. **נראות מצב המערכת** *(Visibility of System Status)* — המשתמש צריך לדעת תמיד מה קורה. ב-SuperWaze: סימוני IsBestNet, IsCheapest, IsFastest על כרטיסי חנויות; רמת ביטחון (high/medium/low) ליד חיזוי התורים.

2. **התאמה בין המערכת לעולם האמיתי** *(Match Between System and Real World)* — שימוש בשפה ומושגים מוכרים לצרכן. ב-SuperWaze: "זמן תור", "עלות נסיעה", "חסכת ₪X" — לא מונחים טכניים.

3. **שליטת המשתמש וחופש** *(User Control and Freedom)* — אפשרות לבטל פעולות. ב-SuperWaze: מחיקת פריטים מרשימה, ביטול דיווח תור, שינוי העדפות.

4. **עקביות ותקנים** *(Consistency and Standards)* — אותו מושג מוצג באותו אופן בכל מקום. ב-SuperWaze: עיצוב Zustand stores אחיד, רכיבי UI משותפים (Button, Toast).

5. **מניעת שגיאות** *(Error Prevention)* — עדיף למנוע שגיאה מלטפל בה. ב-SuperWaze: Validation בשרת ובלקוח (PredictRequestValidator); אישור לפני דיווח תור.

6. **גמישות ויעילות** *(Flexibility and Efficiency)* — קיצורים למשתמשים מנוסים. ב-SuperWaze: עיר מועדפת, סוג רכב ברירת מחדל, רשימת קניות קבועה.

7. **עיצוב מינימליסטי** *(Aesthetic and Minimalist Design)* — הצגת מידע רלוונטי בלבד. ב-SuperWaze: כרטיס חנות מציג עלות נטו בולטת, פירוט מבצעים רק בלחיצה, תורים לפי סוג קופה בנפרד.

2.9.3 עיצוב למובייל (Mobile UX)

עיצוב לאפליקציה ניידת שונה מעיצוב לאתר וב. לפי *Hoober & Berkman (2011)*, האתגרים המרכזיים כוללים:

1. **מסך קטן:** מידע צריך להיות מרוכז ומדורג — התוכן החשוב ביותר למעלה. ב-SuperWaze: עלות נטו בולטת בראש כל כרטיס חנות.
2. **אינטראקציה באגודל:** אלמנטים אינטראקטיביים צריכים להיות בגודל מספיק (לפחות 44×44 פיקסלים). כפתורי דיווח תור ואישור קופה מעוצבים בהתאם.
3. **הקשר שימוש:** המשתמש משתמש באפליקציה תוך כדי תנועה — בדרך לסופר, בתוך החנות. ממשק צריך להיות מהיר וברור גם ללא ריכוז מלא.
4. **חיבור רשת לא יציב:** בתוך מבנה — חיבור עלול להיות חלש. ב-SuperWaze: רשימת קניות נשמרת מקומית; StandingDetector פועל ללא רשת.
5. **צריכת סוללה:** חיישנים ו-GPS צורכים סוללה. ב-SuperWaze: Geofencing בשיטת Significant Changes; Accelerometer בדגימה של 500ms (לא 60Hz).

2.9.4 עיצוב ממשק בעברית (RTL)

עיצוב ממשק בשפה שנכתבת מימין לשמאל *(Right-to-Left — RTL)* מציב אתגרים ייחודיים. לפי *W3C (2022)*, עיצוב RTL דורש:

1. **שיקוף אופקי:** כיוון הקריאה, סדר אלמנטים, אייקונים כיווניים (חצים) — הכל מתהפך.
2. **טקסט דו-כיווני** *(BiDi)*: כאשר עברית ואנגלית מעורבבות (שכיח מאוד ב-SuperWaze — שמות רשתות באנגלית, מונחים טכניים) — יש לטפל בכיוון כל קטע טקסט.
3. **יישור טקסט:** ברירת מחדל ימין; מספרים ומונחים אנגליים נשארים משמאל-לימין בתוך משפט עברי.
4. **טפסים:** תוויות מימין, שדות קלט משמאל (או מימין עם placeholder בעברית).

ב-**SuperWaze**, תמיכת RTL מיושמת באמצעות Tailwind CSS עם מחלקות rtl: ייעודיות בווב, ו-I18nManager.forceRTL(true) ב-React Native. כל רכיבי ה-UI (Button, Toast, AppShell) תומכים ב-RTL באופן מלא.

2.9.5 מערכות קהילתיות (Crowdsourcing)

Crowdsourcing הוא תהליך שבו ארגון פונה לקהל רחב ("ההמון") לביצוע משימות שבעבר בוצעו על ידי מומחים או עובדים. המונח הוטבע על ידי *Howe (2006)* והוא מתאר מודל שבו הערך נוצר על ידי קהילת המשתמשים עצמם.

לפי *Brabham (2013)*, ישנם ארבעה סוגים עיקריים של Crowdsourcing:

סוג	תיאור	דוגמאות
איסוף ידע (Knowledge Discovery)	ההמון אוסף ומארגן מידע	Wikipedia, OpenStreetMap

פתרון בעיות (Distributed Problem Solving)	ההמון פותר בעיות מורכבות	Kaggle, Foldit
יצירת תוכן (Peer Production)	ההמון יוצר תוכן או מוצרים	YouTube, GitHub
איסוף נתוני חיישנים (Participatory Sensing)	ההמון משתף נתונים מחיישני הטלפון	**Waze, SuperWaze**

SuperWaze משתייכת בעיקר לקטגוריית **Participatory Sensing** — משתמשים משתפים נתוני תורים מחיישני הטלפון שלהם (מיקום, תאוצה) ומדיווחים ידניים. הנתונים הללו מזינים את מודלי החיזוי ואת זיהוי הקופות.

2.9.6 Waze כמודל השראה

אפליקציית הניווט **Waze** (שפותחה בישראל ונרכשה על ידי Google ב-2013) היא הדוגמה המוצלחת ביותר ל-Participatory Sensing בהקשר של תחבורה. לפי *Silva, Vaz de Melo, Almeida & Loureiro (2013)*, הצלחת Waze מבוססת על שלושה עקרונות:

1. **איסוף פסיבי:** רוב הנתונים נאספים באופן אוטומטי (מהירות נסיעה = עומס תנועה) — ללא מאמץ מהמשתמש.
2. **דיווח אקטיבי:** המשתמשים מדווחים על אירועים (תאונה, מחסום, מכמונת מהירות) — תורמים ערך ייחודי שלא ניתן לאסוף אוטומטית.
3. **תגמול חברתי:** נקודות, רמות ודירוגים מעודדים השתתפות מתמשכת.

SuperWaze מאמצת את אותו מודל — אך במקום תנועה בכבישים, היא מיישמת אותו על תורים בסופרמרקטים:

מנגנון	SuperWaze	Waze
איסוף פסיבי (Accelerometer)	עמידה ממושכת → תור בקופה	מהירות נסיעה → עומס תנועה
דיווח אקטיבי	דיווח על רמת עומס בקופות	דיווח על תאונה/מחסום
K-Means Clustering	זיהוי קופות מ-checkout_events	זיהוי כבישים מצילומי GPS
מודל חיזוי (EWMA / M/M/c)	זמן המתנה משוער — Wq	זמן הגעה משוער — ETA

2.9.7 אתגרים במערכות קהילתיות

לפי *Kittur et al. (2013)*, מערכות Crowdsourcing מתמודדות עם מספר אתגרים מהותיים:

אתגר	תיאור	פתרון ב-SuperWaze
איכות נתונים	דיווחים שגויים, לא מדויקים או זדוניים	סינון Weighted Double MAD — חריגים מסוננים אוטומטית
מסה קריטית	המערכת זקוקה למספר מינימלי של משתמשים פעילים	ערכי ברירת מחדל כשאין מספיק נתונים; Bayesian Prior
מוטיבציה	למה שמשתמשים ידווחו? מה	דיווח אוטומטי (StandingDetector) —

	התמריץ?	מאמץ אפסי; הצעת **"הגעת לקופה?"**
פרטיות	איסוף מיקום ונתוני חיישנים מעורר חששות פרטיות	הסכמה מפורשת; דיווחי תורים אנונימיים; מיקום נרמל (לא מוחלט)
בעיית הביצה והתרנגולת	ללא משתמשים — אין נתונים; ללא נתונים — אין ערך למשתמשים	**השוואת מחירים ומבצעים** עובדת ללא דיווחי קהילה — חיזוי תורים הוא ערך נוסף

2.9.8 מודל קבלת טכנולוגיה (TAM)

מודל קבלת הטכנולוגיה *(Technology Acceptance Model — TAM)* של *Davis (1989)* מנבא את הנכונות של משתמשים לאמץ טכנולוגיה חדשה. המודל מבוסס על שני גורמים:

1. **תועלת נתפסת** *(Perceived Usefulness)* — עד כמה המשתמש מאמין שהמערכת תשפר את ביצועיו. ב-SuperWaze: חיסכון כספי מוכח (הצגת "חסכת ₪X"), חיסכון בזמן (ניווט + בחירת קופה מהירה).

2. **קלות שימוש נתפסת** *(Perceived Ease of Use)* — עד כמה המערכת קלה לשימוש. ב-SuperWaze: המלצה בלחיצה אחת, דיווח תור אוטומטי, ניווט פשוט.

לפי מחקר מעקב של *Venkatesh & Davis (2000)*, גורמים נוספים המשפיעים על אימוץ כוללים: השפעה חברתית (חברים שמשתמשים), רלוונטיות למשימה (האם עוזרת בקנייה?), ואיכות הפלט (האם ההמלצות טובות?). SuperWaze מתמודדת עם כל הגורמים: מנגנון קהילתי (השפעה חברתית), התמקדות בקנייה (רלוונטיות), ושקיפות מלאה בחישוב (איכות).

2.9.9 סיכום פרק 2

סקירת הספרות בפרק זה הניחה את התשתית המושגית לפרויקט **SuperWaze**. הפרק סקר תשעה תחומי ידע מרכזיים:

1. **מערכות מידע** — סיווג SuperWaze כ-DSS, הפחתת עלויות חיפוש ואסימטריית מידע.
2. **מתודולוגיות פיתוח** — בחירה מנומקת ב-Scrum עם 14 ספרינטים ו-CI/CD.
3. **ארכיטקטורות תוכנה** — מונוליט מודולרי, REST API, SignalR, DI.
4. **מערכות השוואת מחירים** — דור שלישי, חוק שקיפות, אתגרים ייחודיים לשוק מזון.
5. **מודלים סטטיסטיים** — EWMA לחיזוי, Weighted Double MAD לסינון.
6. **תורת התורים** — מודל M/M/c לחיזוי מפורט לכל סוג קופה.
7. **צינורות ETL** — עקרונות עיצוב, נתונים פתוחים, NLP בעברית.
8. **שירותים מבוססי מיקום** — GPS, Geofencing, חיישן תאוצה, Bayesian Prior, ניווט מבוסס אזורים.
9. **HCI וקהילה** — עקרונות שמישות, עיצוב מובייל ו-RTL, Crowdsourcing ומודל TAM.

בפרק הבא (פרק 3 — חקר מצב קיים) ננתח את הפתרונות הקיימים בשוק הישראלי ונבחן כיצד **SuperWaze** מתמודדת עם הפערים שהם משאירים.

פרק 3: חקר מצב קיים

פרק זה מציג ניתוח של הפתרונות הקיימים בשוק הישראלי להשוואת מחירים בסופרמרקטים, מציג תוצאות סקר צרכנים שנערך, ומסכם את הפערים ש**SuperWaze** נועדה למלא. הניתוח מבוסס על בדיקת האפליקציות המרכזיות בתחום, סקר צרכנים וטבלת השוואה מקיפה.

3.1 ניתוח מתחרים

להלן ניתוח מפורט של ארבע פתרונות מרכזיים בתחום השוואת מחירים וקניות חכמות בישראל:

3.1.1 **Pricez** (פרייסז)

Pricez היא אפליקציית השוואת מחירים מובילה המבוססת על נתוני חוק שקיפות המחירים. האפליקציה מאפשרת לחפש מוצר בודד או לבנות רשימת קניות ולראות את המחיר ברשתות שונות.

יתרונות:

1. ממשק נקי ופשוט לחיפוש מוצרים.
2. תמיכה במספר גדול של רשתות.
3. עדכון יומי של מחירים.

חסרונות:

1. השוואה מבוססת על מחיר מדף בלבד — אינה מביאה בחשבון מבצעים וכפל הנחות.
2. אין חיזוי תורים או מידע על עומסים בקופות.
3. אין חישוב עלות נסיעה — החנות הזולה ביותר עשויה להיות רחוקה.
4. אין ניווט פנים-חנות.
5. אין תמיכה בסוגי רכב שונים.

3.1.2 מסלול הזול

"מסלול הזול" הוא אתר אינטרנט המאפשר להשוות מחירי סל קניות בין רשתות. האתר מאפשר להזין רשימת מוצרים ולראות איזו רשת מציעה את הסל הזול ביותר.

יתרונות:

1. השוואה ברמת סל שלם.
2. ממשק ווב נגיש.
3. תמיכה במספר רשתות.

חסרונות:

1. השוואה לפי מחיר מדף בלבד — ללא מבצעים.

2. אין אפליקציית מובייל — אתר בלבד.

3. אין חיזוי תורים, ניווט או חישוב עלות נסיעה.

4. לא תומך בסוגי רכב שונים.

3.1.3 שופרסל אונליין

שופרסל אונליין היא חנות האינטרנט של רשת שופרסל — הרשת הגדולה בישראל. היא מאפשרת הזמנת משלוחים וקנייה מקוונת, אך מוגבלת לרשת אחת בלבד.

יתרונות:

1. חוויית קנייה מלאה — הזמנה, משלוח ואיסוף במקום אחד.

2. מידע מדויק על מלאי וזמינות מוצרים.

3. מבצעי מועדון משולבים במחיר.

חסרונות:

1. מוגבלת לרשת שופרסל בלבד — אינה משווה בין רשתות.

2. אינה מספקת השוואת מחירים מול רשתות אחרות.

3. אין חיזוי תורים, ניווט פנים-חנות או חישוב עלות נסיעה.

4. אין תמיכה בכפל מבצעים (סטאקינג).

3.1.4 חצי חינם / יינות ביתן אונליין

רשתות נוספות מציעות אפליקציות קנייה מקוונות דומות לשופרסל אונליין — מוגבלות לרשת אחת, עם מבצעי מועדון. החיסרונות זהות לשל שופרסל אונליין: אין השוואה בין רשתות, אין חיזוי תורים, אין ניווט ואין חישוב עלות נסיעה.

3.2 טבלת השוואה מקיפה

הטבלה הבאה משווה את היכולות של הפתרונות הקיימים מול היכולות של SuperWaze:

יכולת	Pricez	מסלול הזול	שופרסל אונליין	SuperWaze
השוואת מחירי מדף	✓	✓	✗	✓
השוואת סל שלם	~	✓	✗	✓
התחשבות במבצעים	✗	✗	~	✓
כפל מבצעים (Stacking)	✗	✗	✗	✓
חיזוי תורים (EWMA)	✗	✗	✗	✓
חיזוי לכל סוג קופה (M/M/c)	✗	✗	✗	✓
זיהוי קופות אוטומטי	✗	✗	✗	✓
חישוב עלות נסיעה	✗	✗	✗	✓

תמיכה בסוגי רכב שונים	✗	✗	✗	✓
ניווט פנים-חנות	✗	✗	✗	✓
עלות נטו (סל+נסיעה+זמן)	✗	✗	✗	✓
אפליקציית מובייל	✓	✗	✓	✓
דיווח תור קהילתי	✗	✗	✗	✓
Geofencing	✗	✗	✗	✓
עדכונים בזמן אמת (SignalR)	✗	✗	✗	✓

כפי שעולה מהטבלה, **SuperWaze** היא הפתרון היחיד שמציע מענה לכל 15 היכולות שנבדקו. הפער המרכזי הוא בתחום החיזוי (תורים, זיהוי קופות, עלות נטו) ובתחום המבצעים (כפל הנחות, 5 סוגי מבצע).

3.3 סקר צרכנים

לצורך אימות הצורך במערכת כמו SuperWaze, נערך סקר מקוון בקרב 80 נשאלים — צרכנים באזור אשדוד ובאר שבע. הסקר הופץ דרך Google Forms וכלל 12 שאלות.

3.3.1 מאפייני המדגם

מאפיין	תוצאה
מספר נשאלים	78
טווח גילאים	18-65
מגדר	נשים 62%, גברים 38%
אזור	אשדוד ובאר שבע
תדירות קנייה	פעם בשבוע (45%), 2-3 פעמים (35%), אחר (20%)

3.3.2 ממצאים עיקריים

להלן הממצאים המרכזיים שעלו מהסקר:

1. **87% מהנשאלים מרגישים שמחירי המזון בישראל גבוהים** — מאשר את הצורך בכלי השוואה.
2. **72% אינם משווים מחירים לפני קנייה** — למרות שרוב הצרכנים מודעים לבעיה: הם פונים לחנות הקרובה או המוכרת בלי לבדוק אם היא אכן הזולה ביותר.
3. **68% מעוניינים בתורים ארוכים בקופות** — במיוחד בימי שישי ולפני חגים. מחק זה מחזק את הצורך בחיזוי תורים.
4. **81% היו משתמשים באפליקציה שמשלבת השוואת מחירים עם חיזוי תורים** — אישור ישיר לקונספט המרכזי של SuperWaze.

5. **64% היו מוכנים לדווח על תורים** — אם הדיווח היה קל ומהיר (1-2 לחיצות). מחזק את הבחירה בדיווח אוטומטי (StandingDetector).

6. **76% היו רוצים לראות את עלות הנסיעה בהשוואה** — לא רק מחיר אלא גם דלק וזמן.

7. **58% היו מעוניינים בניווט בתוך החנות** — "לפעמים אני מסתובב ולא מוצא מוצרים". מחזק את הצורך בניווט פנים-חנות.

8. **91% משתמשים בסמארטפון לקניות** — מאשר את ההחלטה לפתח אפליקציית מובייל.

3.3.3 שאלות פתוחות — תובנות נבחרות

הסקר כלל גם שאלות פתוחות שהעלו תובנות מעניינות:

1. *"אני תמיד הולך לשופרסל כי זה הכי קרוב, אבל אולי ברמי לוי יותר זול"* — מחזק את הצורך בחישוב עלות נסיעה.

2. *"ביום שישי אין מצב לעמוד בתור. אם הייתי יודע כמה ממתינים — הייתי בוחר חנות אחרת"* — מחזק את הצורך בחיזוי תורים.

3. *"אני משלם על מבצעי מועדון אבל האפליקציות לא מחשבות אותם"* — מחזק את הצורך בכפל מבצעים (Stacking) בחישוב.

3.4 זיהוי פערים והזדמנות

על בסיס ניתוח המתחרים והסקר, זוהו הפערים המרכזיים שSuperWaze נועדה למלא:

פתרון SuperWaze	מצב קיים	פער
PromoEngine עם 5 סוגים + Stacking	אף אפליקציה לא מחשבת כפל מבצעים	התעלמות ממבצעים
EWMA + M/M/c לכל סוג קופה	אין שום אפליקציה שמספקת חיזוי	חוסר מידע על תורים
עלות נטו = סל + נסיעה + זמן	השוואה לפי מחיר בלבד	התעלמות מעלות נסיעה
ניווט מבוסס אזורים + אופטימיזציית מסלול	הצרכן מתקשה למצוא מוצרים לבד	אין ניווט בחנות
StandingDetector + K-Means + LaneClassifier	אין ground truth	אין זיהוי קופות אוטומטי
בנזין / חשמלי / היברידי / הולך רגל	עלות דלק זהה לרכב חשמלי	אין תמיכה ברכב שונים

3.5 סיכום

חקר המצב הקיים חושף את הפתרונות הקיימים בתחום ומזהה פערים משמעותיים בשישה תחומים מרכזיים: חיזוי תורים, התחשבות במבצעים, עלות נסיעה, ניווט בחנות וזיהוי קופות. סקר הצרכנים אישר את הצורך בכל אחד מהפיצ'רים הללו — 87% מרגישים שהמחירים גבוהים,

81% רוצים שילוב מחירים עם תורים, ו-64% מוכנים לדווח אם זה קל. ממצאים אלה מהווים את הבסיס לאפיון המערכת בפרק 4 ולניתוח הדרישות בפרק 5.

פרק 4: אפיון המערכת

פרק זה מציג את אפיון מערכת **SuperWaze** — הגדרת מטרות המערכת, גבולותיה, האילוצים הפועלים עליה, ותיאור מפורט של תהליכי המחשב המרכזיים. האפיון מבוסס על ניתוח הדרישות שעלו מסקר המשתמשים (פרק 3), מחקר המצב הקיים, וההבנה הטכנולוגית שנרכשה בסקירת הספרות (פרק 2).

4.1 מטרות המערכת

מטרות המערכת מחולקות לשתי קטגוריות: מטרות עסקיות (ערך למשתמש) ומטרות טכניות (אפשור מימוש).

4.1.1 מטרות עסקיות

1. לאפשר לצרכן להשוות מחירי סל קניות בין כל רשתות השיווק הגדולות בישראל בלחיצה אחת.
2. להביא בחשבון מבצעים פעילים (כולל כפל מבצעים) בחישוב עלות הסל, כך שההמלצה תשקף את המחיר האמיתי לתשלום.
3. לספק לצרכן תחזית זמני המתנה בתורים לכל סוג קופה (רגילה, מהירה, שירות עצמי) על בסיס נתוני אמת.
4. לחשב עלות נסיעה מותאמת לסוג הרכב של המשתמש (בנזין, חשמלי, היברידי, הולך רגל) ולשקלל אותה בהמלצה.
5. לספק המלצה אופטימלית לחנות הכדאית ביותר תוך שקלול עלות סל, עלות נסיעה ועלות זמן המתנה.
6. לנווט את המשתמש בתוך החנות בצורה יעילה על בסיס מפת אזורים.
7. לעודד השתתפות קהילתית בדיווח תורים באמצעות זיהוי אוטומטי של שהייה בחנות.

4.1.2 מטרות טכניות

1. עדכון אוטומטי יומי של מחירים ומבצעים מכלל הרשתות באמצעות צינורות ETL מנתוני חוק שקיפות המחירים.
2. מימוש מודל חיזוי תורים דו-שכבתי: EWMA קהילתי לחיזוי כללי, ומודל M/M/c מתורת התורים לחיזוי מפורט לכל סוג קופה.
3. זיהוי אוטומטי של מיקומי קופות פיזיים מדיווחי משתמשים באמצעות K-Means Clustering עם בחירת K אופטימלי (BIC).
4. זיהוי עמידה ממושכת בקופה באמצעות חיישן תאוצה (Accelerometer) ו- Bayesian Prior באפליקציה הניידת.
5. הבטחת ביצועים גבוהים באמצעות שכבת מטמון (Redis), אינדקסים מותאמים ושאילתות מאוחדות (ביטול בעיית N+1).

6. אבטחת המערכת: JWT עם Refresh Tokens ב-HttpOnly Cookies, הגבלת קצב בקשות, ומניעת IDOR.

7. פריסה אוטומטית באמצעות Docker Compose, nginx כ-reverse proxy, ו-CI/CD ב-GitHub Actions.

4.2 גבולות המערכת

הגדרת גבולות המערכת מבהירה מה נכלל במערכת ומה חורג ממנה. גבולות ברורים מסייעים בניהול ציפיות ובתכנון מדויק.

4.2.1 בתוך גבולות המערכת

1. השוואת מחירים בין רשתות שיווק בישראל (שופרסל, רמי לוי, מגה, יינות ביתן, חצי חינם, ויקטורי, אושר עד, טיב טעם ועוד) על בסיס נתוני חוק שקיפות המחירים.

2. ייבוא אוטומטי של מחירים ומבצעים באמצעות צינורות ETL עם תזמון יומי.

3. מנוע מבצעים (Promo Engine) התומך בחמישה סוגי מבצעים: הורדת מחיר (PRICE_CUT), אחוז הנחה (PCT_OFF), רב-קנה (MULTI_BUY), קנה-קבל (BUY_GET) ואחר (OTHER), עם תמיכה בכפל מבצעים (Stacking).

4. חיזוי זמני המתנה בתורים על בסיס דיווחי קהילה (EWMA) ומודל תורת התורים (M/M/c) לכל סוג קופה.

5. זיהוי אוטומטי של מיקומי קופות באמצעות K-Means Clustering ושיוך סוג קופה אוטומטי (LaneClassifier).

6. חישוב עלות נסיעה על בסיס Google Distance Matrix API (או נוסחת Haversine כגיבוי) עם תמיכה בסוגי רכב שונים.

7. ניווט פנים-חנות מבוסס אזורים (zone-based) עם אופטימיזציית מסלול.

8. ניהול רשימות קניות עם מעקב סטטוס פריטים (pending, checked, not_found).

9. מעקב ביקורים בחנויות (UserStoreVisit) עם זיהוי כניסה ויציאה אוטומטי (Geofencing).

10. ממשק ווב (Next.js) ואפליקציית מובייל (React Native/Expo).

11. מערכת אימות והרשאות מלאה (הרשמה, התחברות, חידוש טוקנים).

4.2.2 מחוץ לגבולות המערכת

1. רכישה מקוונת או תשלום דרך האפליקציה — המערכת מספקת מידע בלבד ואינה מהווה חנות מקוונת.

2. ניווט GPS לחנות — המערכת מחשבת מרחק וזמן נסיעה אך אינה מספקת ניווט בזמן אמת (משאירה זאת ל-Waze/Google Maps).

3. סריקת ברקודים פיזית — הוספת מוצרים מתבצעת מתוך קטלוג דיגיטלי ולא ע"י סריקה.

4. ניהול מלאי בחנויות — המערכת מתבססת על נתוני מחירים ומבצעים בלבד ואינה מציגה זמינות מלאי.

5. אינטגרציה ישירה עם מערכות קופה (POS) של הרשתות — נתוני התורים מגיעים מדיווחי משתמשים בלבד.

6. תמיכה בשפות מלבד עברית — הממשק והנתונים בעברית בלבד.

4.3 אילוצים

4.3.1 אילוצים טכניים

אילוץ	תיאור	השפעה על המערכת
תלות בנתוני חוק השקיפות	נדרשים צינורות ETL נפרדים לכל רשת עם טיפול בחריגות	הנתונים מתפרסמים בפורמט XML שונה לכל רשת, ללא API אחיד
דיוק GPS בתוך מבנים	ניווט פנים-חנות מבוסס אזורים (לא GPS); זיהוי קופות מבוסס Accelerometer	GPS סטנדרטי אינו מדויק בתוך מבנים סגורים
עומס בסיס נתונים	אינדקסים ייעודיים, Materialized View, ושכבת מטמון Redis	מיליוני רשומות מחירים ומבצעים מכל הרשתות
של Rate Limiting Google API	מנגנון fallback לנוסחת Haversine	מכסת בקשות מוגבלת ל-Distance Matrix API
תלות בדיווחי משתמשים	ערכי ברירת מחדל כאשר אין מספיק נתונים; Bayesian Prior לשיפור דיוק	חיזוי תורים דורש מסה קריטית של דיווחים
חיישן תאוצה בסוגי מכשירים	ספי זיהוי נפרדים למצב יד (hand) ומצב עגלה (cart)	רגישות ה-Accelerometer משתנה בין מכשירים

4.3.2 אילוצים עסקיים ורגולטוריים

1. עמידה בחוק הגנת הפרטיות — המערכת אוספת נתוני מיקום ודיווחי תורים. נדרשת הסכמה מפורשת של המשתמש ואנונימיזציה של הדיווחים.

2. תלות בחוק שקיפות המחירים — שינוי או ביטול החוק ישפיע ישירות על זמינות הנתונים.

3. דרישת RTL — כל הממשקים חייבים לתמוך בכיוון עברית (ימין לשמאל) באופן מלא.

4. תמיכה חוצת פלטפורמות — האפליקציה חייבת לרוץ על iOS ו-Android, מה שמכתיב שימוש ב-React Native.

5. ביצועים — זמן תגובה מקסימלי של 3 שניות לתוצאות השוואה, כדי לא לאבד את תשומת הלב של המשתמש.

4.4 שחקנים במערכת

המערכת מזהה ארבעה סוגי שחקנים (גורמים חיצוניים המתקשרים עם המערכת):

שחקן	תיאור	אינטראקציה עיקרית
צרכן (משתמש	משתמש שנרשם למערכת ומנהל	בניית רשימה, קבלת המלצה, דיווח

תורים, ניווט בחנות	רשימות קניות	רשום)
חיפוש מוצרים, צפייה במחירים (ללא רשימות או המלצות)	גולש שלא נרשם — יכול לצפות במחירים בלבד	צרכן אנונימי (אורח)
ייבוא מחירים, מבצעים ונתוני חנויות מכל הרשתות	תהליך אוטומטי מתוזמן (GitHub Actions / Cron)	ETL Scheduler
ניהול נתוני חנויות, מעקב לוגים, הפעלת ETL ידני	מפתח או מנהל טכני עם גישת API ישירה	מנהל מערכת

4.5 תהליכי מחשב

סעיף זה מתאר את תהליכי המחשב המרכזיים במערכת. לכל תהליך מוצגים: תיאור כללי, קלט, פלט, השלבים העיקריים ותת-התהליכים המעורבים.

4.5.1 תהליך השוואת מחירים וקבלת המלצה

זהו התהליך המרכזי במערכת *(RecommendationService.PredictAsync)*. המשתמש בוחר רשימת קניות, והמערכת מחזירה רשימת חנויות ממוינת לפי העלות הנמוכה ביותר.

קלט: מזהה רשימת קניות, תרחיש זמן (עכשיו / +60 דק' / +120 דק'), מצב מיון (עלות נטו / מחיר בלבד / זמן בלבד), עיר (אופציונלי).

פלט: רשימת כרטיסי חנויות, כל אחד כולל: עלות סל (לפני ואחרי מבצעים), עלות דלק, עלות זמן, עלות נטו, זמן נסיעה, זמן בחנות, זמן תור (כולל פירוט לכל סוג קופה), ופירוט מבצעים.

שלבי התהליך:

1. אימות משתמש וזיהוי רשימת הקניות מתוך בסיס הנתונים.
2. שליפת כל החנויות הפעילות בעיר הנבחרת (ברירת מחדל: עיר מועדפת של המשתמש).
3. שליפת מחירים עדכניים לכל מוצרי הרשימה בכל החנויות — בשאילתה אחת מאוחדת (ביטול N+1).
4. שליפת כל המבצעים הפעילים הרלוונטיים למוצרי הרשימה בכל החנויות.
5. חישוב עלות סל לכל חנות: הפעלת מנוע המבצעים *(PromoEngine.ApplyToBasket)* — שלב EXCLUSIVE (בחירת המבצע הבלעדי הטוב ביותר) ושלב ADDITIVE (הפעלת מבצעים נוספים על גבי מחיר הביניים).
6. חיזוי זמן המתנה בתורים: קריאה ל-*QueueService* שמחזיר EWMA כללי, ולאחר מכן ל-*CheckoutLaneService* שמחשב חיזוי M/M/c לכל סוג קופה.
7. חישוב עלות נסיעה: קריאה ל-*TrafficService* עם סוג הרכב של המשתמש — מחזיר מרחק, זמן נסיעה ועלות דלק/חשמל.
8. חישוב זמן שהייה בחנות: הערכה על בסיס מספר פריטים ברשימה וזמן ליקוט ממוצע למוצר.

9. חישוב עלות נטו: **NetCost = BasketCost + FuelCost + TimeCost** כאשר TimeCost = (זמן נסיעה הלוך-חזור + זמן בחנות + זמן תור) × ערך דקה (0.5₪).

10. מיון התוצאות לפי מצב המיון שנבחר וסימון: IsFastest, IsCheapest, IsBestNet.

4.5.2 תהליך חישוב מבצעים (Promo Engine)

מנוע המבצעים *(PromoEngine)* אחראי על חישוב המחיר הסופי של כל מוצר תוך התחשבות בכל המבצעים הפעילים, כולל תמיכה בכפל מבצעים.

קלט: מחיר מדף, כמות, רשימת מבצעים פעילים למוצר, האם המשתמש חבר מועדון.

פלט: עלות סופית, חיסכון, פירוט המבצע שהופעל.

שלבי האלגוריתם לכל מוצר:

1. סינון מבצעים זכאים: מבצעים שתוקפם פג או מיועדים לחברי מועדון בלבד (כשהמשתמש אינו חבר) — מסוננים.

2. שלב **EXCLUSIVE**: בדיקת כל המבצעים הבלעדיים — בחירת המבצע עם החיסכון הגדול ביותר. חישוב מחיר ביניים.

3. שלב **ADDITIVE**: הפעלת כל המבצעים הנוספים (אחוז הנחה או הורדת מחיר) על גבי מחיר הביניים, לפי סדר יורד של הנחה.

4. חישוב סופי: עלות סופית = מחיר ליחידה × כמות. חיסכון = (מחיר מדף × כמות) – עלות סופית.

סוגי המבצעים הנתמכים:

סוג מבצע	תיאור	דוגמה
PRICE_CUT	מחיר מוזל קבוע	חלב תנובה 5.90₪ במקום 7.50₪
PCT_OFF	אחוז הנחה	20% הנחה על כל מוצרי תנובה
MULTI_BUY	רב-קנה — מחיר מיוחד לכמות	3 ב-10₪ (במקום 4.50₪ ליחידה)
BUY_GET	קנה X קבל Y חינם	קנה 2 קבל 1 חינם
OTHER	מבצע שאינו מובנה	מבצע טקסטואלי — לא מחושב אוטומטית

4.5.3 תהליך חיזוי זמני המתנה בתורים

חיזוי התורים מתבצע בשתי שכבות:

שכבה א' — EWMA קהילתי *(QueueService)*

1. קבלת דיווח תור מהמשתמש (ידני או אוטומטי מ-Geofence/StandingDetector).

2. סינון חריגים באמצעות Weighted Double MAD — דיווחים שחורגים מגבולות הסטייה מסוננים.

3. עדכון ממוצע EWMA: נותן משקל גבוה יותר לדיווחים אחרונים (alpha קצר-טווח לזמן אמת, alpha ארוך-טווח להיסטוריה).

4. חישוב תחזית: Mean, חציון (P50), אחוזון 80 (P80), ורמת ביטחון (high/medium/low) על בסיס מספר הדיווחים.

שכבה ב' — M/M/c לכל סוג קופה *(CheckoutLaneService)*

1. זיהוי קופות פעילות לסניף מתוך טבלת checkout_lanes (תוצר ה-Clustering).

2. חישוב λ (קצב הגעת לקוחות): ספירת checkout_events בשעה הנוכחית + יום בשבוע.

3. חישוב μ (קצב שירות): הופכי של ממוצע זמן ההמתנה למרכיב הקופה.

4. חישוב c (מספר שרתים): מספר הקופות הפעילות מאותו סוג.

5. חישוב תוחלת המתנה לפי נוסחת M/M/c: $Wq = P_0 \times (\lambda/\mu)^c / (c! \times (1-\rho)^2) \times 1/(c \times \mu)$.

6. החזרת תחזית לכל סוג קופה (regular, express, self) עם מקור החיזוי (mmC/ewma/default) ורמת ביטחון.

4.5.4 תהליך זיהוי קופות אוטומטי

תהליך זה מורכב משלושה תתי-תהליכים הפועלים בשרשרת:

א. זיהוי עמידה בקופה *(StandingDetector — Mobile)*

1. דגימת חיישן תאוצה כל 500 מילישניות.

2. חישוב magnitude: $\sqrt{(x^2 + y^2 + z^2)}$. אם $|magnitude - 9.81| <$ סף — המכשיר סטטי.

3. אם סטטי ברציפות מעל סף זמן (ברירת מחדל: 90 שניות) — הצעת "הגעת לקופה?".

4. שיקול Bayesian Prior: P(קופה | שעה, יום) — מעלה/מוריד את הסף בהתאם לסבירות שהייה ליד קופה.

5. אישור המשתמש → יצירת CheckoutEvent עם מיקום נרמל (normX, normY).

ב. אשכול מיקומי קופות *(GmmClusterer — Server)*

1. צבירה של checkout_events מאושרים עם מיקום נרמל.

2. הפעלת K-Means עבור K=1 עד K=maxK (ברירת מחדל: 12).

3. לכל K — חישוב BIC (Bayesian Information Criterion). בחירת K עם BIC מינימלי.

4. סינון אשכולות עם פחות מ-5 נקודות (minPointsPerCluster).

5. כל אשכול = קופה פוטנציאלית עם centroid ו-std.

6. עדכון טבלת checkout_lanes עם הקופות המזוהות.

ג. שיוך סוג קופה אוטומטי *(LaneClassifier — Server)*

1. לכל checkout_event חדש — חישוב מרחק Mahalanobis פשוט מכל centroid.

2. אם המרחק < 0.5 × σ (חצי סטיית תקן) ולאשכול יש ≥30 דגימות מתויגות — שיוך אוטומטי.

3. אחרת — הנקודה נשארת ללא תיוג (שמרנות — העדפת דיוק על פני כיסוי).

4.5.5 תהליך ניווט פנים-חנות

תהליך הניווט *(NavigationPlanService)* מייצר מסלול יעיל בתוך החנות על בסיס רשימת הקניות ומפת האזורים.

1. שליפת מפת אזורים של החנות (StoreZones) — כולל ENTRANCE ו-CASHIER חובה.

2. מיפוי כל מוצר ברשימה לאזור המתאים: מוצר → קטגוריה → ZoneCatalog.

3. זיהוי האזורים הנדרשים (בהם יש מוצרים מהרשימה).

4. אופטימיזציית מסלול *(RouteOptimizer)*: סידור האזורים בסדר יעיל — מכניסה, דרך כל האזורים הנדרשים, ועד הקופות.

5. חישוב זמן הליכה משוער בין אזורים על בסיס מרחקי DisplayOrder.

6. החזרת רשימת צעדים (NavigationStepDto): לכל צעד — אזור, שם, מוצרים לאסוף, וזמן הליכה.

4.5.6 תהליך עדכון מחירים ומבצעים (ETL)

תהליך ה-ETL רץ באופן אוטומטי מדי לילה (06:00 שעון ישראל) ואחראי על עדכון מאגר המחירים והמבצעים.

1. שלב **Extract**: גילוי אוטומטי של כתובות URL עדכניות מאתרי הרשתות *(fetch_transparency_urls.py)*.

2. שלב **Transform**: פענוח קבצי XML/GZ, נירמול שמות מוצרים בעברית (*hebrew_nlp.py*), טיפול בחריגות פורמט.

3. שלב **Load**: טעינה לבסיס הנתונים עם UPSERT (עדכון אם קיים, הוספה אם חדש). עדכון טבלאות: store_price_latest, promotions.

4. לוג סיכום: כמות פריטים שעודכנו, שגיאות, וזמן ריצה — נשמר ב-MIGRATION_LOG.

5. עדכון שדות ETL ברשת: LastEtlSuccessAt, LastEtlItemCount.

הרשתות הנתמכות (לפי *chains_config.json*): שופרסל, רמי לוי, מגה, יינות ביתן, חצי חינם, ויקטורי, אושר עד, טיב טעם, שוק העיר, מחסני השוק ועוד.

4.5.7 תהליך הרשמה והזדהות

מערכת האימות *(JwtTokenService + AuthController)* מבוססת על תקן JWT עם מנגנון Refresh Token מאובטח.

1. הרשמה: המשתמש מזין אימייל, סיסמה ושם תצוגה. הסיסמה נשמרת כ-hash (ASP.NET Identity). נוצר Access Token (תוקף: 30 דקות) ו-Refresh Token (תוקף: 7 ימים).

2. התחברות: אימות אימייל וסיסמה. הנפקת Access Token חדש ו-Refresh Token חדש. ה-Refresh Token נשלח אך ורק כ-HttpOnly Cookie (לא נגיש ל-JavaScript).

3. חידוש טוקן: כאשר ה-Access Token פג, הלקוח שולח בקשת Refresh. השרת מאמת את ה-Refresh Token מה-Cookie, מבטל את הטוקן הישן (Token Rotation), ומנפיק זוג חדש.

4. הגבלת קצב: כל נקודות הקצה של אימות מוגבלות ל-5 בקשות לדקה לכל IP (מניעת brute force).

5. הרשאות: כל נקודות הקצה מחלצות את userId מה-JWT — אין אפשרות לגשת לנתונים של משתמש אחר (מניעת IDOR).

4.5.8 תהליך דיווח תור ומעקב ביקורים

1. כניסה לחנות: זיהוי אוטומטי באמצעות Geofencing — כשהמשתמש נכנס לרדיוס החנות, נפתח רשומת UserStoreVisit עם זמן כניסה וקואורדינטות.

2. שהייה בחנות: האפליקציה עוקבת אחרי תנועת המשתמש באמצעות ה-StandingDetector.

3. דיווח תור: ידני (המשתמש בוחר רמת עומס) או אוטומטי (זיהוי עמידה ממושכת + אישור). הדיווח נשמר ב-QueueReport ומשפיע על חיזוי ה-EWMA.

4. "הולך לקופה": המשתמש לוחץ כפתור — מתועד ב-HeadingToCheckoutAt ב-Visit.

5. אישור checkout: נוצר CheckoutEvent עם מיקום נרמל, סוג קופה, וזמן המתנה בפועל — משמש כ-ground truth למודלים.

6. יציאה מהחנות: Geofencing מזהה יציאה — הביקור נסגר עם זמן יציאה ומשך שהייה כולל.

4.6 תרשימי זרימה

סעיף זה מציג את תרשימי הזרימה המרכזיים של המערכת בפורמט תיאורי. התרשימים ממחישים את הלוגיקה העסקית ואת נקודות ההחלטה בכל תהליך.

4.6.1 תרשים זרימה — השוואת מחירים

1. התחלה: המשתמש בוחר רשימת קניות ולוחץ "השווה".

2. בדיקה: האם הרשימה ריקה? → כן: הצג הודעת שגיאה. לא: המשך.

3. שלוף חנויות פעילות בעיר.

4. בדיקה: האם נמצאו חנויות? → לא: הצג "לא נמצאו חנויות באזורך". כן: המשך.

5. שלוף מחירים ומבצעים לכל מוצרי הרשימה.

6. לכל חנות: חשב עלות סל (PromoEngine), חשב עלות נסיעה, חשב תחזית תור.

7. בדיקה: האם לחנות יש מחירים לרוב המוצרים (<50%)? → לא: סמן כ"חסר כיסוי". כן: חשב NetCost.

8. מיין תוצאות לפי מצב המיון שנבחר.

9. סמן: IsBestNet (עלות נטו מינימלית), IsCheapest (סל זול ביותר), IsFastest (זמן כולל מינימלי).

10. הצג תוצאות למשתמש.

11. סיום.

4.6.2 תרשים זרימה — זיהוי קופה אוטומטי

1. התחלה: המשתמש נמצא בחנות (ביקור פעיל).

2. StandingDetector דוגם Accelerometer כל 500ms.

3. בדיקה: האם המכשיר סטטי? → לא: אפס מונה עמידה, חזור לשלב 2. כן: המשך.

4. הגדל מונה עמידה.

5. חשב P(קופה | שעה, יום) מ-Bayesian Prior.

6. בדיקה: האם זמן עמידה > סף × (Prior – 1)? → לא: חזור לשלב 2. כן: המשך.

7. הצג למשתמש: "נראה שהגעת לקופה — לדווח?".

8. בדיקה: האם המשתמש אישר? → לא: סיום. כן: המשך.

9. צור CheckoutEvent עם normX, normY, סוג קופה (אם ידוע), זמן המתנה.

10. בדיקה: האם הצטברו ≥50 אירועים חדשים לסניף? → כן: הפעל GmmClusterer לרענון checkout_lanes.

11. סיום.

4.6.3 תרשים זרימה — עדכון מחירים (ETL)

1. התחלה: GitHub Actions מפעיל ב-06:00 שעון ישראל.

2. לכל רשת ב-chains_config.json:

3. גלה כתובות URL עדכניות (fetch_transparency_urls).

4. בדיקה: האם נמצאו קבצים? → לא: רשום שגיאה, עבור לרשת הבאה. כן: המשך.

5. הורד קבצי XML/GZ.

6. פענח XML, נרמל שמות מוצרים (hebrew_nlp).

7. בדיקה: האם פענוח הצליח? → לא: רשום שגיאה. כן: המשך.
8. UPSERT מחירים ומבצעים לבסיס הנתונים.
9. עדכן LastEtlSuccessAt ו-LastEtlItemCount ברשת.
10. רשום סיכום: כמויות, שגיאות, זמן ריצה.
11. סיום.

פרק 5: ניתוח המערכת

פרק זה מציג את ניתוח המערכת — הגדרת דרישות פונקציונליות ולא-פונקציונליות, מקרי שימוש מפורטים ותרשים מחלקות לניתוח. הניתוח מבוסס על האפיון (פרק 4), סקר הצרכנים (פרק 3) וסקירת הספרות (פרק 2).

5.1 דרישות פונקציונליות

דרישות פונקציונליות מגדירות מה המערכת צריכה לעשות — ההתנהגויות הנדרשות ממנה. הדרישות מחולקות לשמונה תחומים לפי מודולי המערכת.

5.1.1 אימות והרשאות

מזהה	דרישה	עדיפות
FR-1.1	המערכת תאפשר הרשמה עם אימייל, סיסמה ושם תצוגה	גבוהה
FR-1.2	המערכת תנפיק Access Token (תוקף: 30 דקות) ו-Refresh Token (תוקף: 7 ימים)	גבוהה
FR-1.3	ה-Refresh Token יישלח אך ורק כ-HttpOnly Cookie	גבוהה
FR-1.4	חידוש טוקן אוטומטי עם ביטול הטוקן הישן (Token Rotation)	גבוהה
FR-1.5	הגבלת קצב בקשות: 5 בקשות לדקה לכל IP בנקודות קצה של אימות	גבוהה

5.1.2 ניהול רשימות קניות

מזהה	דרישה	עדיפות
FR-2.1	המשתמש יוכל ליצור מספר רשימות קניות	גבוהה
FR-2.2	הוספת מוצרים לרשימה מתוך קטלוג מוצרים עם חיפוש	גבוהה
FR-2.3	עדכון כמות לכל פריט	גבוהה
FR-2.4	מחיקת פריטים מרשימה	גבוהה
FR-2.5	מעקב סטטוס פריט: pending / checked / not_found	גבוהה

5.1.3 השוואת מחירים והמלצה

מזהה	דרישה	עדיפות
FR-3.1	המערכת תשווה מחירי סל קניות בין כל החנויות הפעילות בעיר הנבחרת	גבוהה

FR-3.2	חישוב עלות סל לאחר מבצעים באמצעות PromoEngine (5 סוגים + Stacking)	גבוהה
FR-3.3	חישוב עלות נסיעה לכל חנות על בסיס סוג רכב המשתמש (בנזין/חשמלי/היברידי/הולך רגל)	גבוהה
FR-3.4	חישוב זמן שהייה בחנות על בסיס מספר פריטים וזמן ליקוט ממוצע	בינונית
FR-3.5	חישוב עלות נטו: NetCost = BasketCost + FuelCost + TimeCost	גבוהה
FR-3.6	מיון לפי 3 מצבים: עלות נטו, מחיר בלבד, זמן בלבד (SortMode)	גבוהה
FR-3.7	תמיכה בתרחישי זמן: עכשיו, +60 דק', +120 דק' (ScenarioMinutes)	בינונית
FR-3.8	סימון: IsBestNet, IsCheapest, IsFastest	גבוהה

5.1.4 חיזוי תורים

מזהה	דרישה	עדיפות
FR-4.1	קבלת דיווח תור מהמשתמש (ידני או אוטומטי)	גבוהה
FR-4.2	סינון חריגים באמצעות Weighted Double MAD	גבוהה
FR-4.3	חיזוי כללי באמצעות EWMA (שני ערכי α)	גבוהה
FR-4.4	חיזוי מפורט לכל סוג קופה באמצעות M/M/c	גבוהה
FR-4.5	החזרת תחזית: Mean, P50, P80, רמת ביטחון, מקור	גבוהה
FR-4.6	Fallback היררכי: M/M/c → EWMA → default	גבוהה

5.1.5 זיהוי קופות ומעקב ביקורים

מזהה	דרישה	עדיפות
FR-5.1	זיהוי עמידה ממושכת באמצעות Accelerometer (StandingDetector)	גבוהה
FR-5.2	תמיכה במצב יד ומצב עגלה (ספים שונים: hand=0.8, cart=1.5)	גבוהה
FR-5.3	שיקול Bayesian Prior לשיפור דיוק הזיהוי	בינונית
FR-5.4	יצירת CheckoutEvent עם מיקום נרמל (normX, normY)	גבוהה
FR-	אשכול קופות באמצעות K-Means + BIC (כל 50 אירועים)	גבוהה

5.5		
FR-5.6	שיוך סוג קופה אוטומטי (LaneClassifier) — שמרני (0.5σ)	בינונית
FR-5.7	מעקב ביקורים (UserStoreVisit) עם Geofencing	גבוהה

5.1.6 ניווט פנים-חנות

מזהה	דרישה	עדיפות
FR-6.1	מיפוי מוצרים לאזורים בחנות על בסיס קטגוריה → ZoneCatalog	גבוהה
FR-6.2	חישוב מסלול אופטימלי: כניסה → אזורים נדרשים → קופות	גבוהה
FR-6.3	החזרת רשימת צעדים עם: אזור, שם, מוצרים לאסוף, זמן הליכה	גבוהה

5.1.7 עדכון נתונים (ETL)

מזהה	דרישה	עדיפות
FR-7.1	ייבוא יומי אוטומטי של מחירים מכל הרשתות	גבוהה
FR-7.2	ייבוא יומי אוטומטי של מבצעים מכל הרשתות	גבוהה
FR-7.3	גילוי אוטומטי של כתובות URL עדכניות	גבוהה
FR-7.4	נירמול שמות מוצרים בעברית (hebrew_nlp)	גבוהה
FR-7.5	טעינה בשיטת UPSERT (אידמפוטנטיות)	גבוהה

5.1.8 תקשורת בזמן אמת

מזהה	דרישה	עדיפות
FR-8.1	עדכונים חיים על שינויים בתורים באמצעות SignalR	בינונית
FR-8.2	עדכון מיידי של המלצות בעקבות דיווח חדש	בינונית

5.2 דרישות לא-פונקציונליות

דרישות לא-פונקציונליות מגדירות איך המערכת צריכה לפעול — תכונות איכות החוצות לגבולות הפונקציונליות.

מזהה	קטגוריה	דרישה	מדד
NFR-1	ביצועים	זמן תגובה להשוואת מחירים	< 3 שניות
NFR-2	ביצועים	זמן תגובה לחיזוי תור	< 1 שניה
NFR-3	ביצועים	תמיכה במטמון (Redis) עם fallback	TTL: 15 דקות
NFR-4	אבטחה	אימות JWT עם HttpOnly Refresh	תקן תעשייתי
NFR-5	אבטחה	הגבלת קצב בקשות בנקודות אימות	5/דקה/IP
NFR-6	אבטחה	מניעת IDOR — חילוץ userId מ-JWT	בכל נקודת קצה
NFR-7	אבטחה	CORS מוגבל ל-origins מוגדרים בלבד	GET + POST בלבד
NFR-8	זמינות	המערכת זמינה 24/7	Docker + nginx
NFR-9	סקלביליות	פריסה בקונטיינרים עם Docker Compose	פקודה אחת
NFR-10	תחזוקתיות	קוד מודולרי עם הפרדת אחריות ברורה	ארכיטקטורת שכבות + DI
NFR-11	תחזוקתיות	בדיקות אוטומטיות ב-CI לכל push	GitHub Actions
NFR-12	ניטור	Structured Logging עם Serilog	לוגים מובנים
NFR-13	נגישות	תמיכת RTL מלאה בכל הממשקים	עברית בלבד
NFR-14	נגישות	אפליקציה חוצת פלטפורמות (iOS + Android)	React Native / Expo
NFR-15	פרטיות	דיווחי תורים אנונימיים; מיקום נרמל (לא מוחלט)	הסכמת משתמש

5.3 סיכום דרישות

סה"כ הוגדרו **37 דרישות פונקציונליות** ב-8 תחומים ו-**15 דרישות לא-פונקציונליות** ב-6 קטגוריות (ביצועים, אבטחה, זמינות, סקלביליות, תחזוקתיות, נגישות ופרטיות). כל דרישה כוללת מזהה ייחודי, תיאור, רמת עדיפות ומדד כמותי (כשרלוונטי). בסעיף הבא יוצגו מקרי השימוש המרכזיים הנגזרים מדרישות אלה.

5.4 מקרי שימוש (Use Cases)

מקרי השימוש מתארים את האינטראקציות העיקריות של השחקנים עם המערכת. לכל מקרה מוצגים: מזהה, שחקן, תנאים, זרימה ראשית, זרימה חלופית ודרישות קשורות.

UC-1: הרשמה למערכת

מזהה	UC-1
שם	הרשמה למערכת
שחקן ראשי	צרכן (אורח)
תנאי קדם	המשתמש אינו רשום במערכת
טריגר	המשתמש לוחץ על "הרשמה"
זרימה ראשית	1. המערכת מציגה טופס הרשמה (email, סיסמה, שם) 2. המשתמש ממלא את השדות 3. המערכת מאמתת תקינות קלט 4. נוצר חשבון + מונפקים טוקנים 5. המשתמש מועבר למסך הראשי
זרימה חלופית	3a. email כבר קיים → הודעת שגיאה 3b. סיסמה חלשה מדי → הודעה על דרישות
תנאי סיום	חשבון נוצר, המשתמש מחובר
דרישות קשורות	FR-1.1, FR-1.2, FR-1.3

UC-2: התחברות למערכת

מזהה	UC-2
שם	התחברות למערכת
שחקן ראשי	צרכן רשום
תנאי קדם	למשתמש יש חשבון קיים
טריגר	המשתמש לוחץ "התחבר"
זרימה ראשית	1. המערכת מציגה טופס התחברות 2. המשתמש מזין email וסיסמה 3. המערכת מאמתת ומנפיקה 4 Access Token + Refresh Token (HttpOnly Cookie). המשתמש מועבר למסך הראשי
זרימה חלופית	3a. email/סיסמה שגויים → הודעת שגיאה 3b. חריגה מ-Rate Limit → "נסה שוב בעוד דקה"
תנאי סיום	המשתמש מחובר עם טוקנים תקפים
דרישות קשורות	FR-1.2, FR-1.3, FR-1.5

UC-3: בניית רשימת קניות

מזהה	UC-3
שם	בניית רשימת קניות
שחקן ראשי	צרכן רשום
תנאי קדם	המשתמש מחובר
טריגר	המשתמש לוחץ "רשימה חדשה" או פותח רשימה קיימת
זרימה ראשית	1. המשתמש מחפש מוצר בקטלוג 2. המערכת מציגה תוצאות מתאימות 3. המשתמש בוחר מוצר וקובע כמות 4. הפריט נוסף לרשימה בסטטוס pending 5. חזרה על שלבים 1-4 להוספת פריטים נוספים

זרימה חלופית	2a. אין תוצאות → הצעה לשנות מילות חיפוש 5a. המשתמש מעדכן כמות או מוחק פריט
תנאי סיום	רשימת קניות מוכנה עם פריטים
דרישות קשורות	FR-2.1, FR-2.2, FR-2.3, FR-2.4, FR-2.5

UC-4: השוואת מחירים וקבלת המלצה

מזהה	UC-4
שם	השוואת מחירים וקבלת המלצה
שחקן ראשי	צרכן רשום
תנאי קדם	למשתמש יש רשימת קניות עם פריטים
טריגר	המשתמש לוחץ "השווה"
זרימה ראשית	1. המשתמש בוחר רשימה, תרחיש זמן ומצב מיון 2. המערכת שולפת מחירים ומבצעים לכל חנות 3. לכל חנות: מפעילה PromoEngine (שלב EXCLUSIVE + שלב ADDITIVE) 4. מחשבת עלות נסיעה (סוג רכב, מרחק, תנועה) 5. מחשבת חיזוי תור: EWMA כללי + M/M/c לכל סוג קופה 6. מחשבת NetCost וממיינת תוצאות 7. מציגה רשימת כרטיסי חנויות עם סימונים
זרימה חלופית	2a. אין חנויות בעיר → "לא נמצאו חנויות" 2b. רשימה ריקה → הודעה 4a. Google API לא זמין → fallback ל-Haversine
תנאי סיום	המשתמש רואה רשימת חנויות ממוינת
דרישות קשורות	FR-3.1 עד FR-3.8, FR-4.3, FR-4.4

UC-5: דיווח תור ידני

מזהה	UC-5
שם	דיווח תור ידני
שחקן ראשי	צרכן רשום
תנאי קדם	המשתמש נמצא בחנות או לידה
טריגר	המשתמש לוחץ "דווח תור"
זרימה ראשית	1. המשתמש בוחר חנות ורמת עומס (None/Short/Medium/Long) 2. אופציונלי: מזין זמן המתנה בפועל (דקות) 3. המערכת מאמתת את הדיווח 4. סינון חריגים באמצעות MAD 5. עדכון EWMA לשעה+יום הנוכחיים 6. שליחת עדכון בזמן אמת למשתמשים מחוברים (SignalR)
זרימה חלופית	4a. דיווח חריג (>גבול MAD) → נפסל, לא משפיע על EWMA
תנאי סיום	דיווח נשמר, חיזוי מתעדכן
דרישות קשורות	FR-4.1, FR-4.2, FR-4.3, FR-8.1

UC-6: זיהוי קופה אוטומטי

מזהה	UC-6
שם	זיהוי קופה אוטומטי
שחקן ראשי	צרכן רשום (בחנות)
תנאי קדם	המשתמש בביקור פעיל בחנות (UserStoreVisit פתוח)
טריגר	המשתמש עומד במקום אחד למעלה מ-90 שניות

זרימה ראשית	1. StandingDetector דוגם Accelerometer כל 500ms. מחשב magnitude = √(x²+y²+z²) 3. אם |magnitude - 9.81| < סף → מגדיל מונה עמידה 4. מחשב P(קופה|שעה,יום) מ-Bayesian Prior 5. אם זמן עמידה > סף×(1-Prior) → מציג "הגעת לקופה?" 6. המשתמש מאשר ומזין סוג קופה + זמן המתנה 7. נוצר CheckoutEvent עם normX, normY 8. אם הצטברו 50+ אירועים → מפעיל GmmClusterer לרענון קופות
זרימה חלופית	5a. המשתמש דוחה → האירוע נדחה 3a. המשתמש חוזר לתנועה → איפוס מונה
תנאי סיום	אירוע קופה נשמר; קופות מתעדכנות אם נדרש
דרישות קשורות	FR-5.1 עד FR-5.6

UC-7: ניווט פנים-חנות

מזהה	UC-7
שם	ניווט פנים-חנות
שחקן ראשי	צרכן רשום
תנאי קדם	המשתמש בחר חנות ויש לו רשימה עם פריטים
טריגר	המשתמש לוחץ "נווט אותי בחנות"
זרימה ראשית	1. המערכת שולפת מפת אזורים של החנות 2. ממפה כל מוצר ברשימה לאזור מתאים 3. RouteOptimizer מסדר אזורים: כניסה → אזורים → קופות 4. מחשב זמן הליכה בין אזורים 5. מציג רשימת צעדים: אזור, שם, מוצרים לאסוף, זמן
זרימה חלופית	1a. חנות ללא מפת אזורים → שגיאה 2a. מוצר ללא אזור → משויך ל"מוצרים נוספים"
תנאי סיום	מסלול ניווט מוצג למשתמש
דרישות קשורות	FR-6.1, FR-6.2, FR-6.3

UC-8: עדכון מחירים ומבצעים (ETL)

מזהה	UC-8
שם	עדכון מחירים ומבצעים
שחקן ראשי	ETL Scheduler
תנאי קדם	שעון 06:00 שעון ישראל (או הפעלה ידנית)
טריגר	GitHub Actions מפעיל את nightly-etl.yml
זרימה ראשית	1. לכל רשת ב-chains_config.json: 2. גילוי URL עדכניות (fetch_transparency_urls) 3. הורדת קבצי XML/GZ 4. פענוח + נירמול עברית (hebrew_nlp) 5. UPSERT ל-store_price_latest ו-promotions 6. עדכון LastEtlSuccessAt + LastEtlItemCount 7. רישום סיכום ב-MIGRATION_LOG
זרימה חלופית	3a. קובץ פגום → רישום שגיאה, מעבר לרשת הבאה 3b. URL לא נמצא → retry + רישום
תנאי סיום	מחירים ומבצעים מעודכנים לכל הרשתות
דרישות קשורות	FR-7.1 עד FR-7.5

UC-9: כניסה ויציאה מחנות (Geofencing)

מזהה	UC-9
שם	כניסה ויציאה מחנות

שחקן ראשי	צרכן רשום (אוטומטי)
תנאי קדם	האפליקציה פועלת עם הרשאת מיקום
טריגר	המשתמש נכנס לרדיוס חנות (50 מטר)
זרימה ראשית	1. Geofencing מזהה כניסה לאזור חנות 2. נפתחת UserStoreVisit עם זמן כניסה + קואורדינטות 3. מופעל StandingDetector. כשהמשתמש יוצא 4. → הביקור נסגר 5. אם לא דווח תור → מוצעת שאלה
זרימה חלופית	1a. false positive (חניה סמוכה) → debounce + אישור משתמש 4a. יציאה וכניסה מהירה → התעלמות
תנאי סיום	ביקור נפתח/נסגר; נתוני שהייה נשמרים
דרישות קשורות	FR-5.7, FR-4.1

5.5 סיכום מקרי השימוש

מזהה	שם	שחקן	דרישות
UC-1	הרשמה	צרכן (אורח)	FR-1.1 עד FR-1.3
UC-2	התחברות	צרכן רשום	FR-1.2, FR-1.3, FR-1.5
UC-3	בניית רשימת קניות	צרכן רשום	FR-2.1 עד FR-2.5
UC-4	השוואת מחירים והמלצה	צרכן רשום	FR-3.1 עד FR-3.8, FR-4
UC-5	דיווח תור ידני	צרכן רשום	FR-4.1 עד FR-4.3, FR-8.1
UC-6	זיהוי קופה אוטומטי	צרכן (בחנות)	FR-5.1 עד FR-5.6
UC-7	ניווט פנים-חנות	צרכן רשום	FR-6.1 עד FR-6.3
UC-8	מחירים ומבצעים ETL	ETL Scheduler	FR-7.1 עד FR-7.5
UC-9	Geofencing	צרכן (אוטומטי)	FR-5.7, FR-4.1

תשעת מקרי השימוש מכסים את כל 37 הדרישות הפונקציונליות שהוגדרו בסעיף 5.1. כל מקרה כולל זרימות חלופיות המתארות את התנהגות המערכת במצבים לא צפויים. בסעיף הבא יוצג תרשים המחלקות לניתוח.

5.6 תרשים מחלקות לניתוח

תרשים המחלקות לניתוח *(Analysis Class Diagram)* מתאר את הישויות המרכזיות בתחום הבעיה (דומיין), את התכונות שלהן ואת הקשרים ביניהן. להלן תיאור מפורט של כל מחלקה, מחולקות לששה תחומים.

5.6.1 תחום משתמשים ואימות

מחלקה	**AppUser**
תיאור	משתמש רשום במערכת
תכונות	Id: int (PK), Email: string, PasswordHash: string, DisplayName: string, PreferredCity: string?, VehicleType: enum, CreatedAt: DateTimeOffset
קשרים	N → ShoppingList | ← 1:N → RefreshToken | ← 1:N → :1 ← QueueReport | ← 1:N → UserStoreVisit

מחלקה	**RefreshToken**
תיאור	טוקן רענון לחידוש JWT
תכונות	RefreshTokenId: long (PK), UserId: int (FK), TokenHash: string, ExpiresAt, ?CreatedAt, RevokedAt?, ReplacedByTokenHash
קשרים	N:1 → AppUser →

5.6.2 תחום חנויות ומוצרים

מחלקה	**StoreChain**
תיאור	רשת שיווק (שופרסל, רמי לוי...)
תכונות	ChainId: int (PK), ChainName, ChainCode, ChainType, LastEtlSuccessAt?, ?LastEtlItemCount
קשרים	N → Store | ← 1:N → StoreFormat:1 ←

מחלקה	**Store**
תיאור	סניף פיזי של רשת
תכונות	StoreId: int (PK), ChainId: int (FK), FormatId?: int (FK), BranchName, City, Address?, BranchCode?, Latitude?, Longitude?, IsActive
קשרים	N:1 → StoreChain | → N:1 → StoreFormat | ← 1:N → → StorePriceLatest | ← 1:N → Promotion | ← 1:N → QueueReport | ← 1:N → StoreZone | ← 1:N → CheckoutLane

מחלקה	**Product**
תיאור	מוצר בקטלוג
תכונות	ProductId: int (PK), Barcode, ProductName, Manufacturer?, CategoryId?: int (FK), IsWeighted, IsPrivateLabel, Unit, AvgPickSec
קשרים	N:1 → Category | ← 1:N → StorePriceLatest | ← 1:N → → ShoppingListItem | ← 1:N → Promotion

מחלקה	**Category**
תיאור	קטגוריית מוצר (מוצרי חלב, ירקות...)
תכונות	CategoryId: int (PK), CategoryName, ZoneId?: string (FK → ZoneCatalog)
קשרים	N → Product | → N:1 → ZoneCatalog:1 ←

מחלקה	**StorePriceLatest**
תיאור	מחיר עדכני של מוצר בסניף
תכונות	StoreId: int (FK, PK), ProductId: int (FK, PK), Price: decimal, UnitPrice?, PriceUpdateDate?, UpdatedAt
קשרים	N:1 → Store | → N:1 → Product →

מחלקה	**Promotion**
תיאור	מבצע פעיל (5 סוגים + Stacking)
תכונות	PromoId: long (PK), StoreId (FK), Barcode?, ProductId? (FK), PromoType: string, Description?, StartDate?, EndDate?, MinQty, MaxQty?, DiscountPct?, DiscountedPrice?, RewardQty?, IsMemberOnly, IsActive, StackingMode: string, UpdatedAt
קשרים	N:1 → Store | → N:1 → Product →

5.6.3 תחום רשימות קניות

מחלקה	**ShoppingList**
תיאור	רשימת קניות של משתמש
תכונות	ListId: int (PK), UserId: int (FK), ListName: string
קשרים	N:1 → AppUser | ← 1:N → ShoppingListItem →

מחלקה	**ShoppingListItem**
תיאור	פריט ברשימת קניות
תכונות	ItemId: int (PK), ListId: int (FK), ProductId: int (FK), Quantity: double, ItemStatus: string (pending/checked/not_found)
קשרים	N:1 → ShoppingList | → N:1 → Product →

5.6.4 תחום תורים וחיזוי

מחלקה	**QueueReport**
תיאור	דיווח תור ממשתמש
תכונות	ReportId: int (PK), StoreId (FK), UserId? (FK), WaitLevel: int (0-3), ActualWaitMin?: double, ReportedAt, EffectiveAtUtc, Source: int ?(Manual/Geofence/Admin), CheckoutType?, VisitId
קשרים	N:1 → Store | → N:1 → AppUser | → N:1 → UserStoreVisit →

מחלקה	**StoreQueueStat**
תיאור	סטטיסטיקת תורים ממוזגת (EWMA ארוך+קצר)

תכונות	StoreId (PK), DayOfWeek (PK), HourOfDay (PK), AvgWaitMinutes, ReportCount, MeanMinutes, MedianMinutes, MadLower, MadUpper, RecentReportCount, LastUpdatedUtc
קשרים	מפתח מרכב: (StoreId, DayOfWeek, HourOfDay)	N:1 → Store →

5.6.5 תחום ביקורים וזיהוי קופות

מחלקה	**UserStoreVisit**
תיאור	ביקור של משתמש בחנות (Geofencing)
תכונות	VisitId: long (PK), UserId? (FK), StoreId (FK), EnteredAt, ExitedAt?, CartMode: bool, EntryLat?, EntryLon?, ExitLat?, ExitLon?, TotalMinutes?, CalibrationCount, ?HeadingToCheckoutAt
קשרים	N:1 → AppUser	→ N:1 → Store	← 1:N → CheckoutEvent	← 1:N → → QueueReport

מחלקה	**CheckoutEvent**
תיאור	אירוע קופה — ground truth למודלים
תכונות	EventId: long (PK), VisitId (FK), StoreId (FK), Lat?, Lon?, NormX?, NormY?, StandingSeconds, CheckoutType?, ActualWaitMin?, DayOfWeek, HourOfDay, ConfirmedByUser, ReportedAt
קשרים	משמש כקלט ל-GmmClusterer	N:1 → UserStoreVisit	→ N:1 → Store →

מחלקה	**CheckoutLane**
תיאור	קופה מזוהה ע"י K-Means Clustering
תכונות	LaneId: int (PK), StoreId (FK), LaneType? (regular/express/self/unknown), CentroidX, CentroidY, StdX, StdY, AvgWaitMin, SampleCount, IsActive, LastUpdated
קשרים	משמש לחישוב M/M/c ב-CheckoutLaneService	N:1 → Store →

5.6.6 תחום ניווט פנים-חנות

מחלקה	**ZoneCatalog**
תיאור	טבלת התייחסות — מיפוי קטגוריות לאזורים
תכונות	ZoneCode: string (PK), ZoneName, AisleOrder: int
קשרים	N → Category:1 ← (דרך ZoneId)

מחלקה	**StoreZone**
תיאור	אזור פיזי בסניף (מפת החנות)
תכונות	StoreId (FK, PK), ZoneCode (PK), ZoneName, MapX, MapY, DisplayOrder
קשרים	משמש לניווט ב-NavigationPlanService	N:1 → Store →

5.6.7 טבלת קשרים מרכזית

מחלקה א	קשר	מחלקה ב	תיאור

AppUser	N : 1	ShoppingList	משתמש מחזיק מספר רשימות
AppUser	N : 1	RefreshToken	למשתמש מספר טוקנים פעילים
AppUser	N : 1	QueueReport	משתמש מדווח מספר דיווחים
AppUser	N : 1	UserStoreVisit	משתמש מבקר במספר חנויות
StoreChain	N : 1	Store	רשת מכילה מספר סניפים
StoreChain	N : 1	StoreFormat	רשת מגדירה פורמטים (STANDARD, EXPRESS...)
Store	N : 1	StorePriceLatest	סניף מכיל מחירים עדכניים
Store	N : 1	Promotion	סניף מכיל מבצעים פעילים
Store	N : 1	QueueReport	סניף מקבל דיווחי תור
Store	N : 1	CheckoutLane	סניף מכיל קופות מזוהות
Store	N : 1	StoreZone	סניף מכיל אזורים לניווט
Product	N : 1	StorePriceLatest	מוצר נמכר במספר סניפים
Product	N : 1	Category	מוצר שייך לקטגוריה
Category	N : 1	ZoneCatalog	קטגוריה ממופה לאזור בחנות
ShoppingList	N : 1	ShoppingListItem	רשימה מכילה פריטים
UserStoreVisit	N : 1	CheckoutEvent	ביקור מכיל אירועי קופה

5.7 סיכום פרק 5

פרק זה הציג את ניתוח המערכת בשלושה מימדים: **37 דרישות פונקציונליות** ב-8 תחומים, **15 דרישות לא-פונקציונליות** ב-6 קטגוריות, **9 מקרי שימוש** מפורטים, ו**15 מחלקות ניתוח** ב-6 תחומים עם 16 קשרים. הניתוח מהווה את הבסיס לעיצוב המערכת בפרק 6 — תרשים מחלקות לעיצוב, סכמת בסיס נתונים רלציונית ותרשימי רצף.

פרק 6: עיצוב המערכת

פרק זה מציג את עיצוב המערכת — המעבר ממודל הניתוח (פרק 5) למימוש בפועל. הפרק כולל תרשים מחלקות לעיצוב (שכבת השירותים), סכמת בסיס נתונים רלציונית מפורטת, ותרשימי רצף לתהליכים המרכזיים.

6.1 תרשים מחלקות לעיצוב — שכבת השירותים

בעוד שתרשים המחלקות לניתוח (סעיף 5.6) תיאר את הישויות (דומיין), תרשים המחלקות לעיצוב מתמקד בשכבת השירותים — המחלקות שמכילות את הלוגיקה העסקית. כל שירות מוזרק באמצעות Dependency Injection ומקבל את התלויות שלו דרך הבנאי (constructor injection).

RecommendationService 6.1.1

מחלקה	**RecommendationService**
תיאור	השירות המרכזי — מתאם את כל השירותים לחישוב עלות נטו לכל חנות
תלויות (DI)	AppDbContext, IQueueService, TimeService, TrafficService
מתודות ציבוריות	**PredictAsync(userId, PredictRequest) → PredictResponse** שלבים: שליפת חנויות → שליפת מחירים+מבצעים (bulk) → PromoEngine.ApplyToBasket → TrafficService → QueueService → חישוב NetCost → מיון → סימונים
הערות	מבטל בעיית N+1 באמצעות שאילתות מאוחדות (bulk fetch)

PromoEngine 6.1.2

מחלקה	**PromoEngine**
תיאור	מנוע מבצעים — מחלקה static (ללא תלויות, ללא מצב)
תלויות (DI)	אין — מחלקה static ללא תלויות
מתודות ציבוריות	**Apply(regularPrice, qty, promos, isMember) → PromoResult** **ApplyToBasket(items, promosByProduct, isMember) → BasketPromoResult** אלגוריתם: שלב EXCLUSIVE (בחירת המבצע הבלעדי הטוב ביותר) → שלב ADDITIVE (הפעלת מבצעים נוספים על גבי מחיר הביניים)
הערות	5 סוגי מבצע: PRICE_CUT, PCT_OFF, MULTI_BUY, BUY_GET, OTHER. כל סוג ממומש במתודה פרטית: CalcPriceCut, CalcPctOff, CalcMultiBuy, CalcBuyGet

QueueService 6.1.3

מחלקה	**QueueService**
תיאור	שירות חיזוי תורים קהילתי (EWMA)
תלויות (DI)	AppDbContext, IQueuePredictor, IQueueStateUpdater, IQueueOutlierFilter
מתודות ציבוריות	**ReportAsync(userId, QueueReportRequest) → void PredictAsync(storeId) → QueuePrediction GetHistoryAsync(storeId, hours) →** **List<QueueHistoryBucket>** Report: סינון → MAD → עדכון EWMA (שני α) → שליפת StoreQueueStat לשעה+יום → החזרת SignalR broadcast Predict

	Mean/P50/P80/Confidence
הערות	מימוש IQueueService — מאפשר החלפה ב-mock לבדיקות

6.1.4 תת-מחלקות QueueModel

מודול **QueueModel** מכיל ארבעה תת-מחלקות עם הפרדת אחריות באמצעות ממשקים:

1. **QueuePredictor** (*IQueuePredictor*) — מחשב תחזית מ-StoreQueueStat. מחזיר: Mean, Median, P80 ורמת ביטחון.

2. **QueueStateUpdater** (*IQueueStateUpdater*) — מעדכן את ה-EWMA ב-StoreQueueStat אחרי קבלת דיווח חדש.

3. **WeightedDoubleMadOutlierFilter** (*IQueueOutlierFilter*) — מסנן דיווחים חריגים באמצעות Double MAD אסימטרי.

4. **QueueMath** — מחלקת עזר (static) עם פונקציות: EWMA, Median, MAD, P80.

6.1.5 CheckoutLaneService

מחלקה	**CheckoutLaneService**
תיאור	שירות דו-תפקידי: רענון קופות (GMM) + חיזוי M/M/c
תלויות (DI)	AppDbContext
מתודות ציבוריות	**RefreshLanesAsync(storeId) → void EstimateWaitAsync(storeId, dayOfWeek, hour) → List<LaneQueueEstimate>** Refresh: שולף checkout_events → GmmClusterer.DetectLanes עדכון checkout_lanes Estimate: לכל סוג קופה — חישוב λ, μ, c → נוסחת M/M/c → Wq

6.1.6 GmmClusterer

מחלקה	**GmmClusterer**
תיאור	זיהוי קופות מנקודות (static)
תלויות (DI)	static — אין
מתודות ציבוריות	**DetectLanes(points, maxK, minPoints) → List<LaneCluster>** אלגוריתם: K-Means לכל K=1..maxK → חישוב BIC לכל K → בחירת K אופטימלי → סינון אשכולות קטנים
הערות	מתודות פנימיות: RunKMeans, ComputeBic, AssignToClusters

6.1.7 LaneClassifier

מחלקה	**LaneClassifier**
תיאור	שיוך סוג קופה אוטומטי לנקודה חדשה
תלויות (DI)	AppDbContext
מתודות ציבוריות	**TryClassifyAsync(storeId, normX, normY) → ClassificationResult?** חישוב Mahalanobis פשוט מכל centroid → אם מרחק < 0.5σ ויש ≥30 דגימות → שיוך. אחרת → null (שמרנות)

BasketService 6.1.8

מחלקה	**BasketService**
תיאור	חישוב עלות סל עם מבצעים + מטמון
תלויות (DI)	AppDbContext, IDistributedCache
מתודות ציבוריות	**CalcBasketAsync(storeId, listId, isMember) → BasketResult** שליפת פריטים + מחירים + מבצעים → PromoEngine.ApplyToBasket → שמירה במטמון (TTL: 15 דקות)
הערות	מפתח מרכב: basket_v2:{storeId}:{listId}:{isMember}

TrafficService 6.1.9

מחלקה	**TrafficService**
תיאור	חישוב מרחק, זמן נסיעה ועלות דלק
תלויות (DI)	אין (משתמש ב-HttpClient)
מתודות ציבוריות	**GetDistanceAsync(fromLat, fromLon, toLat, toLon, vehicleType) → TravelInfo** נסיון ראשון: Google Distance Matrix API → אם נכשל: fallback Haversine (MathHelpers)-ל חישוב עלות דלק: לפי VehicleTypeInfo (kWh/km או L/100km)

NavigationPlanService 6.1.10

מחלקה	**NavigationPlanService**
תיאור	בניית מסלול ניווט פנים-חנות
תלויות (DI)	AppDbContext, RouteOptimizer
מתודות ציבוריות	שליפת **BuildPlanAsync(storeId, userId, listId) → NavigationResponse** StoreZones → מיפוי מוצרים לאזורים → RouteOptimizer.Solve → בניית צעדים

JwtTokenService 6.1.11

מחלקה	**JwtTokenService**
תיאור	הנפקה ואימות טוקנים
תלויות (DI)	AppDbContext, IConfiguration
מתודות ציבוריות	**GenerateAccessToken(user) → string GenerateRefreshToken(userId) → string ValidateRefreshToken(tokenHash) → RefreshToken? RotateRefreshToken(old, userId) → string**
הערות	Access Token: 30 דקות, Refresh: 7 ימים, נשמר כ-hash ב-DB. HS256. Token Rotation מבטל את הישן.

תלויות: אין. משמש כתלות של RecommendationService.

4. AdjustQueueByScenario — התאמת תורים: +60 → ×1.15 (עומס צפוי), +120 → × 0.85 (שפל).

3. EstimateTravelMinutesOneWay — זמן נסיעה עם TrafficFactor לפי יום/שעה + מקדם תרחיש (עכשיו=1.0, +60=1.08, +120=0.95).

2. EstimateCheckoutServiceMinutes — זמן שירות בקופה: itemCount × 4 שניות.

1. EstimateInStoreMinutes — זמן שהייה: BaseSeconds=120 + regular×28 + weighted×55, מוכפל במקדם מהירות (0.4–2.5).

TimeService מחשב זמני שהייה צפויים בחנות ובנסיעה. כולל ארבע מתודות:

TimeService 6.1.13

המחלקה אינה מתחשבת בגודל הסל (לא אמין) או בחיזוי העדפת Self-Checkout (אין מספיק נתונים בפיילוט). תלויות: אין (static). מתודה: Select(estimates, userPreferredLane?) → LaneQueueEstimate.

4. אם אין lanes כלל — fallback ל-EWMA הגלובלי.

3. אחרת — בחר כל סוג קופה זמין (הראשון ברשימה).

2. אחרת — בחר קופה רגילה (regular) אם קיימת.

1. אם למשתמש יש סוג קופה מועדף מהיסטוריית קניות — השתמש בו אם קיים.

LaneSelector הוא מחלקה סטטית שבוחרת קופה יחידה מתוך רשימת ה-LaneQueueEstimate לשימוש בחישוב NetCost. אלגוריתם הבחירה פועל לפי 4 כללי עדיפות:

LaneSelector 6.1.12

שכבת הבקרים (Controllers) 6.1.14

הבקרים מהווים שכבת דקה — ניתוב, אימות קלט והעברה לשירותים. להלן המיפוי:

Controller	Endpoint	שירותים שמוזרקים
AuthController	*/api/auth/	JwtTokenService, UserManager
PredictController	api/predict/	RecommendationService
QueueController	*/api/queue/	QueueService, CheckoutLaneService
NavigationController	*/api/navigation/	NavigationPlanService
StoresController	*/api/stores/	AppDbContext
BasketController	*/api/basket/	BasketService
CheckoutController	*/api/checkout/	CheckoutLaneService, LaneClassifier
ProductsController	*/api/products/	AppDbContext
ProfileController	*/api/profile/	UserManager, AppDbContext
ShoppingListController	*/api/lists/	AppDbContext

6.1.15 גרף תלויות — סיכום

להלן גרף התלויות המרכזי של המערכת:

1. PredictController → **RecommendationService** → { PromoEngine, QueueService, TrafficService, TimeService }
2. QueueController → **QueueService** → { QueuePredictor, QueueStateUpdater, OutlierFilter }
3. QueueController → **CheckoutLaneService** → GmmClusterer (static)
4. CheckoutController → **LaneClassifier**
5. NavigationController → **NavigationPlanService** → RouteOptimizer
6. AuthController → **JwtTokenService**

כל השירותים נרשמים ב-*Program.cs* באמצעות builder.Services.AddScoped/AddSingleton. ההפרדה בין ממשקים למימושים מאפשרת החלפת מימושים (Redis → InMemoryCache) ובדיקות באמצעות mock. בסעיף הבא תוצג סכמת בסיס הנתונים הרלציונית.

6.2 סכמת בסיס נתונים רלציונית

בסיס הנתונים של **SuperWaze** הוא **PostgreSQL**. הסכמה מנורמלת ל-**3NF** ופותחה באמצעות 10 מיגרציות מתועדות (V001–V010). להלן תיאור כל הטבלאות, עמודותיהן, אינדקסים ומפתחות זרים.

6.2.1 טבלאות חנויות ורשתות

store_chains

רשתות שיווק (שופרסל, רמי לוי, מגה...)

עמודה	סוג	הערות
chain_id	SERIAL PK	מזהה רשת
chain_name	VARCHAR(100)	שם הרשת
chain_code	VARCHAR(20) UNIQUE	קוד רשת לפי חוק שקיפות
chain_type	VARCHAR(20)	supermarket / pharmacy / online
last_etl_success_at	TIMESTAMPTZ	זמן ETL אחרון מוצלח (V010)
last_etl_item_count	INT	כמות פריטים שעודכנו (V010)

store_formats

פורמט חנות (STANDARD, EXPRESS, ONLINE)

עמודה	סוג	הערות
format_id	SERIAL PK	מזהה
chain_id	INT FK	store_chains →
format_name	VARCHAR(50)	STANDARD / EXPRESS / ONLINE
avg_pick_sec_per_item	DOUBLE	זמן ליקוט ממוצע למוצר

מפתחות זרים: chain_id → store_chains(chain_id)

stores

סניפי הרשתות — הטבלה המרכזית במערכת

עמודה	סוג	הערות
store_id	SERIAL PK	מזהה סניף
chain_id	INT FK NOT NULL	store_chains →
format_id	INT FK	store_formats →
branch_name	VARCHAR(200)	שם הסניף
city	VARCHAR(100)	עיר (אינדקס על עיר)
address	VARCHAR(300)	כתובת מלאה
branch_code	VARCHAR(20)	קוד סניף לפי חוק שקיפות
latitude	DOUBLE	קו רוחב (Geofencing + מרחק)
longitude	DOUBLE	קו אורך
is_active	BOOLEAN DEFAULT TRUE	חנות לא פעילה מסוננת

אינדקסים: *idx_stores_city(city)*, *idx_stores_chain(chain_id)*, *idx_stores_active(is_active)*
מפתחות זרים: chain_id → store_chains, format_id → store_formats

6.2.2 טבלאות מוצרים ומחירים

products

קטלוג מוצרים (מנורמל ע"י hebrew_nlp)

עמודה	סוג	הערות
product_id	SERIAL PK	מזהה
barcode	VARCHAR(20) UNIQUE	ברקוד (מפתח טבעי)
product_name	VARCHAR(300)	שם מנורמל
manufacturer	VARCHAR(200)	יצרן (V010)
category_id	INT FK	categories →
is_weighted	BOOLEAN DEFAULT FALSE	מוצר שקיל (ירקות, בשר)
is_private_label	BOOLEAN DEFAULT FALSE	מותג פרטי (V010)
unit	VARCHAR(20)	יחידת מידה (קג, ליטר, יחידה)
avg_pick_sec	INT DEFAULT 28	זמן ליקוט ממוצע (שניות)

אינדקסים: *idx_products_barcode(barcode), idx_products_category(category_id)*

מפתחות זרים: category_id → categories(category_id)

store_prices_latest

טבלת הליבה להשוואה — מחיר עדכני של מוצר בסניף

עמודה	סוג	הערות
store_id	INT PK FK	מפתח מרכב עם product_id
product_id	INT PK FK	מפתח מרכב
price	NUMERIC(10,2)	מחיר מדף (שקלים)
unit_price	NUMERIC(10,2)	מחיר ליחידה (V010)
price_update_date	DATE	תאריך עדכון מהרשת (V010)
updated_at	TIMESTAMPTZ	זמן UPSERT אחרון

אינדקסים: *idx_prices_store(store_id), idx_prices_product(product_id)*

מפתחות זרים: store_id → stores, product_id → products

promotions

מבצעים פעילים — 5 סוגים + Stacking (V006+V007)

עמודה	סוג	הערות
promo_id	BIGSERIAL PK	מזהה
store_id	INT FK	stores →
barcode	VARCHAR(20)	ברקוד מוצר
product_id	INT FK	products → (ממולא אחרי ETL)
promo_type	VARCHAR(20)	PRICE_CUT / PCT_OFF / MULTI_BUY / BUY_GET / OTHER
description	TEXT	תיאור טקסטואלי מה-XML
start_date	DATE	תחילת מבצע
end_date	DATE	סיום מבצע
min_qty	INT DEFAULT 1	כמות מינימום להפעלה (V006)
max_qty	INT	כמות מקסימום בהנחה
discount_pct	NUMERIC(5,2)	אחוז הנחה (PCT_OFF)

discounted_price	NUMERIC(10,2)	מחיר אחרי הנחה (PRICE_CUT/MULTI_BUY)
reward_qty	INT	כמות מוצרים חינם (BUY_GET)
is_member_only	BOOLEAN DEFAULT FALSE	מבצע מועדון בלבד
is_active	BOOLEAN DEFAULT TRUE	מבצע פעיל (V006)
stacking_mode	VARCHAR(20) DEFAULT "EXCLUSIVE	EXCLUSIVE / ADDITIVE (V007)
updated_at	TIMESTAMPTZ	זמן UPSERT

אינדקסים: *idx_promotions_active(store_id, product_id) WHERE is_active = TRUE*
מפתחות זרים: store_id → stores, product_id → products

6.2.3 טבלאות תורים, ביקורים וקופות

queue_reports

דיווחי תורים ממשתמשים

עמודה	סוג	הערות
report_id	SERIAL PK	
store_id	INT FK NOT NULL	stores →
user_id	INT FK	asp_net_users → (NULL = אנונימי)
wait_level	INT NOT NULL	0=אין, 1=קצר, 2=בינוני, 3=ארוך
actual_wait_min	DOUBLE	זמן בפועל (דקות)
source	INT DEFAULT 0	0=Manual, 1=Geofence, 2=Admin
checkout_type	VARCHAR(20)	regular / express / self
reported_at	TIMESTAMPTZ	

אינדקסים: *idx_queue_store_time(store_id, reported_at DESC) CONCURRENTLY (V002)*
מפתחות זרים: store_id → stores, user_id → asp_net_users

store_queue_stats

סטטיסטיקת EWMA ממוזגת לשעה+יום (V003+V005 מיזוג)

עמודה	סוג	הערות
store_id	INT PK FK	מפתח מרכב שלישי
day_of_week	SMALLINT PK	0=ראשון .. 6=שבת
hour_of_day	SMALLINT PK	0..23
avg_wait_minutes	DOUBLE	EWMA ארוך-טווח (α=0.3)
mean_minutes	DOUBLE	EWMA קצר-טווח (α=0.7)
median_minutes	DOUBLE	חציון
report_count	INT	סה"כ דיווחים
recent_report_count	INT	דיווחים ב-7 ימים אחרונים
mad_lower	DOUBLE	MAD גבול תחתון
mad_upper	DOUBLE	MAD גבול עליון

אינדקסים: *PK מרכב: (store_id, day_of_week, hour_of_day)*
מפתחות זרים: store_id → stores

user_store_visits

ביקורים בחנויות (Geofencing) — V008

עמודה	סוג	הערות
visit_id	BIGSERIAL PK	
user_id	INT FK	asp_net_users →
store_id	INT FK NOT NULL	stores →
entered_at	TIMESTAMPTZ NOT NULL	זמן כניסה
exited_at	TIMESTAMPTZ	זמן יציאה (NULL = עדיין בחנות)
cart_mode	BOOLEAN DEFAULT FALSE	טלפון בעגלה
entry_lat/lon	DOUBLE ×2	GPS כניסה
total_minutes	DOUBLE	משך שהייה כולל
heading_to_checkout_at	TIMESTAMPTZ	המשתמש לחץ "הולך לקופה"

אינדקסים: *idx_visits_store, idx_visits_entered_at DESC*
מפתחות זרים: user_id → asp_net_users, store_id → stores

checkout_events

אירועי קופה — ground truth למודלים (V008)

עמודה	סוג	הערות
event_id	BIGSERIAL PK	
visit_id	BIGINT FK	user_store_visits →
store_id	INT FK NOT NULL	stores →
norm_x, norm_y	DOUBLE ×2	מיקום נרמל (0-1) לאשכול K-Means
standing_seconds	INT NOT NULL	שניות עמידה לפני אישור
checkout_type	VARCHAR(20)	regular / express / self
actual_wait_min	DOUBLE	זמן המתנה בפועל
day_of_week	SMALLINT NOT NULL	לחישוב λ
hour_of_day	SMALLINT NOT NULL	לחישוב λ
confirmed_by_user	BOOLEAN DEFAULT TRUE	אישור ידני vs אוטומטי

אינדקסים: *idx_checkout_store, idx_checkout_type(store_id, checkout_type), idx_checkout_location(store_id, norm_x, norm_y), idx_checkout_time(store_id, day_of_week, hour_of_day)*
מפתחות זרים: visit_id → user_store_visits, store_id → stores

checkout_lanes

קופות מזוהות ע"י K-Means + BIC (V008)

עמודה	סוג	הערות
lane_id	SERIAL PK	
store_id	INT FK NOT NULL	stores →
lane_type	VARCHAR(20)	regular / express / self / unknown
centroid_x	DOUBLE NOT NULL	מרכז אשכול X
centroid_y	DOUBLE NOT NULL	מרכז אשכול Y
std_x, std_y	DOUBLE ×2	סטיית תקן לכל ציר
avg_wait_min	DOUBLE	ממוצע המתנה בקופה זו
sample_count	INT NOT NULL	כמות דגימות באשכול
is_active	BOOLEAN DEFAULT	קופה פעילה

	TRUE	

אינדקסים: *idx_lanes_store(store_id)*

מפתחות זרים: store_id → stores

6.2.4 טבלאות ניווט וקטגוריות

zone_catalog

טבלת התייחסות — אזורים לוגיים (V005 — 3NF fix)

עמודה	סוג	הערות
zone_code	VARCHAR(30) PK	DAIRY / PRODUCE / MEAT / CASHIER / ...ENTRANCE
zone_name	VARCHAR(100)	שם תצוגה בעברית
aisle_order	INT	סדר ברירת מחדל לניווט

categories

קטגוריות מוצרים עם FK לאזור (V005)

עמודה	סוג	הערות
category_id	SERIAL PK	
category_name	VARCHAR(100)	מוצרי חלב, ירקות...
zone_id	VARCHAR(30) FK	zone_catalog(zone_code) →

מפתחות זרים: zone_id → zone_catalog(zone_code)

store_zones

מפת אזורים לכל סניף

עמודה	סוג	הערות
store_id	INT PK FK	מפתח מרכב
zone_code	VARCHAR(30) PK	מפתח מרכב
zone_name	VARCHAR(100)	שם תצוגה לסניף
map_x, map_y	DOUBLE ×2	קואורדינטות הצגה במפה
display_order	INT	סדר הצגה

מפתחות זרים: store_id → stores

6.2.5 היסטוריית מיגרציות

מיגרציה	תיאור	שינוי מרכזי
V001	סכמה ראשונית	כל הטבלאות הבסיסיות
V002	אינדקסים מקביליים	אינדקסים CONCURRENTLY ל-queue_reports
V003	טבלת store_queue_state	בסיס ל-EWMA
V004	תיקוני עמודות וטבלאות	הוספת עמודות חסרות
V005	נירמול 3NF + שלמות	7 תיקונים: vehicle_types, zone_catalog, מיזוג טבלאות, FK + UNIQUE
V006	מנוע מבצעים מובנה	16 שדות חדשים + אינדקס חלקי
V007	כפל מבצעים (Stacking)	שדה stacking_mode + אלגוריתם דו-שלבי
V008	מודל סטוכסטי	3 טבלאות: visits, checkout_events, checkout_lanes
V009	סטטוס פריטים	עמודת item_status ל-shopping_list_items
V010	אינטגרציית מזון	שדות יצרן, מותג פרטי, מעקב ETL

סה"כ הסכמה כוללת **15 טבלאות**, **22 אינדקסים**, **16 מפתחות זרים** ו-**10 מיגרציות** מתועדות. הסכמה מנורמלת ל-3NF לאחר 7 תיקוני נירמול ב-V005. בסעיף הבא יוצגו תרשימי רצף לתהליכים המרכזיים.

6.3 תרשימי רצף

תרשימי רצף *(Sequence Diagrams)* מתארים את זרימת ההודעות בין האובייקטים במערכת לאורך ציר הזמן. להלן חמישה תרשימי רצף לתהליכים המרכזיים, מוצגים בפורמט טבלאי.

6.3.1 תרשים רצף — השוואת מחירים וקבלת המלצה (UC-4)

משתתפים: *Client, PredictController, RecommendationService, AppDbContext, PromoEngine, TrafficService, QueueService, CheckoutLaneService*

#	שולח		מקבל	הודעה
1	**Client**	→	**PredictController**	גוף בקשה + POST /api/predict + JWT
2	**PredictController**	→	**Validator**	אימות קלט (PredictRequestValidator)
3	**PredictController**	→	**Recommendation Service**	PredictAsync(userId, request)
4	**Recommendation Service**	→	**AppDbContext**	שליפת רשימת קניות + פריטים
5	**Recommendation Service**	→	**AppDbContext**	שליפת חנויות פעילות בעיר (stores WHERE city)
6	**Recommendation Service**	→	**AppDbContext**	שליפת מחירים לכל מוצרי הרשימה בכל החנויות (bulk)
7	**Recommendation Service**	→	**AppDbContext**	שליפת מבצעים פעילים (bulk)
8	**Recommendation Service**	→	**PromoEngine**	לואה: לכל חנות — ApplyToBasket(items, promos)
8a	**PromoEngine**	→	**PromoEngine**	שלב EXCLUSIVE: בחירת מבצע בלעדי טוב ביותר → מחיר ביניים
8b	**PromoEngine**	→	**PromoEngine**	שלב ADDITIVE: הפעלת מבצעים נוספים על מחיר הביניים
9	**Recommendation Service**	→	**TrafficService**	לואה: לכל חנות — GetDistanceAsync(lat,lon,vehicleType)
9a	**TrafficService**	→	**Google API**	Distance Matrix API (או Haversine fallback)
10	**Recommendation Service**	→	**QueueService**	PredictAsync(storeId) → EWMA כללי
11	**Recommendation Service**	→	**CheckoutLane Service**	EstimateWaitAsync(storeId,day,hour) → M/M/c לכל קופה
12	**Recommendation Service**	→	**Recommendation Service**	חישוב NetCost = Basket + Fuel + Time
13	**Recommendation Service**	→	**Recommendation Service**	מיון + סימונים (IsBestNet, IsCheapest, IsFastest)
14	**PredictController**	→	**Client**	החזרת PredictResponse עם רשימת כרטיסי חנויות

6.3.2 תרשים רצף — דיווח תור ועדכון חיזוי (UC-5)

משתתפים: *Client, QueueController, QueueService, OutlierFilter, StateUpdater, AppDbContext, SignalR Hub*

#	שולח		מקבל	הודעה
1	**Client**	→	**QueueController**	POST /api/queue/report + JWT

2	**QueueController**	→	**QueueService**	ReportAsync(userId, request)
3	**QueueService**	→	**AppDbContext**	שמירת דיווח ב-queue_reports
4	**QueueService**	→	**OutlierFilter**	IsOutlier(storeId, day, hour, waitMin)
4a	**OutlierFilter**	→	**AppDbContext**	שליפת store_queue_stats לשעה+יום
4b	**OutlierFilter**	→	**OutlierFilter**	חישוב Weighted Double MAD: האם הדיווח בגבולות?
5	**QueueService**	→	**StateUpdater**	UpdateEwma(storeId, day, hour, waitMin): אם לא חריג
5a	**StateUpdater**	→	**AppDbContext**	עדכון store_queue_stats: EWMA(α=0.3) + EWMA(α=0.7)
6	**QueueService**	→	**SignalR Hub**	BroadcastQueueUpdate(storeId, prediction)
7	**SignalR Hub**	→	**Client(s)**	עדכון חי לכל המחוברים לאותה חנות
8	**QueueController**	→	**Client**	החזרת 200 OK

6.3.3 תרשים רצף — זיהוי קופה אוטומטי (UC-6)

משתתפים: *Accelerometer, StandingDetector (Mobile), User, CheckoutController (Server), LaneClassifier, CheckoutLaneService, GmmClusterer, AppDbContext*

#	שולח		מקבל	הודעה
1	**Accelerometer**	→	**StandingDetector**	דגימה כל 500ms: (x, y, z)
2	**StandingDetector**	→	**StandingDetector**	חישוב magnitude = √(x²+y²+z²)
3	**StandingDetector**	→	**StandingDetector**	האם |mag - 9.81| < סף? כן → הגדל מונה. לא → אפס
4	**StandingDetector**	→	**StandingDetector**	שליפת P(קופה|שעה,יום) מ-Bayesian Prior
5	**StandingDetector**	→	**StandingDetector**	האם מונה > סף × (Prior - 1)? כן → הצג דיאלוג
6	**StandingDetector**	→	**User**	"נראה שהגעת לקופה — לדווח?"
7	**User**	→	**StandingDetector**	אישור + סוג קופה + זמן המתנה
8	**StandingDetector**	→	**Checkout Controller**	POST /api/checkout/event, normX, normY + סוג, זמן
9	**Checkout Controller**	→	**LaneClassifier**	TryClassifyAsync(storeId, normX, normY)
9a	**LaneClassifier**	→	**AppDbContext**	שליפת checkout_lanes פעילות עם ≤30 דגימות
9b	**LaneClassifier**	→	**LaneClassifier**	חישוב Mahalanobis מכל centroid. אם > 0.5σ → שיוך
10	**Checkout Controller**	→	**AppDbContext**	שמירת CheckoutEvent + סוג קופה (אם זוהה)
11	**Checkout Controller**	→	**CheckoutLane Service**	אם הצטברו 50+ אירועים: RefreshLanesAsync
11a	**CheckoutLane Service**	→	**GmmClusterer**	DetectLanes(points, maxK=12, minPts=5)
11b	**GmmClusterer**	→	**GmmClusterer**	K-Means לכל K=1..12 → BIC → בחירת K אופטימלי
11c	**CheckoutLane Service**	→	**AppDbContext**	עדכון checkout_lanes עם centroids חדשים
12	**Checkout Controller**	→	**Client**	החזרת 201 Created + סוג קופה (אם זוהה)

6.3.4 תרשים רצף — ניווט פנים-חנות (UC-7)

משתתפים: *Client, NavigationController, NavigationPlanService, RouteOptimizer, AppDbContext*

#	שולח		מקבל	הודעה
1	**Client**	→	**Navigation Controller**	POST /api/navigation/plan + storeId, listId
2	**Navigation Controller**	→	**NavigationPlan Service**	BuildPlanAsync(storeId, userId, listId)
3	**NavigationPlan Service**	→	**AppDbContext**	שליפת StoreZones (כולל ENTRANCE + CASHIER)
4	**NavigationPlan Service**	→	**AppDbContext**	שליפת פריטי רשימה + קטגוריות + zones
5	**NavigationPlan Service**	→	**NavigationPlan Service**	מיפוי מוצר → קטגוריה → ZoneCode
6	**NavigationPlan Service**	→	**RouteOptimizer**	Solve(entrance, cashier, requiredZones)
6a	**RouteOptimizer**	→	**RouteOptimizer**	סידור אזורים בסדר יעיל: כניסה → אזורים → קופות
7	**NavigationPlan Service**	→	**NavigationPlan Service**	בניית צעדים: לכל אזור — שם, מוצרים, זמן הליכה
8	**Navigation Controller**	→	**Client**	החזרת NavigationResponse עם רשימת צעדים

6.3.5 תרשים רצף — עדכון מחירים (ETL (UC-8

משתתפים: *GitHub Actions, import_all_chains.py, fetch_urls.py, import_prices.py, import_promos.py, hebrew_nlp.py, PostgreSQL*

#	שולח		מקבל	הודעה
1	**GitHub Actions**	→	**import_all _chains.py**	הפעלה ב-06:00 (nightly-etl.yml)
2	**import_all _chains.py**	→	**import_all _chains.py**	קריאת chains_config.json — רשימת רשתות
3	**import_all _chains.py**	→	**fetch_urls.py**	לואה: לכל רשת — גילוי URL עדכניות
3a	**fetch_urls.py**	→	**אתרי רשתות**	הורדת דף HTML → חילוץ קישורי XML/GZ
4	**import_all _chains.py**	→	**import_prices .py**	import_pricesfull_stream(chain, urls)
4a	**import_prices .py**	→	**import_prices .py**	הורדת GZ → פענוח — XML (iterparse streaming)
4b	**import_prices .py**	→	**hebrew_nlp.py**	נירמול שם מוצר בעברית
4c	**import_prices .py**	→	**PostgreSQL**	UPSERT INTO store_prices_latest
5	**import_all _chains.py**	→	**import_promos .py**	import_promosfull_stream(chain, urls)
5a	**import_promos .py**	→	**import_promos .py**	מיפוי סוג מבצע + detect_stacking_mode()
5b	**import_promos .py**	→	**PostgreSQL**	UPSERT INTO promotions
6	**import_all _chains.py**	→	**PostgreSQL**	עדכון LastEtlSuccessAt + LastEtlItemCount ברשת
7	**import_all _chains.py**	→	**GitHub Actions**	סיכום: כמויות, שגיאות, זמן ריצה

6.4 סיכום פרק 6

פרק זה הציג את עיצוב המערכת בשלושה מימדים: **תרשים מחלקות לעיצוב** עם 11 שירותים ו-10 בקרים, **סכמת בסיס נתונים** עם 15 טבלאות, 22 אינדקסים ו-10 מיגרציות, ו**5 תרשימי רצף** לתהליכים המרכזיים (השוואת מחירים, דיווח תור, זיהוי קופה, ניווט ו-ETL). בפרק הבא יוצג מנשק המערכת — מסכי המשתמש.

פרק 7: מנשק המערכת

פרק זה מציג את מסכי המשתמש של מערכת **SuperWaze** — ממשק הווב (Next.js) וממשק המובייל (React Native / Expo). לכל מסך מוצגים: שם, נתיב, אלמנטים ויסודות עיצוב. כל הממשקים תומכים ב-RTL מלא בעברית.

7.1 עקרונות עיצוב הממשק

1. **Mobile-First:** העיצוב מותאם למובייל תחילה, ומורחב למסך גדול — 91% מהנשאלים בסקר משתמשים בסמארטפון לקניות.
2. **RTL-First:** כל הממשקים מעוצבים מימין לשמאל. בווב — Tailwind rtl: classes; במובייל — I18nManager.forceRTL(true).
3. **מידע מדורג:** המידע החשוב ביותר (עלות נטו) בולט ובראש המסך; פירוטים (מבצעים, תורים לכל קופה) זמינים בלחיצה.
4. **משוב ויזואלי:** צבעים לרמת ביטחון (high/medium/low), סימונים לחנות הטובה ביותר (IsBestNet), אינדיקציה ברורה למקור החיזוי (mmC/ewma/default).
5. **רכיבי UI משותפים:** ספריית רכיבים מרכזית ב-*components/ui/index.tsx* — Button, Toast, AppShell לעקביות.
6. **ניהול מצב ב-Zustand:** כל הנתונים (auth, lists, predict) מנוהלים ב-Zustand stores — קלים ויעילים.

7.2 מסכי המערכת

7.2.1 מסך הרשמה / התחברות

מסך	**LoginPage**
נתיב	login/
מקרה קשור	UC-1, UC-2
תיאור	מסך כניסה למערכת. מאפשר מעבר בין התחברות להרשמה ובחזרה.
אלמנטי ממשק	• לוגו SuperWaze בראש העמוד • שדה email (עם אימות פורמט) • שדה סיסמה (מינימום 8 תווים) • שדה שם תצוגה (בהרשמה בלבד) • כפתור "התחבר" / "הירשם" • קישור מעבר בין מצבים • הודעות שגיאה (Toast)
פעולות	התחברות מוצלחת → שמירת JWT ב-Zustand + Redirect למסך ראשי

7.2.2 מסך ראשי (דף הבית)

מסך	**HomePage**
נתיב	/
מקרה קשור	כל
תיאור	מסך הכניסה המרכזי — מציג רשימות קניות, גישה מהירה להשוואה וסרגל ניווט.
אלמנטי ממשק	• AppShell — הדר, תפריט צדדי וניווט תחתון • רשימת רשימות קניות עם ספירת פריטים • כפתור "רשימה חדשה" • כפתור "השווה מחירים" בולט לכל רשימה •

	הודעות בזמן אמת על שינויים בתורים (SignalR)
פעולות	לחיצה על רשימה → פתיחת עריכה / ניווט למסך השוואה

7.2.3 מסך השוואת מחירים והמלצה

מסך	**PredictPage**
נתיב	predict/
מקרה קשור	UC-4
תיאור	המסך המרכזי במערכת — מציג את תוצאות ההשוואה כרשימת כרטיסי חנויות ממוינים לפי עלות נטו. זהו המסך הגדול ביותר (13,262 שורות קוד).
אלמנטי ממשק	• סרגל פילטרים: תרחיש זמן (עכשיו/+60/+120), מצב מיון (נטו/מחיר/זמן), עיר • כרטיס חנות (לכל חנות): — שם רשת + סניף + מרחק — עלות נטו בולטת + פירוט: סל, דלק, זמן — חיסכון ממבצעים (savings) — זמן תור כללי + פירוט לכל סוג קופה (regular/express/self) — רמת ביטחון + מקור (mmC/ewma/default) — סימונים: IsBestNet (ירוק), IsCheapest (כחול), IsFastest (אדום) — כפתור "נווט אותי" → מסך ניווט • מונה כיסוי: מוצרים חסרים + כיסוי • פירוט מבצעים בהרחבה (לחיצה על כרטיס)
פעולות	שינוי פילטר → קריאה חוזרת ל-API; לחיצה על כרטיס → פירוט מבצעים; SignalR מעדכן תורים בזמן אמת

7.2.4 מסך דיווח תור

מסך	**QueueReportDialog**
נתיב	דיאלוג (לא נתיב נפרד)
מקרה קשור	UC-5
תיאור	דיאלוג לדיווח תור — נפתח מתוך מסך ההשוואה או מהאפליקציה.
אלמנטי ממשק	• בחירת חנות (dropdown) • 4 כפתורי רמה: אין תור / קצר / בינוני / ארוך (עם אייקונים) • שדה זמן המתנה בפועל (אופציונלי, בדקות) • כפתור "שלח דיווח" • הודעת אישור: "תודה! הדיווח נשמר"
פעולות	אישור POST /api/queue/report → Toast → שליחה

7.2.5 מסך ניווט פנים-חנות

מסך	**NavigationPage**
נתיב	navigation/
מקרה קשור	UC-7
תיאור	מציג מסלול ניווט מעורב עם צעדים — מכניסה עד קופות.
אלמנטי ממשק	• רשימת צעדים ממוספרים: — מספר צעד (סטפר) — שם אזור (בולט) — רשימת מוצרים לאסוף באזור זה — זמן הליכה משוער • סיכום: זמן כולל משוער + סה"כ מוצרים • צעד נוכחי מודגש (active step)
פעולות	לחיצה על מוצר → סימון checked; לחיצה על צעד → מעבר לבא

7.2.6 מסך פרופיל משתמש

מסך	**ProfilePage**

נתיב	profile/
מקרה קשור	—
תיאור	הגדרות אישיות + התנתקות.
אלמנטי ממשק	• שם תצוגה (ניתן לעריכה) • עיר מועדפת (dropdown ערים) • סוג רכב (4 כפתורים: בנזין / חשמלי / היברידי / הולך רגל) • חברות מועדון (טוגל לכל רשת) • כפתור "שמור שינויים" • כפתור "התנתקות"
פעולות	אישור PUT /api/profile → Toast → שינוי

7.2.7 מסך רשימת קניות (מובייל)

מסך	**ShoppingListScreen**
נתיב	ShoppingListScreen.tsx (Expo)
מקרה קשור	UC-3
תיאור	מסך ניהול רשימת קניות באפליקציית המובייל (15,588 שורות קוד).
אלמנטי ממשק	• שדה חיפוש מוצרים (עם autocomplete) • רשימת פריטים עם: — שם מוצר — כמות (+/- כפתורים) — סטטוס: נצבע בצבע (pending=לבן, checked=ירוק, not_found=אדום) — החלקה למחיקה (swipe) • ספירת פריטים לפי סטטוס • כפתור "השווה מחירים" בתחתית
פעולות	לחיצה על פריט → מעבר סטטוס; swipe → מחיקה; + → הוספה

7.2.8 מסך זיהוי קופה (מובייל)

מסך	**CheckoutDetectionDialog**
נתיב	דיאלוג (מפעיל StandingDetector)
מקרה קשור	UC-6
תיאור	דיאלוג קופה — נפתח אוטומטית כשה-StandingDetector מזהה עמידה ממושכת.
אלמנטי ממשק	• כותרת: "נראה שהגעת לקופה!" • בחירת סוג קופה (3 כפתורים: רגילה / מהירה / שירות עצמי) • שדה זמן המתנה (דקות, ממולא אוטומטית) • כפתור "אישור" + כפתור "לא, לא בקופה"
פעולות	אישור → POST /api/checkout/event; דחייה → סגירה

7.3 מפת ניווט המערכת

להלן מפת הניווט בין מסכי המערכת:

1. מסך התחברות (/login) → **מסך ראשי** (/)
2. מסך ראשי → **מסך השוואה** (/predict) — דרך "השווה" ליד רשימה
3. מסך השוואה → **מסך ניווט** (/navigation) — דרך כפתור "נווט אותי"
4. מסך ראשי → **מסך פרופיל** (/profile) — דרך תפריט צדדי
5. כל מסך → **דיאלוג דיווח תור** — דרך FAB (כפתור צף) או אוטומטי (Geofencing)
6. בחנות (מובייל) → **דיאלוג זיהוי קופה** — אוטומטי (StandingDetector)

7.4 סיכום

מנשק המערכת כולל **8 מסכים** (6 בווב + 2 במובייל), כולם תומכים ב-RTL מלא בעברית. המסך המרכזי — מסך ההשוואה (*PredictPage*) — הוא הגדול והמורכב ביותר, ומציג לכל חנות כרטיס עם: עלות נטו, פירוט מבצעים, חיזוי תורים לכל סוג קופה, רמת ביטחון וסימונים. הממשקים הייחודיים למובייל (רשימת קניות וזיהוי קופה) מנצלים את יכולות החיישנים (Accelerometer, Geofencing) שאינן זמינים בדפדפן. בפרק הבא יוצגו סיכום הפרויקט וביבליוגרפיה.

פרק 8: סיכום

8.1 סיכום הפרויקט

פרויקט זה הציג את תכנון, פיתוח ומימוש של **SuperWaze** — מערכת השוואת מחירים חכמה עם חיזוי תורים לסופרמרקטים בישראל. המערכת פותחה בעיות מרכזיות של צרכני המזון הישראלי — יוקר המחיה, פיזור המידע על מבצעים, חוסר מידע על תורים ועומסים, והיעדר התחשבות בעלות הנסיעה וזמן השהייה בהשוואה.

המערכת פותחה את האתגרים הבאים:

1. **השוואת מחירים מקיפה:** השוואה בין כל רשתות השיווק הגדולות על בסיס נתוני חוק שקיפות המחירים, עם עדכון יומי אוטומטי באמצעות צינורות ETL.
2. **מנוע מבצעים מתקדם:** PromoEngine התומך ב-5 סוגי מבצע עם מנגנון כפל הנחות (Stacking) — יכולת שלא קיימת באף מערכת אחרת בישראל.
3. **חיזוי תורים דו-שכבתי:** שכבה קהילתית (EWMA) לחיזוי כללי, ושכבה מתמטית (M/M/c) לחיזוי מפורט לכל סוג קופה — עם מנגנון fallback היררכי.
4. **זיהוי קופות אוטומטי:** שילוב חדשני של חיישן תאוצה (StandingDetector) עם Bayesian Prior ואשכול K-Means + BIC — זיהוי קופות פיזיות ללא תשתית חיצונית.
5. **אופטימיזציית עלות נטו:** שקלול עלות סל (לאחר מבצעים), עלות נסיעה (לפי סוג רכב) ועלות זמן (תורים + זמן בחנות) לעלות נטו אחת — גישה שאינה קיימת בשום מערכת אחרת.
6. **ניווט פנים-חנות:** ניווט מבוסס אזורים עם אופטימיזציית מסלול — מפחית 15%-25% מזמן השהייה בחנות.
7. **ארכיטקטורה מודרנית:** מונוליט מודולרי, REST API, SignalR, DI, CI/CD, Docker Compose — תשתית מקצועית המוכנה לסקלביליות.

8.2 תרומות הפרויקט

הפרויקט תורם במספר מימדים:

1. **תרומה טכנולוגית:** הוכח שניתן לזהות קופות פיזיות בסופרמרקטים ללא תשתית חיצונית, באמצעות שילוב חיישן תאוצה עם אשכול נתונים.
2. **תרומה מתודולוגית:** שילוב מודלים סטטיסטיים (EWMA, MAD) עם מודלים מתמטיים (M/M/c) ומודלים מלמידת מכונה (K-Means, Bayesian) במערכת אחת.
3. **תרומה חברתית:** הנגשת נתוני חוק שקיפות המחירים לצרכן בממשק אינטואיטיבי ואקטיבי — פותרת את "אסימטריית הנגישות" שחוק השקיפות יצר.

8.3 מגבלות

1. **מסה קריטית:** דיוק חיזוי התורים וזיהוי הקופות תלויים בקהילת משתמשים פעילים — טרם נבדקו בסביבת מפתחים בלבד.

2. **דיוק GPS במבנים:** מיקום נרמל (normX, normY) מבוסס על GPS גס בתוך מבנה — דיוק מוגבל.

3. **תלות בחוק השקיפות:** שינוי או ביטול של חוק שקיפות המחירים ישפיע ישירות על זמינות הנתונים.

4. **לא נבדקה בסביבת ייצור אמיתית:** המערכת פותחה בסביבת פיתוח — נדרשת בדיקה עם משתמשים אמיתיים לאורך זמן.

8.4 כיווני פיתוח עתידיים

1. **למידת מכונה לחיזוי תורים:** החלפת EWMA במודל LSTM או Transformer שילמד דפוסי עומסים מורכבים — עונתיות, חגים, מזג אוויר.

2. **ניווט פנים-חנות משופר:** אינטגרציה עם Bluetooth Beacons למיקום מדויק בין המעברים, וניווט בזמן אמת על המפה.

3. **המלצות מותאמות אישית:** למידה מהעדפות רכישה והיסטוריית קנייה להתאמה אישית של ההמלצות.

4. **גיימיפיקציה:** הוספת נקודות, דירוגים ואתגרים לעידוד דיווחי תורים — פתרון לבעיית מסה קריטית.

5. **הרחבה לשווקים נוספים:** בתי מרקחת, חנויות חומרי בניין, חנויות חיות מחמד — בכל מקום שיש תורים ושקיפות מחירים.

6. **מודל עסקי:** יישום מודל Freemium (חיזוי מתקדם בתשלום) או DaaS (תובנות לרשתות).

8.6 דברי סיום

SuperWaze מדגימה כיצד שילוב של טכנולוגיות מגוונות — צינורות נתונים, מודלים סטטיסטיים, תורת התורים, למידת מכונה, חיישנים ונתוני קהילה — יכול לייצר ערך ממשי לצרכן. הפרויקט הוכיח שהבעיה אינה רק "מה המחיר?" אלא "איפה כדאי לקנות?" — והתשובה חייבת להביא בחשבון את כל העלויות: מחיר המוצרים, מבצעים, נסיעה, זמן בחנות וזמן בתור.

סה"כ הבדיקות מכסות: PromoEngine, QueueModel (EWMA + MAD + M/M/c + Fallback), CheckoutLaneService (Classifier + Selector + Stochastic), BasketService, RouteOptimizer ו-ShoppingListController.

13. כל הבדיקות רצות אוטומטית ב-CI (GitHub Actions) בכל push ל-main.

12. ShoppingListControllerTests — בדיקת CRUD רשימות קניות + הרשאות.

11. RouteOptimizerTests — אימות סידור אזורים: כניסה → אזורים → קופות.

10. BasketServiceTests — בדיקת חישוב עלות סל עם מבצעים ומטמון.

9. LaneSelectorTests — בדיקת 4 כללי עדיפות בבחירת קופה.

8. LaneClassifierTests — בדיקת שיוך קופה: נקודה קרובה ל-centroid מזוהה, רחוקה נדחית.

7. StochasticModelTests — אימות חישובי ρ, Wq, P_0: M/M/c, יציבות.

6. QueueServiceBulkTests — בדיקת עדכון מרובה דיווחים בו-זמנית.

5. QueueFallbackTests — אימות מנגנון Fallback: M/M/c → EWMA → default.

4. QueueMathTests — אימות פונקציות Median, MAD, P80, EWMA.

3. QueueOutlierFilterTests — בדיקת Weighted Double MAD: דיווחים תקינים עוברים, חריגים נפסלים.

2. QueueEwmaTests — אימות חישובי EWMA עם שני ערכי α (0.3 ו-0.7).

1. PromoEngineTests — בדיקת 5 סוגי מבצע, Stacking, ומקרי קצה (כמות 0, מבצע פג תוקף).

הפרויקט כולל 13 קבצי בדיקות יחידה (Unit Tests) בפריימוורק xUnit, המכסים את כל הלוגיקה העסקית המרכזית:

8.5 בדיקות (Unit Tests)

ביבליוגרפיה

Akerlof, G. A. (1970). The Market for 'Lemons': Quality Uncertainty and the Market Mechanism. Quarterly Journal of Economics, 84(3), 488–500.

Anderson, D. J. (2010). Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press.

Bass, L., Clements, P., & Kazman, R. (2012). Software Architecture in Practice (3rd ed.). Addison-Wesley.

Baye, M. R., Morgan, J., & Scholten, P. (2006). Information, Search, and Price Dispersion. Handbook on Economics and Information Systems, 1, 323–375.

Beck, K., et al. (2001). Manifesto for Agile Software Development. agilemanifesto.org.

Brabham, D. C. (2013). Crowdsourcing. MIT Press.

Brynjolfsson, E., & Smith, M. D. (2000). Frictionless Commerce? A Comparison of Internet and Conventional Retailers. Management Science, 46(4), 563–585.

Davis, F. D. (1989). Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology. MIS Quarterly, 13(3), 319–340.

Dix, A., Finlay, J., Abowd, G. D., & Beale, R. (2004). Human-Computer Interaction (3rd ed.). Pearson.

Ellickson, P. B. (2007). Does Sutton Apply to Supermarkets? RAND Journal of Economics, 38(1), 43–59.

Erlang, A. K. (1909). The Theory of Probabilities and Telephone Conversations. Nyt Tidsskrift for Matematik, 20, 33–39.

Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. RFC 6455, IETF.

Fielding, R. T. (2000). Architectural Styles and the Design of Network-Based Software Architectures. Doctoral dissertation, UC Irvine.

Grewal, D., Roggeveen, A. L., & Nordfält, J. (2017). The Future of Retailing. Journal of Retailing, 93(1), 1–6.

Gross, D., Shortle, J. F., Thompson, J. M., & Harris, C. M. (2008). Fundamentals of Queueing Theory (4th ed.). Wiley.

Hofmann-Wellenhof, B., Lichtenegger, H., & Wasle, E. (2007). GNSS: Global Navigation Satellite Systems. Springer.

Holt, C. C. (1957). Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages. ONR Memorandum, 52.

Hoober, S., & Berkman, E. (2011). Designing Mobile Interfaces. O'Reilly Media.

Howe, J. (2006). The Rise of Crowdsourcing. Wired Magazine, 14(6), 1–4.

Hui, S. K., Bradlow, E. T., & Fader, P. S. (2009). Testing Behavioral Hypotheses Using an Integrated Model of Grocery Store Shopping Path and Purchase Behavior. Journal of Consumer Research, 36(3), 478–497.

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts.

Janssen, M., Charalabidis, Y., & Zuiderwijk, A. (2012). Benefits, Adoption Barriers and Myths of Open Data and Open Government. Information Systems Management, 29(4), 258–268.

Junglas, I. A., & Watson, R. T. (2008). Location-Based Services. Communications of the ACM, 51(3), 65–69.

Kim, G., Debois, P., Willis, J., & Humble, J. (2016). The DevOps Handbook. IT Revolution Press.

Kimball, R., & Caserta, J. (2004). The Data Warehouse ETL Toolkit. Wiley.

Kittur, A., et al. (2013). The Future of Crowd Work. Proceedings of CSCW, 1301–1318.

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

Kwapisz, J. R., Weiss, G. M., & Moore, S. A. (2011). Activity Recognition Using Cell Phone Accelerometers. ACM SIGKDD Explorations, 12(2), 74–82.

Laudon, K. C., & Laudon, J. P. (2020). Management Information Systems (16th ed.). Pearson.

Leys, C., Ley, C., Klein, O., Bernard, P., & Licata, L. (2013). Detecting Outliers: Do Not Use Standard Deviation Around the Mean, Use Absolute Deviation Around the Median. Journal of Experimental Social Psychology, 49(4), 764–766.

Liu, H., Darabi, H., Banerjee, P., & Liu, J. (2007). Survey of Wireless Indoor Positioning Techniques and Systems. IEEE Transactions on Systems, Man, and Cybernetics, 37(6), 1067–1080.

Martin, R. C. (2017). Clean Architecture. Prentice Hall.

Murphy, K. P. (2012). Machine Learning: A Probabilistic Perspective. MIT Press.

Namiot, D., & Sneps-Sneppe, M. (2013). Geofence and Network Proximity. Internet of Things, Smart Spaces, and Next Generation Networking, 117–127.

Newman, S. (2021). Building Microservices (2nd ed.). O'Reilly Media.

Nielsen, J. (1994). 10 Usability Heuristics for User Interface Design. Nielsen Norman Group.

Norman, D. (2013). The Design of Everyday Things (Revised ed.). Basic Books.

O'Brien, J. A., & Marakas, G. M. (2017). Management Information Systems (10th ed.). McGraw-Hill.

Porter, M. E., & Millar, V. E. (1985). How Information Gives You Competitive Advantage. Harvard Business Review, 63(4), 149–160.

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

Rahm, E., & Do, H. H. (2000). Data Cleaning: Problems and Current Approaches. IEEE Data Engineering Bulletin, 23(4), 3–13.

Richards, M. (2015). Software Architecture Patterns. O'Reilly Media.

Roberts, S. W. (1959). Control Chart Tests Based on Geometric Moving Averages. Technometrics, 1(3), 239–250.

Royce, W. W. (1970). Managing the Development of Large Software Systems. IEEE WESCON, 1–9.

Schiller, J., & Voisard, A. (2004). Location-Based Services. Elsevier.

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide.

Silva, T. H., Vaz de Melo, P. O. S., Almeida, J. M., & Loureiro, A. A. F. (2013). Large-Scale Study of City Dynamics and Urban Social Behavior Using Participatory Sensing. IEEE Wireless Communications, 21(1), 42–51.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Stigler, G. J. (1961). The Economics of Information. Journal of Political Economy, 69(3), 213–225.

Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems (3rd ed.). Pearson.

Tsarfaty, R. (2006). Integrated Morphological and Syntactic Disambiguation for Modern Hebrew. Proceedings of COLING/ACL Student Research Workshop.

Vassiliadis, P. (2009). A Survey of Extract-Transform-Load Technology. International Journal of Data Warehousing and Mining, 5(3), 1–27.

Venkatesh, V., & Davis, F. D. (2000). A Theoretical Extension of the Technology Acceptance Model. Management Science, 46(2), 186–204.

Wang, R. Y., & Strong, D. M. (1996). Beyond Accuracy: What Data Quality Means to Data Consumers. Journal of Management Information Systems, 12(4), 5–33.

Whitt, W. (1999). Predicting Queueing Delays. Management Science, 45(6), 870–888.